

GNU Emacs NEWS

History of User-Visible Changes

Covering versions: 31.0.91, 31.0.90

Emacs 31.0.91 – NEWS

GNU Emacs NEWS -- history of user-visible changes.

Copyright (C) 2022-2026 Free Software Foundation, Inc. See the end of the file for license conditions.

Please send Emacs bug reports to `'bug-gnu-emacs@gnu.org'`. If possible, use `'M-x report-emacs-bug'`.

This file is about changes in Emacs version 31.

See file HISTORY for a list of GNU Emacs versions and release dates. See files NEWS.30, NEWS.29, ..., NEWS.18, and NEWS.1-17 for changes in older Emacs versions.

You can narrow news to a specific version by calling `'view-emacs-news'` with a prefix argument or by typing `'C-u C-h C-n'`.

Temporary note: +++ indicates that all relevant manuals in doc/ have been updated. --- means no change in the manuals is needed. When you add a new item, use the appropriate mark if you are sure it applies, and please also update docstrings as needed.

Installation Changes in Emacs 31.1

Unexec dumper removed.

The traditional unexec dumper, deprecated since Emacs 27, has been removed.

The portable dumper now works on m68k a.out targets.

Emacs's old 'ctags' program is no longer built or installed.

You are encouraged to use Universal Ctags <<https://ctags.io/>> instead. For now, to get the old 'ctags' behavior you can run `'etags --ctags'` or use a shell script named `'ctags'` that runs `'etags --ctags "$@"'`.

Changed GCC default options on 32-bit x86 systems.

When using GCC 4 or later to build Emacs on 32-bit x86 systems, `'configure'` now defaults to using the GCC options `'-mfpmath=sse'` (if the host system supports SSE2) or `'-fno-tree-sra'` (if not). These GCC options work around GCC bug 58416, which can cause Emacs to behave incorrectly in rare cases.

New configure option '--with-systemduserunitdir'.

This allows to specify the directory where the user unit file for systemd is installed; default is `'${prefix}/usr/lib/systemd/user'`.

Startup Changes in Emacs 31.1

In compatible terminals, 'xterm-mouse-mode' is turned on by default.

For these terminals the mouse will work by default. A compatible terminal is one that supports Emacs setting and getting the OS selection data (a.k.a. the clipboard) and mouse button and motion events. With `'xterm-mouse-mode'` enabled, you must use Emacs keybindings to copy to the OS selection instead of terminal-specific keybindings.

You can keep the old behavior by putting `'(xterm-mouse-mode -1)'` in your init file.

site-start.el is now loaded before the user's early init file.

Previously, the order was `early-init.el`, `site-start.el` and then the user's regular init file, but now `site-start.el` comes first. This allows site administrators to customize things that can normally only be done from `early-init.el`, such as adding to `'package-directory-list'`.

Emacs prepares a User Lisp directory by default.

If you have a subdirectory `"user-lisp/"` in your Emacs configuration directory, then Lisp files in it and its subdirectories will now be recursively byte-compiled, scraped for autoload cookies and ensured to be in `'load-path'` by default. You can disable the feature by setting `'user-lisp-auto-scrape'` to `nil`, or set the `'user-lisp-directory'` user option to process any other directory on your system. You can also invoke the `'prepare-user-lisp'` command manually at any time. See the Info node `"(emacs) User Lisp Directory"` for more details.

Changes in Emacs 31.1

'line-spacing' now supports specifying spacing above the line.

Previously, only spacing below the line could be specified. The user option can now be set to a cons cell to specify spacing both above and below the line, which allows for vertically centering text.

New face 'margin' for the window margin display.

A new basic face `'margin'` is used by default for text displayed in the left and right margin areas, which are used by various packages for per-line annotations. Its background defaults to the frame default, so existing behavior is unchanged for users who do not customize this new face.

Display strings shown in the margins now inherit the unspecified face attributes from the `'margin'` face, if the string itself does not fully specify its face. If your code relied on the face of the underlying buffer text to serve as default for the unspecified face attributes of strings displayed in the margin, you must now set those face attributes in the margin string itself via `'propertize'`.

'prettify-symbols-mode' attempts to ignore undisplayable characters.

Previously, such characters would be rendered as, e.g., white boxes.

'standard-display-table' now has more extra slots.

`'standard-display-table'` has been extended to allow specifying glyphs that are used for borders around child frames and menu separators on TTY frames.

Call the command `'standard-display-unicode-special-glyphs'` to set up the `'standard-display-table'`'s extra slots with Unicode characters. Please see the documentation of that command to see which slots of the display table it changes.

Child frames are now supported on TTY frames.

This supports use-cases like Posframe, Corfu, and child frames acting like tooltips.

To enable tooltips on TTY frames, call `'tty-tip-mode'`.

The presence of child frame support on TTY frames can be checked with `'(featurep 'tty-child-frames)'`.

Recent versions of Posframe and Corfu are known to use child frames on TTYs if they are supported.

Several font-lock face variables are now obsolete.

The following variables are now obsolete: `'font-lock-builtin-face'`, `'font-lock-comment-delimiter-face'`, `'font-lock-comment-face'`, `'font-lock-constant-face'`, `'font-lock-doc-face'`, `'font-lock-doc-markup-face'`, `'font-lock-function-name-face'`, `'font-lock-keyword-face'`, `'font-lock-negation-char-face'`, `'font-lock-preprocessor-face'`, `'font-lock-string-face'`, `'font-lock-type-face'`, `'font-lock-variable-name-face'`, and `'font-lock-warning-face'`.

These variables contributed both to confusion about the relation between faces and variables, and to inconsistency when major mode authors used one or the other (sometimes interchangeably). We always recommended using faces directly, and not creating variables going by the same name.

If you have customized these variables, you should now customize the corresponding faces instead, using something like:

```
M-x customize-face RET font-lock-string-face RET
```

If you have been using these variables in Lisp code (for example, in font-lock rules), simply quote the symbol, to use the face directly instead of its now-obsolete variable.

New char-table 'special-mirror-table' for mirroring special glyphs.

This char-table is used to mirror special glyphs (truncation and continuation) when the user has defined an alternative representation for those characters via display tables.

find-func.el commands now have history enabled.

The 'find-function', 'find-library', 'find-face-definition', and 'find-variable' commands now allow retrieving previous input using the usual minibuffer history commands. Each command has a separate history.

New minor mode 'find-function-mode' replaces 'find-function-setup-keys'.

The new minor mode defines the keys at a higher precedence level than the old function, one more usual for a minor mode. To restore the old behavior, customize 'find-function-mode-lower-precedence' to non-nil.

'find-function' can now find 'cl-defmethod' invocations inside macros.

New minor mode 'prettify-special-glyphs-mode'.

The new minor mode prettifies the special character glyphs (truncation and continuation) on TTY frames (and GUI frames without fringes). You can customize the associated new face 'special-glyphs'.

Minibuffer and Completions

*Support for immediate display of the "*Completions*" buffer.*

Whenever a minibuffer with completion is opened, the "*Completions*" buffer will now be displayed immediately if the completion property 'eager-display', set by the completion table, is non-nil. This property can be overridden for different completion categories by customizing 'completion-category-overrides'. Alternatively, the new user option 'completion-eager-display' can be set to t to force eager display of "*Completions*" for all minibuffers, or nil to suppress this for all minibuffers.

Support for completion category inheritance.

You can now define completion categories that inherit properties from existing categories, using the new function 'define-completion-category'.

*Support for updating "*Completions*" as you type.*

If the "*Completions*" buffer is displayed, it will now be updated as you type if the completion property 'eager-update', set by the completion table, is non-nil. This property can be overridden for different completion categories by customizing 'completion-category-overrides'. Alternatively, the new user option 'completion-eager-update' can be set to t to make "*Completions*" always be updated as

you type, or nil to suppress this always. Note that for large or inefficient completion tables this can slow down typing.

New optional value of 'minibuffer-visible-completions'.

If the value of this option is 'up-down', only the '<UP>' and '<DOWN>' arrow keys move point between candidates shown in the "*Completions*" buffer display, while '<RIGHT>' and '<LEFT>' arrows move point in the minibuffer window.

'RET' chooses the completion selected with 'M-<UP>/M-<DOWN>'.

If a completion candidate is selected with 'M-<UP>' or 'M-<DOWN>', hitting 'RET' will exit completion with that as the result. This works both in minibuffer completion and in-buffer completion. This supersedes 'minibuffer-completion-auto-choose', which previously provided similar behavior; that variable is now nil by default.

New user option 'completion-pcm-leading-wildcard'.

This option configures how the partial-completion style does completion. It defaults to nil, which preserves the existing behavior. When it is set to t, the partial-completion style behaves more like the substring style, in that a string being completed can match against a candidate anywhere in the candidate string.

'completion-styles' now can contain lists of bindings.

In addition to being a symbol naming a completion style, an element of 'completion-styles' can now be a list of the form '(STYLE ((VARIABLE VALUE) ...))' where STYLE is a symbol naming a completion style. VARIABLE will be bound to VALUE (without evaluating it) while the style is executing. This allows multiple

references to the same style with different values for completion-affecting variables like 'completion-pcm-leading-wildcard' or 'completion-ignore-case'. This also applies for the styles configuration in 'completion-category-overrides' and 'completion-category-defaults'.

Navigating "*Completions*" now accommodates 'completions-format'.

When 'completions-format' is set to 'vertical', typing 'n', 'TAB' or 'M-<DOWN>' in the "*Completions*" buffer (the latter also in the minibuffer) now moves point to the completion candidate in the next line in the current column, and wraps to the next column when typed on the last completion candidate of the current column. Likewise, typing 'p', 'S-TAB' or 'M-<UP>' moves point to the completion candidate in the previous line or wraps to the previous column. Previously, these keys ignored the vertical format, i.e., moved point only to the item in the same line of the next or previous column, in accordance with the default horizontal format. In vertical format, typing '<LEFT>' and '<RIGHT>' in the "*Completions*" buffer (and when 'minibuffer-visible-completions' is non-nil, also in the minibuffer) moves point only within the current line, analogously to how, in horizontal format, '<DOWN>' and '<UP>' move point only within the current column.

Selected completion candidate is preserved across `*Completions*` updates.

When point is on a completion candidate in the `*Completions*` buffer (because of `'minibuffer-next-completion'` or for any other reason), point will still be on that candidate after `*Completions*` is updated with a new list of completions. The candidate is automatically deselected when the `*Completions*` buffer is hidden.

`*Completions*` is now displayed faster when there are many candidates.

As before, if there are more completion candidates than can be displayed in the current frame, only a subset of the candidates is displayed. This process is now faster: only that subset of the candidates is actually inserted into `*Completions*` until you run a command which interacts with the text of the `*Completions*` buffer. This optimization only applies when `'completions-format'` is `'horizontal'` or `'one-column'`.

New user option `'crm-prompt'` for `'completing-read-multiple'`.

This option configures the prompt format of `'completing-read-multiple'`. By default, the prompt indicates to the user that the completion command accepts a comma-separated list. The prompt format can include the separator description and the separator string, which are both stored as text properties of the `'crm-separator'` regular expression.

New user option `'completion-preview-sort-function'`.

This option controls how Completion Preview mode sorts completion candidates. If you use this mode together with an in-buffer completion popup interface, such as the interfaces that the GNU ELPA packages Corfu and Company provide, you can set this option to the same sort function that your popup interface uses for a more integrated experience.

(`'completion-preview-sort-function'` was already present in Emacs 30.1, but as a plain Lisp variable, not a user option.)

New user option `'completion-preview-inhibit-functions'`.

This option provides fine-grained control over Completion Preview mode activation. You can use it to specify arbitrary conditions in which to inhibit the mode's operation.

New mode `'minibuffer-nonselected-mode'`.

This mode, enabled by default, directs the attention to the active minibuffer window using the `'minibuffer-nonselected'` face in case when the minibuffer window is no longer selected, but the minibuffer is still waiting for input.

`'read-multiple-choice'` now uses the minibuffer to read a character.

It still can use `'read-key'` when the variable `'read-char-choice-use-read-key'` is non-nil.

`'map-y-or-n-p'` now uses the minibuffer to read a character.

It still can use `'read-key'` when the variable `'y-or-n-p-use-read-key'` is non-nil.

'flex' completion style rewritten to be faster and more accurate.

Completion and highlighting use a new superior algorithm. For example, pattern "scope" now ranks 'elisp-scope-*' functions well above 'dos-codepage' and 'test-completion'. Pattern "botwin" finds 'menu-bar-bottom-window-divider' before 'ibuffer-other-window'.

Mouse

New mode 'mouse-shift-adjust-mode' extends selection with 'S-<mouse-1>'.

When enabled, you can use the left mouse button with the '<Shift>' modifier to extend the boundaries of the active region by dragging the mouse pointer.

'context-menu-mode' now includes a "Send to..." menu item.

The menu item enables sending current file(s) or region text to external (non-Emacs) applications or services. See send-to.el for customizations.

The mouse now drags lines in character increments again.

Dragging a horizontal or vertical line like the mode line or the lines dividing side-by-side windows now by default happens in increments of the corresponding frame's character size again. This is the behavior described in the manual and was the default behavior before 'window-resize-pixelwise' was added for Emacs 24.1. To drag in pixel increments as with Emacs 24 through Emacs 30 you now have to set 'window-resize-pixelwise' to t.

Windows

New commands to modify window layouts of frames.

'window-layout-rotate-clockwise' ('C-x w r <RIGHT>') and its counterpart 'window-layout-rotate-anticlockwise' ('C-x w r <LEFT>') rotate an entire window layout. 'window-layout-flip-topdown' ('C-x w f <DOWN>', 'C-x w f <UP>') and 'window-layout-flip-leftright' ('C-x w f <LEFT>', 'C-x w f <RIGHT>') flip the window layout vertically and horizontally. 'window-layout-transpose'

('C-x w t') reorganizes windows such that every horizontal split becomes a vertical split and vice versa. 'rotate-windows' ('C-x w o <RIGHT>') and its counterpart 'rotate-windows-back' ('C-x w o <LEFT>') rotate windows in cyclic ordering.

New user option 'rotate-windows-change-selected'.

This controls whether 'rotate-windows' and 'rotate-windows-back' change the selected window. If nil, the selected window does not change. The default is t, which means the new selected window will be the one that winds up at the location of the previously-selected window.

Windmove commands now move to skipped windows if invoked twice in a row.

The new user option 'windmove-allow-repeated-command-override' controls this behavior: if it is non-nil, invoking the same windmove command twice overrides the

'no-other-window' property, allowing navigation to windows that would normally be skipped. The default is t; customize it to nil if you want the old behavior.

New hook 'window-deletable-functions'.

This abnormal hook gives its client a way to save a window from getting deleted implicitly by functions like 'kill-buffer', 'bury-buffer' and 'quit-restore-window'.

Buffer-local window change functions run in their buffers now.

Running the buffer-local version of each of the abnormal hooks 'window-buffer-change-functions', 'window-size-change-functions', 'window-selection-change-functions' and 'window-state-change-functions' will make the respective buffer temporarily current for running the hook.

'window-buffer-change-functions' is run for removed buffers too.

The buffer-local version of 'window-buffer-change-functions' may be run twice now: Once for the buffer removed from the respective window and once for the buffer now shown in that window.

New user option 'quit-window-kill-buffer'.

This option specifies whether 'quit-window' should preferably kill or bury the buffer shown by the window to quit. The default is nil. Customize it to t to always kill the buffer; customize to a list of major modes to kill if the buffer's major mode is one of those.

New user option 'kill-buffer-quit-windows'.

This option has 'kill-buffer' call 'quit-restore-window' to handle the further destiny of any window showing the buffer to be killed.

'split-window' can optionally resurrect deleted windows.

A new optional argument REFER of 'split-window' makes it possible to, instead of making a new window object, reuse an existing, deleted one. This can be used to preserve the identity of windows when swapping or transposing them.

New window parameter 'quit-restore-prev'.

This parameter is set up by 'display-buffer' when it detects that the window used already has a 'quit-restore' parameter. Its presence gives 'quit-restore-window' a way to undo a sequence of buffer display operations more intuitively.

'quit-restore-window' handles new values for BURY-OR-KILL argument.

The values 'killing' and 'burying' are like 'kill' and 'bury' but assume that the actual killing or burying of the buffer is done by the caller.

New user option 'quit-restore-window-no-switch'.

With this option set, 'quit-restore-window' will delete its window more aggressively rather than switching to some other buffer in it.

The user option 'display-comint-buffer-action' has been removed.

It has been obsolete since Emacs 30.1. Use '(category . comint)' instead. Another user option 'display-tex-shell-buffer-action' has been removed too for which you can use '(category . tex-shell)'.

New user option 'split-window-preferred-direction'.

Functions called by 'display-buffer' split the selected window when they need to create a new window. A window can be split either vertically, one below the other, or horizontally, side by side. This new option determines which direction will be tried first, when both directions are possible according to the values of 'split-width-threshold' and 'split-height-threshold'. The default value is 'longest', which means to prefer to split horizontally if the window's frame is a "landscape" frame, and vertically if it is a "portrait" frame. (A frame is considered to be "portrait" if its vertical dimension in pixels is greater or equal to its horizontal dimension, otherwise it is considered to be "landscape".) Previous versions of Emacs always tried to split vertically first, so to get previous behavior, you can customize this option to 'vertical'. The value 'horizontal' always prefers the horizontal split.

The default value of 'split-width-threshold' is changed from 160 to 150.

After splitting it is still comfortable to read text filled to 75 columns.

New optional argument INDIRECT for 'get-buffer-window-list'.

With this argument non-nil, 'get-buffer-window-list' will include in the return value windows whose buffers share their text with BUFFER-OR-NAME.

New 'display-buffer' action alist entry 'reuse-indirect'.

With such an entry, 'display-buffer-reuse-window' may also choose a window whose buffer shares text with the buffer to display.

New variable 'window-state-normalize-buffer-name'.

When bound to non-nil, 'window-state-get' will normalize 'uniquify' managed buffer names by removing 'uniquify' prefixes and suffixes. This helps to restore window buffers across Emacs sessions.

New action alist entry 'this-command' for 'display-buffer'.

You can use this in 'display-buffer-alist' to match buffers displayed during the execution of particular commands.

New command 'other-window-backward' ('C-x 0').

This moves in the opposite direction of 'other-window' and is for its default keybinding consistent with 'repeat-mode'.

New functions 'combine-windows' and 'uncombine-window'.

'combine-windows' is useful to make a new parent window for several adjacent windows and subsequently operate on that parent. 'uncombine-window' can then be used to restore

the window configuration to the state it had before running `'combine-windows'`.

New function 'window-cursor-info'.

This function returns a vector of pixel-level information about the physical cursor in a given window, including its type, coordinates, dimensions, and ascent.

Frames

New function 'frame-deletable-p'.

If this function returns nil, the following call to `'delete-frame'` might fail to delete its argument FRAME or might signal an error. It is therefore advisable to use this function as part of a condition that determines whether to call `'delete-frame'`.

New function 'frame-use-time'.

This function is the frame equivalent of the function `'window-use-time'` for a window. The result is the `'window-use-time'` of a frame's most recently used window.

New functions 'get-mru-frames' and 'get-mru-frame'.

`'get-mru-frames'` returns a list of frames sorted by their most recent use time from among all frames, or those visible or iconified on the same terminal as the selected frame. Child frames can be excluded. A single frame can be excluded, for example, the selected frame. `'get-mru-frame'` returns the single most recently used frame.

After deleting, 'delete-frame' now selects the most recently used frame.

Previously, after deleting a specified frame, `'delete-frame'` would select the oldest visible frame on the same terminal. To revert to the old behavior, set the new user option `'delete-frame-choose-selected'` to nil.

New value 'force' for user option 'frame-inhibit-implied-resize'.

This will inhibit implied resizing while a new frame is made and can be useful on tiling window managers where the initial frame size should be specified by external means.

New user option 'alter-fullscreen-frames'.

This option is useful to maintain a consistent state when attempting to resize fullscreen frames. It defaults to `'inhibit'` on NS builds which means that a fullscreen frame would not change size. It defaults to nil everywhere else which means that the window manager is supposed to either resize the frame and change the fullscreen status accordingly or keep the frame size unchanged. The value `t` means to first reset the fullscreen status and then resize the frame.

New functions to set frame size and position in one compound step.

`'set-frame-size-and-position'` sets the new size and position of a frame in one compound step. Both, size and position, can be specified as with the corresponding frame parameters `'width'`, `'height'`, `'left'` and `'top'`.

'`set-frame-size-and-position-pixelwise`' is similar but has a more restricted set of values for specifying size and position.

New commands '`split-frame`' and '`merge-frames`'.

'`split-frame`' moves a specified number of windows from an existing frame to a newly-created frame. '`merge-frames`' merges all windows from two frames into one of these frames and deletes the other one.

Frames can now be renamed to "`F<number>`" on text terminals.

Unlike with other frame names, an attempt to rename to "`F<number>`" throws an error when a frame of that name already exists.

New frame parameters '`cloned-from`' and '`undeleleted`'.

The frame parameter '`cloned-from`' is set to the frame from which the new frame is cloned using the command '`clone-frame`'.

The frame parameter '`undeleleted`' is set to t when a frame is undeleted using the command '`undelelete-frame`'.

These are useful if you need to detect a cloned frame or undeleted frame in hooks like '`after-make-frame-functions`' and '`server-after-make-frame-hook`'.

Frames now have unique ids and the new function '`frame-id`'.

Each non-tooltip frame is assigned a unique integer id. This allows you to unambiguously identify frames even if they share the same name or title. When '`undelelete-frame-mode`' is enabled, each deleted frame's id is stored for resurrection. The function '`frame-id`' returns a frame's id (in C, use the frame struct member '`id`').

New commands '`select-frame-by-id`', '`undelelete-frame-by-id`'.

The command '`select-frame-by-id`' selects a frame by ID and undeletes it if deleted. The command '`undelelete-frame-by-id`' undeletes a frame by its ID. When called interactively, both functions prompt for an ID.

Mode Line

New definitions for mode line faces under dark background mode.

The faces '`mode-line`' and '`mode-line-highlight`' now have separate definitions for the dark background mode. Previously, these two faces looked the same in both the light and dark background modes. To get the previous visuals for these two faces, customize them to have the colors "`grey75`" and "`grey40`", respectively, regardless of the background mode.

New user option '`mode-line-collapse-minor-modes`'.

If non-nil, minor mode lighters on the mode line are collapsed into a single button. The value could also be a list to specify minor mode lighters to hide or show. The default value is nil, which retains the previous behavior of showing all minor mode

lighters.

New user option 'mode-line-modes-delimiters'.

This option allows changing or removing the delimiters shown around the major mode and list of minor modes in the mode line. The default retains the existing behavior of inserting parentheses.

New minor mode 'mode-line-invisible-mode'.

This minor mode makes the mode line of the current buffer invisible. The command `'mode-line-invisible-mode'` toggles the visibility of the current-buffer's mode line. The default is to show the mode line of every buffer.

Standard mode line no longer specifies minimum widths.

The default values for the `'mode-line-position'` variable and `'mode-line-format'` user option no longer specify any minimum widths. If you use a proportional font for your mode line, you may need to customize the values of these variables to include minimum widths again.

Tab Bars and Tab Lines

New commands 'split-tab' and 'merge-tabs'.

`'split-tab'` moves a specified number of windows from an existing tab to a newly-created tab. `'merge-tabs'` merges all windows from two tabs into one of these tabs and closes the other tab.

New abnormal hook 'tab-bar-auto-width-functions'.

This hook allows you to control which tab-bar tabs are auto-resized.

'mouse-face' properties are now supported on the 'tab-bar'.

`'tab-bar'` tab "buttons" are now highlighted when the mouse pointer hovers over them. You can customize the new face `'tab-bar-tab-highlight'`.

New abnormal hook 'tab-bar-post-undo-close-tab-functions'.

This hook allows you to operate on a reopened tab.

This is useful when you define custom tab parameters that may need adjustment when a tab is restored, and avoids advice.

Tabs are closed now on releasing the middle mouse button.

Previously, closing the tab-bar's tabs was on pressing the button.

New user option 'tab-bar-define-keys'.

This controls which key bindings tab-bar creates. Values are `t`, the default, which defines all keys and is backwards compatible, `'numeric'` (tab number selection only), `'tab'` (`'TAB'` and `'S-TAB'` keys only), `nil` (which defines none).

This is useful to avoid key binding conflicts, such as when folding in outline mode using `'TAB'` keys, or when a user wants to define her own tab-bar keys without first having to remove the defaults.

New variable `'tab-bar-format-tab-help-text-function'`.

This variable may be overridden with a user-provided function to customize help text for tabs displayed on the tab-bar. Help text is normally shown in the echo area or via tooltips. See the variable's docstring for arguments passed to a help-text function.

New variable `'tab-bar-truncate'`.

When non-nil, it truncates the tab bar, and therefore prevents wrapping and resizing the tab bar to more than one line.

New user option `'tab-line-define-keys'`.

When t, the default, it redefines window buffer switching keys such as `'C-x <LEFT>'` and `'C-x <RIGHT>'` to tab-line specific variants for switching tabs.

New command `'tab-line-move-tab-forward'` (`'C-x M-<RIGHT>'`).

Together with the new command `'tab-line-move-tab-backward'` (`'C-x M-<LEFT>'`), it can be used to move the current tab on the tab line to a different position.

New command `'tab-line-close-other-tabs'`.

It is bound to the tab's context menu item `"Close other tabs"`.

New user option `'tab-line-exclude-buffers'`.

This user option controls where `'tab-line-mode'` should not be enabled in a buffer. The value must be a condition which is passed to `'buffer-match-p'`.

New user option `'tab-line-close-modified-button-show'`.

With this user option, if non-nil (the default), the tab close button will change its appearance if the tab buffer has been modified.

New user option `'tab-line-tabs-window-buffers-filter-function'`.

This user option controls which buffers should appear in the tab line. By default, this is set to not filter the buffers.

New faces `'tab-line-active'` and `'tab-line-inactive'`.

These inherit from the `'tab-line'` face, but the faces actually used on the tab lines are now these two: the selected window uses `'tab-line-active'`, non-selected windows use `'tab-line-inactive'`.

Help

New keybinding `'C-h u'` for `'apropos-user-option'`.

IDLWAVE has been moved to GNU ELPA.

The version bundled with Emacs is out-of-date, and is now marked as obsolete. Use `'list-packages'` to install the `'idlwave'` package from GNU ELPA instead.

New faces `'header-line-active'` and `'header-line-inactive'`.

These inherit from the `'header-line'` face, but the faces actually used on the header lines are now these two: the selected window uses `'header-line-active'`, non-selected windows use `'header-line-inactive'`.

In `'customize-face'`, the "Font family" attribute now supports completion.

`'process-adaptive-read-buffering'` is now nil by default.

Setting this variable to a non-nil value reduces performance and leads to wrong results in some cases. We believe that it is no longer useful; please contact us if you still need it for some reason.

`'byte-compile-cond-use-jump-table'` is now obsolete.

Modified settings for an enabled theme now apply immediately.

Evaluating a `'custom-theme-set-faces'` or `'custom-theme-set-variables'` call for an enabled theme causes the settings to apply immediately, without a need to re-load the theme.

`'describe-variable'` now automatically says if `'setopt'` is needed.

If a user option has a defcustom `':set'` function, users will normally need to set it with `'setopt'` for it to take an effect. If the docstring doesn't already mention `'setopt'`, the `'describe-variable'` command will now add a note about this automatically.

New user option `'eldoc-help-at-pt'` to show help at point via ELDoc.

When enabled, display the `'help-at-pt-kbd-string'` via ELDoc. This setting is an alternative to `'help-at-pt-display-when-idle'`.

New user option `'native-comp-async-on-battery-power'`.

Customize this to nil to disable starting new asynchronous native compilations while AC power is not connected.

New user option `'show-paren-not-in-comments-or-strings'`.

If this option is non-nil, it tells `'show-paren-mode'` not to highlight the parens that are inside comments and strings. If set to `'all'`, `'show-paren-mode'` will never highlight parens that are inside comments or strings. If set to `'on-mismatch'`, mismatched parens inside comments and strings will not be highlighted. If set to nil (the default), highlight the parens wherever they are.

New user option 'view-lossage-auto-refresh'.

If this option is non-nil, the lossage buffer in 'view-lossage' will be refreshed automatically for each new input keystroke and command performed.

Change in SVG foreground color handling.

SVG images no longer have the 'fill' attribute set to the value of ':foreground' or the current text foreground color. The 'currentcolor' CSS attribute continues to be set as before.

This change should result in more consistent display of SVG images.

To use the ':foreground' or current text color ensure the 'fill' attribute in the SVG is set to 'currentcolor', or set the image spec's ':css' value to 'svg {fill: currentcolor;}'.

Errors signaled by 'emacsclient' connections can now enter the debugger.

If 'debug-on-error' is non-nil, errors signaled by Lisp programs executed due to 'emacsclient' connections will now enter the Lisp debugger and show the backtrace. If 'debug-on-error' is nil, these errors will be sent to 'emacsclient', as before, and will be displayed on the terminal from which 'emacsclient' was invoked.

Empty string arguments to emacsclient are no longer ignored.

Emacs previously discarded arguments to emacsclient of zero length, such as in 'emacsclient --eval "(length (pop server-eval-args-left))" ""'. These are no longer discarded.

Emacs now uses the 'setrgbf' and 'setrgbb' terminfo capabilities.

Emacs now uses 24-bit colors on terminals that support the 'setrgbf' and 'setrgbb' user-defined terminfo capabilities. These are supported by more terminals and applications than the old capabilities, 'setf24' and 'setb24', which are now obsolete.

New user option 'xterm-update-cursor' to update cursor display on TTYs.

When enabled, Emacs sends Xterm escape sequences on Xterm-compatible terminals to update the cursor's appearance. Emacs can update the cursor's shape and color. For example, if you use a purple bar cursor on graphical displays then when this option is enabled Emacs will use a purple bar cursor on

compatible terminals as well. See the Info node "(emacs) Cursor Display" for more information.

New command 'copy-theme-options'.

You can use this command to copy options from a theme into your user configuration.

New user option 'multiple-terminals-merge-keyboards'.

Customizing this option to non-nil disables entering single-keyboard mode in most cases in which Emacs would by default enter that mode. This can make things work better for some cases of X forwarding; see the docstring for the new option.

Emacs now comes with Org v9.8.

See the file `"etc/ORG-NEWS"` for user-visible changes in Org.

New user option 'compilation-search-extra-path'.

`compile.el` will now use paths specified in both `'compilation-search-extra-path'` and `'compilation-search-path'`, when doing search. `'compilation-search-extra-path'` is consulted first. One possible use case of this option is to add new search paths on a per-project basis with directory-local variables.

Editing Changes in Emacs 31.1

Commands for keyboard translation.

`'key-translate'` is now interactive. It prompts for a key to translate from, and another to translate to, and sets `'keyboard-translate-table'`. The new command `'key-translate-remove'` prompts for a key/translation pair, with `'completing-read'`, and removes the translation from the translation table.

Internationalization

Emacs now supports Unicode Standard version 17.0.

New input method 'greek-polytonic'.

This input method has support for polytonic and archaic Greek characters.

New language environment and input method for Tifinagh.

The Tifinagh script is used to write the Berber languages.

New input methods for Northern Iroquoian languages.

Input methods are now implemented for Haudenosaunee languages in the Northern Iroquoian language family: `'mohawk-postfix'` (Mohawk [Kanien'kéha / Kanyen'kéha / Onkwehonwehnéha]), `'oneida-postfix'` (Oneida [On'yote'a·ká· / Onyota'a:ká: / Ukwehuwehnéha]), `'cayuga-postfix'` (Cayuga [Gayogo'ho:nqhnéha:]), `'onondaga-postfix'` (Onondaga [Onyda'gegá]), `'seneca-postfix'` (Seneca [Onödowá'ga:']), and `'tuscarora-postfix'` (Tuscarora [Skarù·rę]). Additionally, there is a general-purpose `'haudenosaunee-postfix'` input method to facilitate writing in the orthographies of the six languages simultaneously.

New input methods for languages based on Burmese.

These include: Burmese, Burmese (visual order), Shan, and Mon.

New language environment and input methods for Syriac languages.

A new language environment for languages that use the Syriac script: Classical Syriac, Aramaic, and others. Two new input methods for these languages: Syriac and Syriac (phonetic).

'visual-wrap-prefix-mode' now supports variable-pitch fonts.

When using `'visual-wrap-prefix-mode'` in buffers with variable-pitch fonts, the wrapped text will now be lined up correctly so that it is exactly below the text after the prefix on the first line.

New commands 'unix-word-rubout' and 'unix-filename-rubout'.

Unix-words are words separated by whitespace regardless of the buffer's `syntax table`. In a Unix terminal or shell, `'C-w'` kills by Unix-word. The new commands `'unix-word-rubout'` and `'unix-filename-rubout'` allow you to bind keys to operate more similarly to the terminal.

New user option 'kill-region-dwim'.

This option, if non-nil, modifies the fall-back behavior of `'kill-region'` (`'C-w'`) if no region is active, and will kill the last word instead of raising an error. Note that if you have disabled Transient Mark mode you might prefer to use `'unix-word-rubout'`, as this feature relies on there being an active region.

New user option 'delete-pair-push-mark'.

This option, if non-nil, makes `'delete-pair'` push a mark at the end of the region enclosed by the deleted delimiters. This makes it easy to act on that region. For example, we can highlight it using `'C-x C-x'`.

Electric Pair mode

Electric Pair mode can now pair multiple delimiters at once.

You can now insert or wrap text with multiple sets of parentheses and other matching delimiters at once with Electric Pair mode, by providing a prefix argument when inserting one of the delimiters.

Electric Pair mode now supports multi-character paired delimiters.

`'electric-pair-pairs'` and `'electric-pair-text-pairs'` now allow using strings for multi-character paired delimiters.

To use this, add a list to both electric pair user options: `'("/*" . "*/")'`.

You can also specify to insert an extra space after the first string pair: `'("/*" " */" t)'`.

New user option 'electric-indent-actions'.

This user option specifies a list of actions to reindent. The possible elements for this list are: `'yank'`, reindent the yanked text; `'before-save'`, indent the whole buffer before saving it.

You can now use `'M-~'` during `'C-x s'` (`'save-some-buffers'`).

Typing `'M-~'` while saving some buffers means not to save the buffer and also to mark it as unmodified. This is an alternative way to mark a buffer as unmodified which doesn't require switching to that buffer.

New minor mode `'delete-selection-local-mode'`.

This mode sets `'delete-selection-mode'` buffer-locally. This can be useful for enabling or disabling the features of `'delete-selection-mode'` based on the state of the buffer, such as for the different states of modal editing packages.

New user option `'exchange-point-and-mark-highlight-region'`.

When set to nil, this modifies `'exchange-point-and-mark'` so that it doesn't activate the mark if it is not already active. The default value is t, which retains the old behavior. This variable has no effect when Transient Mark mode is off.

You can now use `'M-s t'` to swap FROM and TO during `'query-replace'`.

Likewise during `'query-replace-regexp'`. The original binding of `'M-s'` (`'next-matching-history-element'`) is now available on `'M-s M-s'` or `'M-s s'` for query replace minibuffer input.

New commands for filling text using semantic linefeeds.

The new command `'fill-paragraph-semelf'` fills a paragraph of text using `"semantic linefeeds"`, whereby a newline is inserted after every sentence. The new command `'fill-region-as-paragraph-semelf'` fills a region of text using semantic linefeeds as if the region were a single paragraph. You can set the variable `'fill-region-as-paragraph-function'` to the value `'fill-region-as-paragraph-semelf'` to enable functions like `'fill-paragraph'` and `'fill-region'` to fill text using `"semantic linefeeds"`.

Temporary files are named differently when `'file-precious-flag'` is set.

When the user option `'file-precious-flag'` is set to a non-nil value, Emacs now names the temporary file it creates while saving buffers using the original file name with `".tmp"` appended to it. Thus, if saving the buffer fails for some reason, and the temporary file is not renamed back to the original file's name, you can easily identify which file's saving failed.

`'C-u C-x .'` clears the fill prefix.

You can now use `'C-u C-x .'` to clear the fill prefix, similarly to how you could already use `'C-u C-x C-n'` to clear the goal column.

New prefix argument for 'C-/' in Direed and Proced modes.

The Direed and Proced major modes bind mode-specific undo commands to the same keys to which 'undo' is globally bound, 'C-/', 'C-_' and 'C-x u'. These commands did not previously accept a prefix argument. Now a numeric prefix argument specifies a repeat count, just like it already did for 'undo'.

New minor mode 'center-line-mode'.

This mode keeps modified lines centered horizontally according to the value of 'fill-column', by calling 'center-line' on each non-empty line of the modified region.

New command 'unfill-paragraph'.

This is the inverse of 'M-q' ('fill-paragraph').

Changes in Specialized Modes and Packages in Emacs 31.1

Project

New command 'project-root-find-file'.

It is equivalent to running 'project-any-command' with 'find-file'.

New command 'project-customize-dirlocals'.

It is equivalent to running 'project-any-command' with 'customize-dirlocals'.

Improved prompt for 'project-switch-project'.

The prompt now displays the chosen project on which to invoke a command.

'project-prompter' values may be called with up to three arguments.

These allow callers of the value of 'project-prompter' to specify a prompt string; prompt the user to choose between a subset of all the known projects; and disallow returning arbitrary directories. See the docstring of 'project-prompter' for a full specification of these new optional arguments.

'project-current' has a new optional argument, MAYBE-PROMPT.

If 'project-current' is called with this argument non-nil, then it is passed to the 'project-prompter' to use as a prompt string. Callers can use this to indicate the reason for which or context in which Emacs should ask

the user to select a project.

New command 'project-find-matching-buffer'.

It can be used when switching between projects with similar file trees (such as Git worktrees of the same repository). It supports being invoked standalone or from the 'project-switch-commands' dispatch menu.

New variable 'project-find-matching-buffer-function'.

Major modes can set this to major mode-specific functions to control how 'project-find-matching-buffer' finds matching buffers.

New user option 'project-list-exclude'.

This user option describes projects that should always be skipped by 'project-remember-project'.

New user option 'project-prune-zombie-projects'.

This user option controls the automatic deletion of projects from 'project-list-file' that cannot be accessed when prompting for a project.

The value must be an alist where each element must be in the form:

```
(WHEN . PREDICATE)
```

where WHEN specifies where the deletion will be performed, and PREDICATE is a function which takes one argument, and must return non-nil if the project should be removed.

New command 'project-save-some-buffers' bound to 'C-x p C-x s'.

This is like 'C-x s', but only for this project's buffers.

'project-remember-project' can now be called interactively.***'project-shell' and 'project-eshell' support numeric prefix buffer naming.***

They now accept numeric prefix arguments to select or create numbered shell sessions. For example, 'C-2 C-x p s' switches to or creates a buffer named "*name-of-project-shell<2>*". By comparison, a plain universal argument as in 'C-u C-x p s' always creates a new session.

'project-switch-to-buffer' re-uniquifies buffer names while prompting.

When 'uniquify-buffer-name-style' is non-nil, 'project-switch-to-buffer' changes the buffer names to only make them unique within the given project, during completion. That makes some items shorter.

'project-switch-to-buffer' uses 'project-buffer' as completion category.

The category defaults are the same as for 'buffer' but any user customizations would need to be re-added.

'project-mode-line' can now show project name only for local files.

If the value of 'project-mode-line' is 'non-remote', project name and the Project menu will be shown on the mode line only for projects with local files.

The "VC-aware" project backend caches the current project and its name.

The duration for which the values are cached depends on whether it is called from 'non-essential' context, and it is determined by variables 'project-vc-cache-timeout'

and `'project-vc-non-essential-cache-timeout'`.

Network Security Manager (NSM)

NSM warns about TLS 1.1 by default.

It has been deprecated by RFC 8996, published in 2021.

NSM warns about DHE and RSA key exchange by default.

Emacs now warns about ephemeral Diffie-Hellman key exchange, and static RSA key exchange, also when `'network-security-level'` is customized to its default `'medium'` value.

Etags

New command-line options for handling unrecognized programming languages.

The new command-line option `'--no-fallback-lang'` disables attempts to parse as Fortran or C/C++ files whose programming language `'etags'` could not determine. This allows to avoid false positives and reduces the time required to scan directories with many such files. Another new option `'--no-empty-file-entries'` disables generation of file entries in tags tables for files in which no tags were found.

Delete Selection mode

New face 'delete-selection-replacement' for the replacement text.

This comes with a change to how we track what is considered "the replacement text", which should be more robust now, and is made more clear by the highlighting.

Editorconfig

'editorconfig-apply' is declared obsolete.

You can now use `'editorconfig-display-current-properties'` without having to call `'editorconfig-apply'`.

Auth Source

Non-existing or empty files are ignored in 'auth-sources'.

File-based data stores are ignored in `'auth-sources'`, if the underlying data file does not exist. This is relevant, if a new secret is stored in such a file; the first usable entry of `'auth-sources'` is selected as target. If you want also not existing files to be selected, set the user option `'auth-source-ignore-non-existing-file'` to nil.

'auth-sources' set to nil means using the password cache only.

Autoinsert

New condition for 'auto-insert-alist'.

'`auto-insert-alist`' now also allows to have a predicate taking no argument as conditions. These types of conditions should be declared with '`(predicate FUNCTION)`'. This allows to trigger '`auto-insert`' with finer grained control.

Register

New commands '`buffer-to-register`' and '`file-to-register`'.

These allow users to interactively store files and buffers in registers. Killed buffers stored in a register using '`buffer-to-register`' are automatically converted to a file-query value if the buffer was visiting a file.

The "`*Register Preview*`" buffer shows only suitable registers.

That was already the case for the "fancy" UI but is now also true in the default UI you get, i.e., when '`register-use-preview`' is '`traditional`'.

The "`*Register Preview*`" buffer shows sorted items.

Tree-sitter

New user option '`treesit-enabled-modes`'.

You can customize it either to `t` to enable all available tree-sitter based modes, or to select a list of tree-sitter based modes to enable. Depending on customization, it modifies the variable '`major-mode-remap-alist`' from the corresponding variable '`treesit-major-mode-remap-alist`' prepared by tree-sitter based mode packages.

New user option '`treesit-auto-install-grammar`'.

It controls the automatic installation of tree-sitter grammar libraries needed for tree-sitter based modes, if these grammar libraries are not available when such modes are turned on.

'`treesit-extra-load-path`' now is a customizable user option.

The first directory in the list is used as the default directory to install the language grammar when '`treesit-auto-install-grammar`' is '`ask`', '`ask-dir`' or '`always`'.

'`treesit-language-source-alist`' supports keywords.

The language and URL are mandatory, but remaining data can use keywords like

```
(json "https://github.com/tree-sitter/tree-sitter-json" :commit "4d770d3")
```

The file `treesit-x.el` defines a number of simple tree-sitter modes.

Using the new macro '`define-treesit-generic-mode`', generic modes are defined including, but not limited to, '`gitattributes-generic-ts-mode`'. Visiting a file in such mode asks for confirmation before installing its tree-sitter grammar. Then it highlights the visited file according to the syntax defined by the grammar.

New command '`treesit-cycle-sexp-thing`'.

It cycles the type of navigation for commands that move across sexp's and lists, such as 'treesit-forward-sexp', 'treesit-forward-list', 'treesit-down-list', and 'treesit-up-list'. The type can be either 'list', the default, or 'sexp'. With the default 'list' type, these commands move using syntax tables for symbols and using the thing 'list' for lists. With the 'sexp' type, these commands move across nodes defined by the tree-sitter thing 'sexp' in 'treesit-thing-settings'.

Indirect buffers can have their own parser list.

Before, indirect buffers share their base buffer's parser list and parsers. Now they can have their own parser list.

New variable 'treesit-language-remap-alist'.

This variable allows a user to remap one language into another, such that creating a parser for language A actually creates a parser for language B. By extension, any font-lock rules or indentation rules for language A will be applied to language B instead.

This is useful for reusing font-lock rules and indentation rules of language A for language B, when language B is a strict superset of language A.

New accessor functions for each setting in 'treesit-font-lock-settings'.

Now users can access a setting's query, feature, enable flag, and override flag by 'treesit-font-lock-setting-query', 'treesit-font-lock-setting-feature', 'treesit-font-lock-setting-enable', and 'treesit-font-lock-setting-override'.

New tree-sitter thing 'list'.

Unlike the existing thing 'sexp' that defines both lists and atoms, 'list' defines only lists to be navigated by 'forward-sexp'. The new command 'treesit-forward-sexp-list' uses 'list' to move across lists. But to move across atoms inside the list it uses 'forward-sexp-default-function'.

New tree-sitter based functions for moving by lists.

If a major mode defines 'list' in 'treesit-thing-settings', tree-sitter setup for these modes sets 'forward-list-function' to 'treesit-forward-list', 'up-list-function' to 'treesit-up-list', and 'down-list-function' to 'treesit-down-list'. This enables the 'forward-list', 'up-list', and 'down-list' motion commands for those modes.

Tree-sitter enabled modes now properly support 'show-paren-mode'.

They do that by letting 'show-paren-mode' use the results of parsing by the tree-sitter library. The new function 'treesit-show-paren-data' is used to communicate the tree-sitter parsing results to 'show-paren-mode'.

Tree-sitter enabled modes now properly support 'hs-minor-mode'.

All commands from hideshow.el can selectively display blocks defined by the new tree-sitter thing 'list'.

New tree-sitter thing 'comment'.

The new variable `'forward-comment-function'` is set to the new function `'treesit-forward-comment'` if a major mode defines the thing `'comment'`.

New function 'treesit-query-eagerly-compiled-p'.

This function returns non-nil if a query was eagerly compiled.

New function 'treesit-query-source'.

This function returns the string or sexp source query of a compiled query.

New function 'treesit-language-display-name'.

This new function returns the display name of a language given the language symbol. For example, `'cpp'` is translated to `"C++"`. A new variable `'treesit-language-display-name-alist'` holds the translations of language symbols where that translation is not trivial.

New function 'treesit-merge-font-lock-feature-list'.

This function merges two tree-sitter font-lock feature lists. It returns a new font-lock feature list with no duplicates in the same level. It can be used to merge font-lock feature lists in a multi-language major mode.

New function 'treesit-replace-font-lock-feature-settings'.

Given two tree-sitter font-lock settings, it replaces the feature in the second font-lock settings with the same feature in the first font-lock settings. In a multi-language major mode it is sometimes necessary to replace features from one of the major modes with others, that are better suited to the new multilingual context.

New function 'treesit-simple-indent-modify-rules'.

Given two tree-sitter indent rules, it replaces, adds, or prepends rules in the old rules with new ones, then returns the modified rules. In a multi-language major mode it is sometimes necessary to modify rules from one of the major modes to better suit the new multilingual context.

New variable 'treesit-aggregated-simple-imenu-settings'.

This variable allows major modes to setup Imenu for multiple languages.

New variable 'treesit-aggregated-outline-predicate'.

This variable allows major modes to setup `'outline-minor-mode'` for multiple languages.

New function 'treesit-simple-indent-add-rules'.

This new function makes it easier to customize indent rules for tree-sitter modes.

New variable 'treesit-simple-indent-override-rules'.

Users can customize this variable to add simple custom indentation rules for tree-sitter major modes.

New variable 'treesit-languages-require-line-column-tracking'.

Now Emacs can optionally track line and column numbers for buffer edits and send that information to tree-sitter parsers. Parsers of languages in this list will receive line and column information. This is only needed for very few languages. So far only Haskell is known to need it.

New function 'treesit-tracking-line-column-p'.

New function to check if a buffer is tracking line and column for buffer edits.

New function 'treesit-parser-tracking-line-column-p'.

New function to check if a parser is receiving line and column information.

'treesit-language-at-point-function' is now optional.

Multi-language major modes can rely on the default return value from 'treesit-language-at' that uses the new function 'treesit-parsers-at'.

New function 'treesit-query-with-optional'.

When used in 'treesit-font-lock-rules', 'treesit-query-with-optional' returns a default query plus the valid queries from a list of optional queries.

New function 'treesit-query-with-fallback'.

When used in 'treesit-font-lock-rules', 'treesit-query-with-fallback' selects the first valid query from a list.

Tree-sitter thing functions now work better with multiple parsers.

The following functions now handle better the case when there are multiple parsers at point: 'treesit-thing-prev', 'treesit-thing-next', 'treesit-navigate-thing', 'treesit-thing-at'. When there are multiple parsers at point, instead of using whatever 'treesit-node-at' returns at point, these functions now try every relevant parser in descending order of relevance. (Deeper-embedded parsers have higher relevance.) These functions now also take an additional optional argument, `PARSER`, that allows caller to specify a parser or language to use. That also means 'treesit-beginning/end-of-defun' now can move across parsers.

New command 'treesit-explore'.

This command replaces 'treesit-explore-mode'. It turns on 'treesit-explore-mode' if it is not on, and pops up the explorer buffer if it is already on.

'treesit-explore-mode' now supports local parsers.

Now 'treesit-explore-mode' (or 'treesit-explore') prompts for a parser rather than a language, and it is now possible to select a local parser at point to explore.

Tree-sitter query predicates ':equal', ':match', and ':pred' are deprecated.

Use ':eq?', ':match?', and ':pred?' instead. The change is because newer tree-sitter libraries mandate query predicates to end with a question mark. Emacs will

transparently convert `'equal'`, `'match'`, and `'pred'` to `'eq?'`, `'match?'`, and `'pred?'`, respectively, so existing queries still work fine with the latest tree-sitter library. The predicate `'equal'` is changed to `'eq?'` to better follow tree-sitter's convention. Also, the `'match?'` predicate can now take the regexp as either the first or second argument, so it works with both tree-sitter convention (regexp arg second) and Emacs convention (regexp arg first).

Track changes

New variable `'track-changes-undo-only'` to distinguish undo changes.

Hideshow

New command `'hs-cycle'`.

This command cycles the visibility state of the current block between hide the parent block, hide the nested blocks only and show all the blocks.

New command `'hs-toggle-all'`.

This command hides or shows all the blocks in the current buffer.

`'hs-hide-level'` no longer hides all the blocks in the current buffer.

If `'hs-hide-level'` was not inside a code block, it would hide all the blocks in the buffer like `'hs-hide-all'`. Now it should only hide all the second level blocks.

New user option `'hs-display-lines-hidden'`.

If this option is non-nil, Hideshow displays the number of hidden lines next to the ellipsis. By default this is disabled.

New user option `'hs-show-indicators'`.

This user option determines if Hideshow should display indicators to show and toggle the block hiding. If non-nil, the indicators are enabled. By default this is disabled.

New user option `'hs-indicator-maximum-buffer-size'`.

This user option limits the display of Hideshow indicators to buffers that are not too large. By default, buffers larger than 2MB have the indicators disabled; the value of nil will activate the indicators regardless of the buffer size.

New user option `'hs-indicator-type'`.

This user option determines which indicator type should be used for the block indicators.

The possible values can be: `'fringe'`, display the indicators in the fringe (the default); `'margin'`, display the indicators in the margin; nil, display the indicators at end-of-line.

The new icons `'hs-indicator-show'` and `'hs-indicator-hide'` can be used to customize the indicators appearance only if `'hs-indicator-type'` is set to `'margin'` or `nil`.

The hiding behavior of some hideshow commands has changed.

`'hs-hide-block'`, `'hs-hide-level'`, `'hs-cycle'` and `'hs-toggle-hiding'` now hide the innermost block to which the current line belongs instead of the block after cursor position.

To restore the old behavior, set the new user option `'hs-hide-block-behavior'` to `'after-cursor'`.

New user option `'hs-cycle-filter'` for visibility-cycling with `'TAB'`.

This user option controls the positions on the headline of hideable blocks where the `'TAB'` key cycles the blocks' visibility.

The variable `'hs-special-modes-alist'` is now obsolete.

Instead of customizing Hideshow for a mode by setting the elements of `'hs-special-modes-alist'`, such as `START`, `COMMENT-START`, `FORWARD-SEXP-FUNC`, etc., major mode authors should set the corresponding buffer-local variables `'hs-block-start-regexp'`, `'hs-c-start-regexp'`, `'hs-forward-sexp-function'`, etc.

`'hs-hide-level'` can now hide comments too.

This is controlled by `'hs-hide-comments-when-hiding-all'`.

New minor mode `'hs-indentation-mode'`.

This buffer-local minor mode configures `'hs-indentation-mode'` to detect blocks based on indentation.

The new user option `'hs-indentation-respect-end-block'` can be used to adjust the hiding range for this minor mode.

C-ts mode

New user option `'c-ts-mode-enable-doxygen'`.

By default, this is `nil`, and the Doxygen comment blocks in C/C++ sources are highlighted like other comments. When non-`nil`, Doxygen comment blocks are syntax-highlighted if the Doxygen grammar library is available.

Csharp-ts mode

Renamed feature in `'treesit-font-lock-feature-list'`.

The feature `'property'` has been renamed to `'attribute'`, since this is what it is called in the general C# community.

Go-ts mode

New unit test commands.

Three new commands are now available to run unit tests.

The `'go-ts-mode-test-function-at-point'` command runs the unit test at point. If a region is active, it runs all the unit tests under the region. It is bound to `'C-c C-t t'` in `'go-ts-mode'`.

The `'go-ts-mode-test-this-file'` command runs all unit tests in the current file. It is bound to `'C-c C-t f'` in `'go-ts-mode'`.

The `'go-ts-mode-test-this-package'` command runs all unit tests under the package of the current buffer. It is bound to `'C-c C-t p'` in `'go-ts-mode'`.

The `'go-ts-mode-build-tags'` user option is available to set a list of build tags for the test commands.

The `'go-ts-mode-test-flags'` user option is available to set a list of additional flags to pass to the go test command line.

Lua-ts mode***New user option 'lua-ts-auto-close-block-comments'.***

When non-nil, inserting a block comment `--[[` will close it by inserting its respective `]]`. By default, this is disabled.

Java-ts mode***New user option 'java-ts-mode-enable-doxygen'.***

By default, this is nil, and the Doxygen comment blocks in Java sources are highlighted like other comments. When non-nil, Doxygen comment blocks are syntax-highlighted if the Doxygen grammar library is available.

New user option 'java-ts-method-chaining-indent-offset'.

Now method chaining is indented by 8 spaces rather than 4, and this option controls how much is indented for method chaining.

JSON-ts mode***New command 'json-ts-jq-path-at-point'.***

This command copies the path of the JSON element at point to the kill-ring, formatted for use with the `'jq'` utility.

PHP-ts mode

'php-ts-mode' now depends on 'mhtml-ts-mode'.

The direct dependency on `'js-ts-mode'`, `'css-ts-mode'` and `'html-ts-mode'` has now been replaced by `'mhtml-ts-mode'`. Navigation, Outline and Imenu work for all languages, and code maintenance is easier.

'php-ts-mode-run-php-webserver' can now accept a custom "php.ini" file.

You can use the new optional argument `CONFIG` when calling `'php-ts-mode-run-php-webserver'` to pass an alternative `"php.ini"` file to the built-in Web server. Interactively, when invoked with a prefix argument, `'php-ts-mode-run-php-webserver'` prompts for the config file as well as for other connection parameters.

The user option 'php-ts-mode-css-fontify-colors' has been removed.

`'mhtml-ts-mode-css-fontify-colors'` replaces this option.

New user option 'php-ts-mode-html-relative-indent'.

In buffers containing both PHP and HTML, this option allows you to define how the PHP code should be indented relative to the position of the HTML tags.

New user option 'php-ts-mode-html-indent-offset'.

Offset of PHP code block relative to HTML tags.

New user option 'php-ts-mode-find-sibling-rules'.

Rules for finding siblings of a PHP file.

New user option 'php-ts-mode-phpdoc-highlight-errors'.

When non nil, it highlights unknown PHPDOC tags using `'font-lock-warning-face'` so that the user can identify them more easily.

New command 'php-ts-mode-show-ini'.

Show the location of the PHP ini files. If the current buffer is associated to a remote PHP file, show the remote PHP ini files.

Rust-ts mode

New user option 'rust-ts-mode-fontify-number-suffix-as-type'.

Rust number literals may have an optional type suffix. When this option is non-nil, this suffix is fontified using `'font-lock-type-face'`.

YAML-ts mode

New user option 'yaml-ts-mode-yamllint-options'.

Additional options for `'yamllint'`, the command used for Flymake's YAML support.

EIEIO

New value 'warn' for 'eieio-backward-compatibility'.

This is the new default value and causes warnings to be emitted at run-time for the use of the associated deprecated features. `(setq eieio-backward-compatibility t)` can be used to recover the previous silence.

Text mode

New commands to convert between ASCII and full-width characters.

New commands `'fullwidth-region'` and `'fullwidth-word'` convert ASCII characters in region or in the word at point to the corresponding full-width characters, which are customarily used instead of ASCII characters in

CJK texts. For example, `'A'` is converted to `'𐀀'`, `'1'` is converted to `'𐀁'`, etc. Companion commands `'halfwidth-region'` and `'halfwidth-word'` perform the opposite conversion.

Texinfo mode

Texinfo mode now can auto-close the ``' pairs.

Now inserting ```` in `'texinfo-mode'` will close it by inserting its respective `'`, if `'electric-pair-mode'` is enabled.

ASM mode

'asm-mode-set-comment-hook' is obsolete.

You can now set `'asm-comment-char'` from `'asm-mode-hook'` instead.

Ibuffer

New column 'recency' in Ibuffer display.

The user option `'ibuffer-formats'` configures the Ibuffer formats. Add `'recency'` to the format to display the column.

New value 'title' for the user option 'ibuffer-use-header-line'.

Display column titles in the header line if `'ibuffer-use-header-line'` is set to `'title'`.

New user option 'ibuffer-human-readable-size'.

When non-nil, buffer sizes are shown in human readable format.

'define-ibuffer-op' prompts can now be functions.

The prompts `'opstring'` and `'active-opstring'` can now either be strings or functions. This is useful when your prompts can benefit from dynamic content.

New Ibuffer-dedicated faces.

New faces `'ibuffer-marked'`, `'ibuffer-deletion'`, `'ibuffer-title'`, and `'ibuffer-filter-group-name'`. By default, they inherit from the general-purpose faces

Ibuffer previously used, to preserve previous behavior.

ELDoc

New ELDoc function 'elisp-eldoc-funcall-with-docstring'.

This function includes the current function's docstring in the ELDoc echo area and can be used as a more detailed alternative to 'elisp-eldoc-funcall'.

New user option 'elisp-eldoc-funcall-with-docstring-length'.

This user option specifies how long function docstrings must be displayed in 'elisp-eldoc-funcall-with-docstring'. If set to 'short' (the default), only display the first sentence of the

docstring. Otherwise, if set to 'full', display the full docstring.

New user option 'elisp-eldoc-docstring-length-limit'.

This user option controls the maximum length of docstrings in character units that 'elisp-eldoc-funcall-with-docstring' and 'elisp-eldoc-var-docstring-with-value' will show. By default, it is set to 1000 characters.

Buffer Menu

New user option 'Buffer-menu-human-readable-sizes'.

When non-nil, buffer sizes are shown in human readable format. The default is nil, which retains the old format.

Term

The terminal emulator now supports auto-margins control.

Term mode now handles DECAWM escape sequences that control whether text automatically wraps at the right margin:

- \e[?7h enables auto-margins (default) - \e[?7l disables auto-margins

When auto-margins is disabled, characters that would go beyond the right margin are discarded, which matches the behavior of physical terminals and other terminal emulators. Control sequences and escape sequences are still processed correctly regardless of margin position.

SMerge mode

New 'repeat-map' for SMerge conflict resolution commands.

With 'repeat-mode' enabled, after invoking an SMerge command (for example, 'C-c ^ n'), you can repeat further SMerge commands by typing just the final key (for example, 'n', 'p', 'u', 'l').

New command 'smerge-extend' extends a conflict over surrounding lines.

New command `'smerge-refine-exchange-point'` to jump to the other side.

When used inside a refined chunk, it jumps to the matching position in the "other" side of the refinement: if you are in the new text, it jumps to the corresponding position in the old text and vice versa.

New user option `'smerge-refine-shadow-cursor'`.

When `'smerge-refine'` shows the conflict diffs at word granularity, a "shadow cursor" is now displayed in the "lower" version when point is in the "upper" version, and vice versa. The "shadow cursor" is just the character corresponding to the position where `'smerge-refine-exchange-point'` would jump, shown in a new distinct face `'smerge-refine-shadow-cursor'`, by default a box face.

`'smerge-refine-regions'` can compare regions in different buffers.

Cursor Sensor mode

New direction `'moved'` used when the cursor moved within the active area.

Image Dired

`'image-dired-show-all-from-dir'` takes the same first argument as `'dired'`.

This allows passing a string with wildcards, or a cons cell where the first element is a list and the rest is a list of files.

Bound unused letters in `'image-dired-thumbnail-mode-map'`.

For a more comfortable navigation experience (as in, no modifier keys), the keys `'f'`, `'b'`, `'n'`, `'p'`, `'a'` and `'e'` are now bound to the same functions as their `'C-'` counterparts.

Browse URL

New user option `'browse-url-transform-alist'`.

This user option is an alist that allows transforming URLs before asking a web browser to load them. For example, it could be used like this:

```
(add-to-list 'browse-url-transform-alist '("www.google.com" . "www.duckduckgo.com"))
```

New command `'browse-url-qutebrowser'` for Qutebrowser.

For better integration with Qutebrowser, set `'browse-url(-secondary)-browser-function'` to `'browse-url-qutebrowser'`.

New GTK-native launch mode.

For better Wayland support, the pgtk toolkit exposes a new `'x-gtk-launch-uri'` browse-url handler and uses it by default when URLs are browsed from a PGTK frame. For other frames, we fall back to the default URL launch function. This change allows us to properly raise browser windows under Wayland using the xdg_activation_v1 protocol.

'RET' can visit URLs in read-only buffers.

In some keymaps such as `'ansi-osc-hyperlink-map'`, `'browse-url-button-map'`, `'goto-address-highlight-keymap'`, `'bug-reference-map'` it is possible now to visit URLs by typing just `'RET'` instead of `'C-c RET'` in read-only buffers.

Removed support for some obsolete web browsers.

Conkeror (obsolete since Emacs 28.1), gnome-moz-remote (obsolete since Emacs 25.1), and gnudoit (obsolete since Emacs 25.1).

'browse-url-firefox-program' now supports LibreWolf and Zen Browser.

LibreWolf, Floorp and Zen Browser, three popular Firefox forks, have been added to the programs that are automatically recognizable as Firefox workalikes. Emacs will set `'browse-url-firefox-program'` to the first one of these found on your system.

CL-Lib

Derived types (i.e. 'cl-deftype') can now be used as method specializers.***Some cl-lib functions and macros are now built-in.***

These functions or macros have been added to Emacs Lisp, and the old names are now aliases for the built-in equivalents:

- `'cl-incf'` renamed to `'incf'` - `'cl-decf'` renamed to `'decf'` - `'cl-oddp'` renamed to `'oddp'` - `'cl-evenp'` renamed to `'evenp'` - `'cl-plusp'` renamed to `'plusp'` - `'cl-minusp'` renamed to `'minusp'` - `'cl-member-if'` renamed to `'member-if'`

`'cl-member-if'` is marked obsolete. The other names are deprecated too, and will be marked as obsolete in a future release.

'cl-labels' now also accepts '(FUNC EXP)' bindings, like 'cl-flet'.

Such bindings make it possible to compute which function to bind to FUNC.

'cl-block' names are now lexically scoped, as documented.***'cl-locally' is now obsolete.***

It is an alias for the `'progn'` special-form.

'cl-declare' is now obsolete; use 'defvar' instead.***'cl-gensym' is now obsolete; use 'gensym' instead.******New macro 'cl-with-accessors'.***

This macro is similar to `'with-slots'`, but uses accessor functions instead of slot names. It is useful when slots' accessor functions are used repeatedly, such as reading from a slot and then writing to that slot. Symbol macros are created for the accessor functions using `'cl-symbol-macrolet'`, so that they can be used with `'setq'` and `'setf'`.

Whitespace

'whitespace-cleanup' now adds missing newline at end of file.

If 'whitespace-style' includes 'missing-newline-at-eof' (which is the default), the 'whitespace-cleanup' function will now add the newline.

'whitespace-mode' now can prettify page delimiter characters ('^L').

If 'page-delimiters' is set in 'whitespace-style', or the new minor mode 'whitespace-page-delimiters-mode' is on, the page delimiter characters ('^L') are displayed as a pretty horizontal line that spans the entire width of the window. The new 'whitespace-page-delimiter' face can be used to customize the appearance.

New user option 'whitespace-global-mode-buffers'.

Normally, 'global-whitespace-mode' skips special buffers whose name starts with an asterisk "*". This user option provides an override: it contains a list of regular expressions used to match the names of special buffers in which 'global-whitespace-mode' should turn on. The default value preserves the existing exception for the "*scratch*" buffer.

Bookmark

Bookmark history now saves each bookmark only once.

Previously, the variable 'bookmark-history' accumulated duplicate bookmark names when bookmark features were used interactively. This made their history larger than necessary for frequent bookmark users. Bookmark names are now saved uniquely.

New user option 'bookmark-bmenu-type-column-width'.

This user option controls the width of the type column on the bookmark menu 'bookmark-bmenu-list'. The default value is 8 which is backwards compatible.

New hook 'bookmark-after-load-file-hook'.

This hook is run by 'bookmark-load' after loading a bookmark file. This hook can be used, for example, to reconcile 'bookmark-alist' against bookmark state that you, or a package that you use, maintains.

Recentf

You can now regularly auto-save recently opened files.

Customize user option 'recentf-autosave-interval' to the number of seconds between auto saving recently opened files. For example, customize this variable to 300 to save recently opened files every 5 minutes. From Lisp, use 'setopt' not 'setq'.

If 'recentf-autosave-interval' is nil, auto saving is disabled; this is the default.

This user option is in sympathy with save-place, and savehist autosave timers.

New user option 'recentf-show-messages'.

'recentf-save-list' can print a message when saving the recentf list. The new option, if set to nil, suppresses this message.

New user option 'recentf-suppress-open-file-help'.

By default, invoking 'recentf-open-files' displays a message saying what action clicking or typing 'RET' on the item at point executes, and tabbing between items in the "*Open Recent*" buffer likewise displays such messages. To suppress these messages, customize the user option 'recentf-suppress-open-file-help' to non-nil. The default value of this option is nil.

New user option 'recentf-exclude-ignored-extensions'.

Add the new predicate function 'recentf-exclude-file-by-extension-p' to the list that is the value of the user option 'recentf-exclude' to ignore files with certain extensions. By default, adding this function to 'recentf-exclude' ignores files whose extensions are listed in 'completion-ignored-extensions'; you can specify a different list of extensions by customizing the new user option 'recentf-exclude-ignored-extensions'.

Saveplace

You can now regularly auto-save places.

Customize user option 'save-place-autosave-interval' to the number of seconds between auto saving places. For example, customize this variable to 300 to save places every 5 minutes. From Lisp, use 'setopt' not 'setq'.

If 'save-place-autosave-interval' is nil, auto saving is disabled; this is the default.

This user option is in sympathy with recentf, and savehist autosave timers.

Savehist

The history file can be modified by external tools.

Emacs can now handle this case gracefully by merging the external and internal history information. This feature is activated only when 'savehist-additional-variables' is nil.

Savehist no longer saves additional variables more than once.

If you configured 'savehist-additional-variables' with variables that were also dynamically accumulated in minibuffer history during minibuffer use, they are now saved only once in the file specified by 'savehist-file'. Previously, they were saved twice.

Rectangle Mark

New user option to control whether empty rectangle selections are shown.

The new user option `'rectangle-indicate-zero-width-rectangle'` can be used to disable the default display of empty rectangular selections. The default is `t`; set it to `nil` to disable the indication (which causes a horizontal shift of text on display, and thus could be annoying).

Message

"In-Reply-To" header contains only a message id.

The "In-Reply-To" header created when replying to a message now contains only the originating message's id, conforming to RFC 5322. The previous behavior included additional information about the originating message. The new user option `'message-header-use-obsolete-in-reply-to'`, `nil` by default, can be set to a non-`nil` value to restore the previous behavior.

'message-subject-re-regexp' default value is derived from 'mail-re-regexps'.

`'mail-re-regexps'` is a new user option that is easier to customize than `'message-subject-re-regexp'`. `'message-subject-re-regexp'` is still honored if it was already set.

'message-strip-subject-re' now matches case-insensitively.

'message-change-subject' inserts current subject into "future history".

Hashcash support has been removed.

It is believed to no longer be useful as a method to fight spam. The `'message-generate-hashcash'` option is now obsolete and has no effect.

Gnus

Replying to icalendar events now supports specifying a comment.

When called with a prefix argument, accepting, declining, or tentatively accepting an icalendar event will prompt for a comment to add to the response.

Hashcash support has been removed.

It is believed to no longer be useful as a method to fight spam. The `'spam-use-hashcash'` hook is now obsolete and has no effect.

Add 'M-i' keybinding as the symbolic prefix in the group keymap.

The symbolic prefix is another kind of universal prefix that is used in Gnus, see "(gnus) Symbolic Prefixes" in the Gnus manual.

Sorting selected groups is now possible with 'gnus-topic-mode'.

gnus-dbus.el is now obsolete.

System sleep integration is now independent of D-Bus.

The system sleep integration previously provided by customizing the variable `'gnus-dbus-close-on-sleep'` is now deprecated. A new system using the builtin `system-sleep.el` library is now available by customizing `'gnus-close-on-sleep'`. This will work on all systems that the `'system-sleep'` library supports.

Sieve

New keybinding to refresh buffer in 'sieve-manage-mode'.

`'sieve-refresh-scriptlist'` is now bound to `'g'` to refresh the contents of the current sieve buffer.

Button

New function 'unbuttonize-region'.

It removes all the buttons in the specified region.

Disabling 'button-mode' now removes all buttons in the current buffer.

Shell

Shell buffers now support bookmarks.

You can now bookmark local and remote shell buffers using the bookmark menu `'bookmark-bmenu-list'`, or by using the command `'bookmark-set'`. Shell bookmarks can be loaded via the menu and by using the command `'bookmark-jump'`, which opens a bookmarked shell, restores its buffer name, its current directory, and creates a remote connection, if necessary. You can customize `'shell-bookmark-name-function'`.

New command to complete the shell history.

`'comint-complete-input-ring'` (`'C-x <UP>'`) is like `'minibuffer-complete-history'` but completes on comint inputs.

'ansi-osc-directory-tracker' now respects remote directories.

Remote directories are now retained when changes to `'default-directory'` are detected by this filter. For example, `"/ssh:hostname:/home/username"` would have been stripped to just `"/home/username"` before.

Eshell

New interactive command 'eshell-clear'.

This command scrolls the screen so that only the current prompt is visible, optionally erasing all the previous input/output as well. Previously, the Eshell built-in command `'eshell/clear'` supported this (e.g., to call it via `'M-x'`), but this new command behaves more consistently if you have a partially-typed command at the Eshell prompt.

New user option 'eshell-command-async-buffer'.

This option lets you tell `'eshell-command'` how to respond if its output buffer is already in use by another invocation of `'eshell-command'`, much like `'async-shell-command-buffer'` does for `'shell-command'`. By default, this will prompt for confirmation before creating a new buffer when necessary. To restore the previous behavior, set this option to `'confirm-kill-process'`.

'eshell-execute-file' is now an interactive command.

Interactively, this now prompts for a script file to execute. With the prefix argument, it will also insert any output into the current buffer at point.

'eshell-command' and 'eshell-execute-file' can now set where stderr goes.

These functions now take an optional ERROR-TARGET argument to control where to send the standard error output. See the "(eshell) Entry Points" node in the Eshell manual for more details.

You can now loop over ranges of integers with the Eshell 'for' command.

When passing a range like `'BEGIN..END'` to the Eshell `'for'` command, Eshell will now iterate over each integer between BEGIN and END, not including END.

Conditional statements in Eshell now use an 'else' keyword.

Eshell now prefers the following form when writing conditionals:

```
if {conditional} {true-subcommand} else {false-subcommand}
```

The old form (without the `'else'` keyword) is retained for compatibility.

You can now chain conditional statements in Eshell.

When using the newly-preferred conditional form in Eshell, you can now chain together multiple `'if'/'else'` statements. For more information, see "(eshell) Control Flow" in the Eshell manual.

Eshell's built-in 'wait' command now accepts a timeout.

By passing `'-t'` or `'--timeout'`, you can specify a maximum time to wait for the processes to exit. Additionally, you can now wait for external processes by passing their PIDs.

New hook 'eshell-after-initialize-hook'.

This hook runs after an Eshell session has been fully initialized, immediately before running `'eshell-post-command-hook'` for the first time.

Improved history Isearch.

History Isearch in Eshell is reworked. Two new commands `'eshell-isearch-backward-regexp'` and `'eshell-isearch-forward-regexp'` are added for incrementally searching through the input history. `'eshell-isearch-backward-regexp'` is bound to `'M-r'` by default, and `'M-s'` is freed for normal search commands. If you would

like to restore the previous key-bindings for the non-incremental search commands, put in your configuration:

```
(with-eval-after-load 'em-hist (keymap-set eshell-hist-mode-map "M-r"  
#'eshell-previous-matching-input) (keymap-set eshell-hist-mode-map "M-s"  
#'eshell-next-matching-input))
```

New user option 'eshell-history-isearch'.

When `'eshell-history-isearch'` is nil (the default), Isearch commands search in the buffer contents. If you customize it to t, those commands only search in input history. If you customize it to the symbol `'dwim'`, those commands search in input history only when the point is after the last prompt.

Eshell 'alias' command now sorts the alias list.

When adding an alias interactively, Eshell now sorts the list of aliases before saving the alias file. This maintains the stability of the list of aliases to make the diff between versions more readable if you store your aliases in version control.

Mail Utils

New user option 'mail-re-regexps'.

This contains the list of regular expressions used to match `"Re:"` and international variants of it when modifying the Subject field in replies.

Mairix

'mairix-search' now keeps its own minibuffer history.

Imap

'imap-authenticate' can now use PLAIN authentication.

"AUTH=PLAIN" support is auto-enabled if the IMAP server supports it. If you do not wish to use "AUTH=PLAIN", pass a specific authentication type to `'imap-open'` for `'imap-authenticate'` to use, or remove `'plain'` from `'imap-authenticators'`.

Rmail

'rmail-re-abbrevs' default value is now derived from 'mail-re-regexps'.

`'mail-re-regexps'` is a new user option that is easier to customize than `'rmail-re-abbrevs'`. `'rmail-re-abbrevs'` is still honored if it was already set.

New user options for formatting Rmail summary lines.

`'rmail-summary-sender-function'` and `'rmail-summary-recipient-function'` control how the sender/recipient fields are displayed in the summary. `'rmail-summary-address-width'` controls the width of that field.

New user option 'rmail-mime-save-action'.

This option specifies an action to take after saving a MIME attachment. Predefined values include visiting the file in Emacs, jumping to the file in Direx, or opening the file with an external program. You can also provide a custom function.

Rmail now detects email messages from suspicious sender addresses.

If the "From" header of a message contains a suspicious email address, Rmail will now highlight it in a distinct face and provide a 'help-echo' tooltip explaining the reason. (What exactly is considered as suspicious email addresses is determined by the function 'textsec-suspicious-p', which see.) This is controlled by the new user option 'rmail-detect-suspicious-headers', whose default value is non-nil; customize to nil to disable the check.

Sendmail***Sending an email via sendmail.el checks for suspicious addressees.***

The command 'mail-send', used to send email in Mail mode, now checks the addressees for suspicious email addresses. If such addresses are found, the command will show them and the reason they are considered suspicious, and will request a confirmation before sending the message. This follows the behavior of Message mode, and affects users who customize 'mail-user-agent' to the value

'sendmail-user-agent'.

SHR***SHR now slices large images into rows.***

Sliced images allow for more intuitive scrolling up/down by letting you scroll past each slice, instead of jumping past the entire image. Previously, SHR sliced images when zoomed to their original size, no matter how large or small that was. Now, SHR slices any images taller than 'shr-sliced-image-height'. For more information, see the "(eww) Advanced" node in the EWW manual.

You can now customize the image zoom levels to cycle through.

By customizing 'shr-image-zoom-levels', you can change the list of zoom levels that SHR cycles through when calling 'shr-zoom-image'.

New user option 'shr-fill-text'.

When 'shr-fill-text' is non-nil (the default), SHR will fill text according to the width of the window. If you customize it to nil, SHR will leave the text as-is; in that case, EWW will automatically enable 'visual-line-mode' when displaying a page so that long lines are visually wrapped at word boundaries.

EWW***EWW now enables 'visual-wrap-prefix-mode' when 'shr-fill-text' is nil.***

By default, `'shr-fill-text'` is `t`, and EWW fills the text according to the width of the window. If you customize `'shr-fill-text'` to `nil`, EWW will now automatically turn on `'visual-wrap-prefix-mode'` in addition to `'visual-line-mode'`, so that long lines are wrapped at word boundaries near window edge and the continuation lines are indented using prefixes computed from surrounding context.

New user option `'eww-guess-content-type-functions'`.

The value is a list of functions that EWW should call to determine the content-type of Web pages which don't have a valid `'Content-Type'` header. The default value is a function that considers a page with an HTML `'doctype'` declaration to have context-type `"text/html"`.

`'eww-switch-to-buffer'` falls back to `'eww'`.

When there is no EWW buffer, `'eww-switch-to-buffer'` falls back to calling `'eww'`.

URL Queue

`'url-queue-retrieve'` now makes use of some url request variables.

The variables `'url-request-data'`, `'url-request-method'`, and `'url-request-extra-headers'` can now be bound around a call to `'url-queue-retrieve'`. Binding them has the same effect as for `'url-retrieve'`.

CC mode

New type of `'c-offsets-alist'` element.

The `'cdr'` of such an alist element may now be a syntactic symbol. A source line with a syntactic element whose symbol is the `'car'` of that alist element is indented as though it were the `'cdr'`.

Enums now have their own syntactic symbols.

The new symbols `'enum-open'`, `'enum-close'`, `'enum-intro'` and `'enum-entry'` are used in the analysis of enum constructs. Previously, they were given `'brace-list-open'`, etc. These are fully described in the `"(ccmode) Enum Symbols"` node of the CC mode manual.

Enums are now, by default, indented like classes, not brace-lists.

To get the old behavior back, add an element `'(enum-open . brace-list-open)'` to `'c-offsets-alist'` in your CC mode style, or amend `'c-offsets-alist'` likewise in any of the other ways detailed in the `"(ccmode) Config Basics"` node of the CC mode manual.

Emacs Lisp mode

Semantic highlighting support for Emacs Lisp.

`'emacs-lisp-mode'` can now use code analysis to highlight more symbols more accurately. Customize the new user option `'elisp-fontify-semantically'` to non-`nil` to enable this feature, and see the Info node `"(emacs) Semantic Font Lock"` for more information.

Checkdoc no longer warns about missing footer lines in some cases.

Emacs Lisp libraries have traditionally ended with a footer line (sometimes referred to as "terminating comment"). Their purpose was to easily detect files that had been truncated in transit on ancient and less reliable connections:

```
;; some-cool-package.el ends here
```

'checkdoc' will no longer warn if that line is missing for packages that explicitly only support Emacs 30.1 or later, as specified in the "Package-Requires" header. The reason for keeping the warning for packages that support earlier versions of Emacs is that package.el in those versions can't install packages where that line is missing.

This change affects both 'checkdoc' and the corresponding flymake backend.

Checkdoc no longer warns about wide docstrings.

The Checkdoc warning for wide docstrings duplicates the byte-compiler warning added in Emacs 28.1. This redundancy is now removed.

New user option 'checkdoc-arguments-missing-flag'.

Set this to nil to disable warnings for function arguments that are not documented in docstrings.

Checkdoc will now flag incorrect formatting in warnings.

This affects calls to 'warn', 'lwarn', 'display-warning', and 'message-box'.

New user option 'checkdoc-allow-quoting-nil-and-t'.

Customizing this option to a non-nil value stops checkdoc from warning for docstrings where symbols 'nil' and 't' are in quotes.

The default of 'checkdoc-verb-check-experimental-flag' is now nil.

In most cases, having it enabled leads to a large amount of false positives.

New function 'checkdoc-batch'.

It checks the buffer in batch mode, prints all found errors and signals the first found error.

New file-local variable 'lisp-indent-local-overrides'.

This variable can be used to locally override the indent specification of symbols.

IELM

IELM input history is now saved also when the IELM process is killed.

When you kill the IELM process with 'C-c C-c', the input history is now saved to the file specified by 'ielm-history-file-name', just like when you exit the Emacs session or kill the IELM buffer.

New value 'point' for user option 'ielm-dynamic-return'.

When `'ielm-dynamic-return'` is set to `'point'`, typing `'RET'` has dynamic behavior based on whether point is inside an sexp. While point is inside an sexp typing `'RET'` inserts a newline, and otherwise Emacs proceeds with evaluating the expression. This is useful when `'electric-pair-mode'`, or a similar automatic pairing mode, is enabled.

Doc View***Dedicated buffer for plain text contents.***

When switching to the plain text contents with `'doc-view-open-text'`, Doc View now creates a dedicated buffer to display it. `'C-c C-c'` gets you back to the real Doc View buffer if it still exists.

New commands to save and restore pages in buffer-local registers.

Doc View can store the current page to buffer-local registers with the new command `'doc-view-page-to-register'` (bound to `'m'`), and later the stored page can be restored with `'doc-view-jump-to-register'` (bound to `''`).

Doc View can generate imenu indices for DjVu and ODF documents.

When the `'djvused'` program is available, Doc View can now generate an imenu index for DjVu files from its outline. Indices for Open Document Format (ODF) files as used by OpenOffice and LibreOffice are generated using the `'mutool'` program after their initial conversion to PDF format. The name of the `'djvused'` program can be customized by changing the user option `'doc-view-djvused-program'`.

Ispell***The default value of 'ispell-help-timeout' has changed.***

The default value is now 30 seconds, as the old value was too short to allow reading the help text.

Ispell can now save spelling corrections as abbrevs.

In the Ispell command loop, type `'C-u'` immediately before selecting a replacement to toggle whether that correction will be saved as a global abbrev expansion for its misspelling. The new user option `'ispell-save-corrections-as-abbrevs'` determines whether abbrev saving is enabled by default.

Flyspell***New user option 'flyspell-delay-use-timer'.***

By default, Flyspell waits after so-called `"delayed"` commands by calling `'sit-for'`. If you customize this option to non-nil, Flyspell instead sets up a timer to perform spell-checking after a short delay, which allows idle timers and other code to run during this delay period. We consider making this behavior the default in a future Emacs version, so we invite Flyspell users to enable this new option and report issues.

'turn-on-flyspell' and 'turn-off-flyspell' are obsolete.

To unconditionally enable `'flyspell-mode'` from a hook, use this instead:

```
(add-hook 'text-mode-hook #'flyspell-mode)
```

Tramp

Tramp signals 'remote-file-error' in case of connection problems.

This is a subcategory of `'file-error'`. Therefore, all checks for `'file-error'` in `'condition-case'`, `'ignore-error'`, `'error-conditions'` and alike still work.

New command 'tramp-cleanup-bufferless-connections'.

Connection-related objects for which no associated buffers exist, except for Tramp internal buffers, are flushed. This is helpful to prune connections after you close remote-file buffers without having to either cherry-pick via `'tramp-cleanup-connection'` or clear them all via `'tramp-cleanup-all-connections'`.

External methods can now be used in multi-hop connections.

This is implemented for `'tramp-sh'` methods, like `"/scp:user@host|sudo::"`.

New command 'tramp-dired-find-file-with-sudo'.

This command, bound to `'@'` in Dired, visits the file or directory on the recent Dired line with superuser, or root, permissions.

'C-x x @' is now bound to 'tramp-revert-buffer-with-sudo'.

You can use `'C-u C-x x @'` to select a Tramp method other than the default, `"sudo"`.

'tramp-file-name-with-method' can now be set as connection-local variable.

New optional connection methods "surs" and "sudors".

These connection methods are similar to `"su"` and `"sudo"`, but they use the modern `'su-rs'` and `'sudo-rs'` commands.

Connection method "kubernetes" supports now optional namespace.

The host name for Kubernetes connections can be of kind `[CONTAINER.]POD[%NAMESPACE]`, in order to specify the namespace to be used. This overrides the setting in `'tramp-kubernetes-namespace'`, if any.

Different proxies for the same destination host name can be specified.

A typical example are docker containers, which run on different hosts under the same docker name. When the user option `'tramp-show-ad-hoc-proxies'` is non-nil, such ad-hoc multi-hop file names can be used in parallel. Example: on both remote hosts `"host1"` and `"host2"` there is a docker container `"name"`, respectively:

```
/ssh:user1@host1|docker:name: /ssh:user2@host2|docker:name:
```

This feature is experimental.

Implementation of filesystem notifications for connection method "smb".

Remote process support has been rewritten for the "smb" connection method.

For more information, see "(tramp) Running remote processes on MS Windows hosts" in the Tramp manual.

New functions to extend the set of operations with a remote implementation.

The new functions `'tramp-add-external-operation'` and `'tramp-remove-external-operation'` allow to add an implementation for other operations but the defined set of magic file name operations. This can be used by external ELPA packages for performance optimizations in special cases. For more information, see "(tramp) New operations" in the Tramp manual.

New user option 'tramp-propagate-emacsclient-tramp'.

When this option is non-nil, Tramp propagates the environment variable `EMACSCIENT_TRAMP` with a proper value to remote processes. This is helpful if you want to start emacsclient on a remote host from a process started inside Emacs.

Isearch and Replace

Typing 'd' during 'query-replace' shows the diff buffer with replacements.

Diff

'diff-mode' now refrains from automatically refining big hunks.

What is big is defined by the new `'diff-refine-threshold'` user option.

Refining a hunk shows a "shadow cursor" at the beginning/end of region.

By default, the shadow cursor looks like an empty rectangle the size of a character cell. It is displayed at the beginning or the end of the refined region, to better show where the refined region starts or ends. This can be controlled by the new user option `'smerge-refine-shadow-cursor'`, which also affects SMerge mode, see below.

New command 'diff-kill-ring-save'.

This command copies to the `'kill-ring'` a region of text modified according to diffs in the current buffer, but without applying the diffs to the original text. If the selected range extends a hunk, the command attempts to look up and copy the text in between the hunks.

New command 'diff-revert-and-kill-hunk' bound to 'u' and 'C-c M-u'.

This command reverts the hunk at point (i.e., applies the reverse of the hunk), and then removes the hunk from the diffs. This is useful to undo or revert changes, committed and uncommitted, when you are in buffers generated by `'C-x v ='` and `'C-x v D'`. When the region is active, the command reverse-applies and kills hunks that the

region overlaps.

'v' is now bound to 'vc-next-action' in read-only Diff mode buffers.

'diff-file-prev' and 'diff-hunk-prev' always move to start of header.

Previously, 'diff-file-prev' and 'diff-hunk-prev' would move when point is after the corresponding file or hunk header, but not when inside it. Now they will always move to the start of the current header.

New command 'diff-delete-other-hunks' bound to 'C-c RET n'.

This command deletes all hunks other than the current hunk. It is useful to prepare a `"*vc-diff"` buffer for committing a single hunk. When the region is active, it deletes all hunks that the region does not overlap.

'vc-version-diff' and 'vc-root-version-diff' changed default for REV1.

They suggest the previous revision as the default for REV1, not the last one as before. This makes them different from 'vc-diff' and 'vc-root-diff' when those are called without a prefix argument.

'diff-apply-hunk' now supports creating and deleting files.

'diff-apply-buffer' supports creating files but not deleting them, yet.

Diff mode's application and killing commands now consider the region.

If the region is active, 'diff-apply-hunk', 'diff-apply-buffer' and 'diff-hunk-kill' now apply or kill all hunks that the region overlaps. Otherwise, they have their existing behavior.

'diff-apply-buffer' can reverse-apply.

With a prefix argument, it now reverse-applies hunks. This matches the existing prefix argument to 'diff-apply-hunk'.

's' is now bound to 'diff-split-hunk' in read-only Diff mode buffers.

Ediff

Ediff's copy commands now apply to all changes with 'C-u' prefix.

The Ediff copy commands, bound to 'a', 'b', 'ab', etc., now copy all changes when supplied with a universal prefix argument via 'C-u':

- 'C-u a' copies all changes from buffer A to buffer B (in 2-way diff) or to buffer C (in 3-way diff or merge).
- 'C-u b' copies all changes from buffer B to buffer A (in 2-way diff) or to buffer C (in 3-way diff or merge).
- 'C-u a b' copies all changes from buffer A to buffer B.
- 'C-u b a' copies all changes from buffer B to buffer A.
- 'C-u a c' copies all changes from buffer A to buffer C.
- 'C-u b c' copies all changes from buffer B to buffer C.
- 'C-u c a' copies all changes from buffer C to buffer A.
- 'C-u c b' copies all changes from buffer C to buffer B.

Ediff now supports more flexible custom window layouts.

Custom implementations of `'ediff-window-setup-function'` no longer need to display `*all*` Ediff windows. Any of the A, B, C, and control windows can be left undisplayed and the corresponding variable set to nil. This change enables custom layouts without a control panel window.

Dired***New user option 'dired-create-empty-file-in-current-directory'.***

When non-nil, `'dired-create-empty-file'` creates a new empty file and adds an entry for it (or its topmost new parent directory if created) under the current subdirectory in the Dired buffer by default (otherwise, it adds the new file (and new subdirectories if provided) to whichever directory the user enters at the prompt). When nil, `'dired-create-empty-file'` acts on the default directory by default.

Note that setting this user option to non-nil makes invoking `'dired-create-empty-file'` outside of a Dired buffer raise an error (like other Dired commands that always prompt with the current subdirectory, such as `'dired-create-directory'`).

New user option 'dired-check-symlinks' allows disabling validity checks.

Dired uses `'file-truename'` to check symbolic link validity when fontifying them, which can be slow for remote directories. Setting `'dired-check-symlinks'` to nil disables these checks. Defaults to t, can be set as a connection-local variable.

New user option 'dired-hide-details-hide-absolute-location'.

When Dired's `'dired-hide-details-mode'` is enabled, also hide the `'default-directory'` absolute location, typically displayed as the first line in a Dired buffer.

With `'dired-hide-details-hide-absolute-location'`:

```
project: (100 GiB available)
```

Without `'dired-hide-details-hide-absolute-location'`:

```
/absolute/path/to/my/important/project: (100 GiB available)
```

Clicking on base name of directory reverts buffer.

When `'dired-make-directory-clickable'` is non-nil, clicking on the base name of the directory now reverts the Dired buffer.

'dired-copy-filename-as-kill' supports project-relative names.

With a new value of the prefix argument (1), this command copies file names relative to the root directory of the current project.

Warning when Dired displays a file name with a literal newline.

If Dired uses an `'ls'` implementation that supports the `'-b'` switch, then on visiting a directory that contains a file whose name has a newline, and Dired displays that character as a literal newline, Emacs now automatically pops up a buffer warning that such a display can be problematic for Dired and showing a way to change the display to use the unproblematic character `'\n'`.

New user option 'dired-auto-toggle-b-switch'.

When this user option is non-nil and Dired uses an `'ls'` implementation that supports the `'-b'` switch and `'dired-listing-switches'` does not include the `'-b'` switch, then on visiting a directory containing a file whose name has a newline, Emacs automatically adds the `'-b'` switch and redisplay the directory in Dired to show `'\n'` in the file name instead of a literal newline. This prevents executing many Dired operations on such a file from failing and signaling an error. The default value of this user option is nil.

New Dired handling of errors from 'ls'.

When invoking a Dired command causes `'ls'` to emit an error message, Emacs now displays the message in a popped up buffer instead of outputting it in the Dired buffer and signaling an error.

Grep

Grep results can be edited to reflect changes in the originating file.

Like Occur Edit mode, typing `'e'` in the `"*grep*" buffer will now make the 'grep' results editable. The edits will be reflected in the buffer visiting the originating file. Typing 'C-c C-c' will leave the Grep Edit mode.`

Imenu

New user option 'imenu-allow-duplicate-menu-items'.

This specifies whether Imenu can include duplicate menu items. Duplicate items are now allowed by default (option value t), which restores the behavior before Emacs 29. Customize this to nil to get the behavior of Emacs 29 and Emacs 30.

Time Stamp

'time-stamp' can up-case, capitalize and down-case date words.

This control can be useful in languages in which days of the week and/or month names are capitalized only at the beginning of a sentence. For details, see the built-in documentation for user option

`'time-stamp-format'.`

Because this feature is new in Emacs 31.1, do not use it in the local variables section of any file that might be edited by an older version of Emacs.

Some historical 'time-stamp' conversions now warn.

'time-stamp-pattern' and 'time-stamp-format' had quietly accepted several 'time-stamp' conversions (e.g., "%:y") that have been deprecated since Emacs 27.1. These now generate a warning with a suggested migration.

Merely having '(add-hook 'before-save-hook 'time-stamp)' in your Emacs init file does not expose you to this change. However, if you set 'time-stamp-format' or 'time-stamp-pattern' with a file-local variable, you may be asked to update the value.

TeX modes***New Xref backend for TeX modes.***

The new backend ('tex-etags') is on by default, and improves the functionality of the standard 'xref' commands in TeX buffers. You can restore the standard 'etags' backend with the 'xref-etags-mode' toggle.

BibTeX mode***New user options facilitate customization of BibTeX and biblatex entries.***

Entry definitions via the user options 'bibtex-BibTeX-aux-entry-alist', 'bibtex-biblatex-aux-entry-alist', 'bibtex-BibTeX-aux-opt-alist', and 'bibtex-biblatex-aux-opt-alist' take precedence over 'bibtex-BibTeX-entry-alist' and 'bibtex-biblatex-entry-alist'. These user options now support the definition of aliases that inherit the definition of another entry.

'bibtex-user-optional-fields' has been renamed to 'bibtex-aux-opt-alist'.

The old name is an obsolete alias.

'bibtex-include-OPTkey' is now obsolete and its default is nil.

Use 'bibtex-aux-opt-alist' instead.

New user option 'bibtex-entry-ask-for-key'.

When enabled, 'bibtex-entry' asks for a key.

'bibtex-string-file-path' and 'bibtex-file-path' are lists of directories.

For backward compatibility, considered obsolete, these user options may still be strings with colon separated lists of directories.

Midnight mode***Change for activating the mode.***

Putting '(require 'midnight)' in your init file no longer activates the mode. Now, one needs to say '(midnight-mode +1)' instead.

Python mode

New 'repeat-map' for Python indentation commands.

The commands `'python-indent-shift-left'` and `'python-indent-shift-right'` can now be repeated using `'repeat-mode'`. With `'repeat-mode'` enabled, after invoking one of these commands via `'C-c <'` or `'C-c >'`, you can press `'<'` or `'>'` to repeat the command.

Prefer "python" for 'python-interpreter' and 'python-shell-interpreter'.

On recent versions of mainstream GNU/Linux distributions, `"python"` either does not exist or it points to Python 3. These user options now default to using `"python"`, falling back to `"python3"` if it does not exist. If `"python"` points to Python 2 on your system, you now have to customize these variables to `"python3"` if you want to use Python 3 instead.

Python 2 support is now optional and disabled by default.

Since Python 2 EOL was over 5 years ago, this release removes Python 2-only builtins such as `"file"` from the default highlighting in `'python-mode'` and `'python-ts-mode'`. If you would like them highlighted, customize the new user option `'python-2-support'` to a non-nil value and restart Emacs.

Support of 'electric-layout-mode' added.

'DEL' now deletes the text in the active region.

When point is between indentation, the command `'python-indent-dedent-line-backspace'` (by default bound to `'DEL'`) now deletes the text in the region and deactivates the mark if Transient Mark mode is enabled, the mark is active, and prefix argument is 1.

'python-eldoc-function-timeout' now accepts floating-point numbers.

To allow for finer-grained adjustment of timeout for `'python-eldoc-function'`, `'python-eldoc-function-timeout'` now accepts floating-point numbers as well as integers.

The default value of 'python-shell-completion-setup-code' is changed.

A new function is added to the setup code. Users who modify this option may need to update the value for the Python shell completion to work.

Tmm Menubar

A new shortcut to navigate to previous menu.

The hardcoded `'^'` shortcut gets you back to the previous menu.

New user option 'tmm-shortcut-inside-entry'.

When non-nil, highlight the character shortcut in the menu entry's string instead of prepending it and `'tmm-mid-prompt'` to said entry.

Foldout

Improved behavior of 'foldout-exit-fold' with negative prefix argument.

When `'foldout-exit-fold'` is called with a negative argument (so that the exited fold remains visible), the position of point and window view are preserved.

New command `'foldout-widen-to-current-fold'`.

This command widens the view to the current fold level when in a fold, or behaves like `'widen'` if not in a fold.

MPC

New user option `'mpc-notifications'`.

When non-nil, MPC (the Emacs front-end to Music Player Daemon) displays a desktop notification when the song changes, using `'notifications-notify'`. The notification's title and body can be customized using the new user options `'mpc-notifications-title'` and `'mpc-notifications-body'`.

New user option `'mpc-crossfade-time'`.

When non-nil, MPC will crossfade between songs for the specified number of seconds. Crossfading can be toggled using the command `'mpc-toggle-crossfade'` or from the MPC menu.

New command `'mpc-describe-song'`.

This command displays information about the currently playing song or song at point in the `"*MPC-Songs*"` buffer. The list of tags to display can be customized using the new user option `'mpc-song-viewer-tags'` and the appearance of the list with the new faces `'mpc-table-key'`, `'mpc-table-value'`, and `'mpc-table-empty'`.

New command `'mpc-server-stats'`.

This command displays information about the connected MPD server. The appearance of the list can be customized with the new faces `'mpc-table-key'` and `'mpc-table-value'`.

VC

Printing root branch logs has moved to `'C-x v b L'`.

Previously, the command to print the root log for a branch was bound to `'C-x v b l'`. It has now been renamed from `'vc-print-branch-log'` to `'vc-print-root-branch-log'`, and bound to `'C-x v b L'`. This is more consistent with the rest of the `'C-x v'` keymap, and makes room for a new fileset-specific branch log command.

To undo this change you can use

```
(keymap-global-set "C-x v b l" #'vc-print-root-branch-log) (with-eval-after-load
'vc-dir (keymap-set vc-dir-mode-map "b l" #'vc-print-root-branch-log))
```

New command `'C-x v b l'` (`'vc-print-fileset-branch-log'`).

This command prints the log of VC changes to the current fileset on another branch.

VC Annotate for Mercurial repositories shows changeset hashes.

To restore showing revision numbers instead of changeset hashes, customize the new user option `'vc-hg-annotate-show-revision-numbers'` to non-nil.

New commands to handle repositories with multiple working trees.

Some VCS support more than one working tree with the same backing revisions store, such as with Git's `'worktree'` subcommand and Mercurial's `'share'` extension. Emacs now has some commands to manage other working trees:

- `'C-x v w c'`: Add a new working tree. - `'C-x v w w'`: Visit this file in another working tree. - `'C-x v w k'`: Kill buffers visiting this file in other working trees. - `'C-x v w s'`: Like `'C-x p p'` but limited to other working trees. - `'C-x v w a'`: Copy or move fileset changes to another working tree. - `'C-x v w A'`: Copy or move all changes to another working tree. - `'C-x v w x'`: Delete a working tree you no longer need. - `'C-x v w R'`: Relocate a working tree to another file name.

The new user option `'vc-no-confirm-moving-changes'` controls whether `'C-x v w a'` and `'C-x v w A'` ask for confirmation when moving changes between working trees. The default is to ask for confirmation.

In addition, Lisp programs that extend VC can invoke the new backend functions to obtain a list of other working trees, and to add, remove and relocate them.

Using 'e' from Log View mode to modify change comments now works for Git.***New user option 'vc-allow-rewriting-published-history'.***

Some VCS commands can change your copy of published change history without warning. In VC we try to detect before that happens, and stop. You can customize this option to permit rewriting history even though Emacs thinks it is dangerous.

'vc-clone' is now an interactive command.

When called interactively, `'vc-clone'` now prompts for the remote repository address, and the directory into which to clone the repository. It tries to automatically determine the VC backend for cloning, or prompts for that, too.

'vc-clone' now accepts an optional argument OPEN-DIR.

When the argument is non-nil, the function switches to a buffer visiting the directory into which the repository was cloned.

'vc-revert' is now bound to '@' in VC Directory.***'vc-revert' is now additionally bound to 'C-x v @'.***

This is in addition to `'C-x v u'`.

'vc-revert' now works on directories listed in VC Directory.

Reverting a directory means reverting changes to all files inside it.

New global minor mode 'vc-auto-revert-mode'.

This is like `'global-auto-revert-mode'` but limited to VCS-tracked files. As compared with VC's existing, default support for reverting files after VCS operations, the new mode is a more reliable way to ensure that Emacs reverts buffers visiting tracked files when VCS operations change the contents of those files.

New commands to cherry-pick and revert revisions.

The commands `'vc-cherry-pick'`, `'vc-revert-or-delete-revision'`, `'vc-revert-revision'` and `'vc-delete-revision'` let you copy revisions between branches, revert and delete revisions. From Log View buffers, you can use `'C'` to cherry-pick the revision at point or all marked revisions, and `'R'` to undo the revision at point or all marked revisions.

New commands to rewind branches.

In Log View mode, `'x'` deletes revisions newer than the revision at point from the history of the current branch, though without undoing the changes made by those revisions to the working tree. `'X'` is similar except that it does remove the changes from the working tree.

New command 'log-edit-done-strip-cvs-lines'.

This command strips all lines beginning with `"CVS:"` from the buffer. It is intended to be added to the `'log-edit-done-hook'` so that `'vc-cvs-checkin'` behaves like invoking `'cvs commit [files...]'` from the command line.

New user options 'vc-resolve-conflicts' and 'vc-*-resolve-conflicts'.

They control whether to mark a conflicted file as resolved when saving. You can now control it globally, with `'vc-resolve-conflicts'`, or for specific backends with `'vc-bzr-resolve-conflicts'`, `'vc-hg-resolve-conflicts'`, and `'vc-svn-resolve-conflicts'`.

New value for 'vc-git-resolve-conflicts'.

The option now accepts the symbol `'default'` as a value, which is its default value. Effectively, the default value hasn't changed, since `'vc-resolve-conflicts'` defaults to `t`, the previous default value for `'vc-git-resolve-conflicts'`.

VC Directory can now automatically add and remove marks on other lines.

When you try to use a mark or unmark command where doing so would only be permitted if other lines were marked or unmarked first, Emacs will now ask you if you'd like to change the marks on those other lines. For example, if you try to mark a file contained within a directory that is already marked, Emacs will offer to unmark the directory, first. Previously, Emacs would simply refuse to make any changes. You can customize `'vc-dir-allow-mass-mark-changes'` to restore the old behavior or dispense with the prompting.

'C-x v x' and VC Directory's 'd' command can now delete unregistered files.

Previously, these commands could only delete registered files. To restore the old, more limited behavior for VC Directory, you can do

```
(keymap-set vc-dir-mode-map "d" #'vc-dir-clean-files)
```

New VC Directory bindings 'z d' and 'D' to delete Git stashes.

These correspond to the existing 'z p' to pop a stash and 'P' to pop the stash at point (deleting the stash at point is also bound to 'C-k').

New VC Directory command 'V' ('vc-dir-root-next-action').

This is like 'v' ('vc-next-action') but applies to the whole VC Directory buffer, ignoring the position of point and any marks. This is useful to check in all local changes at once.

VC Directory can now register files when checking in mixed filesets.

Previously, if some files to be checked in were unregistered but others were added, removed or edited, Emacs would refuse to proceed. Now Emacs prompts to register the unregistered files, so that all files in the fileset are in a compatible state for a checkin.

'C-x v v' handles missing and removed files more consistently.

Missing files are those which have been removed from the filesystem but which are still tracked by version control. Removed files are those scheduled to be removed from version control in the next commit. Previously, different backends were inconsistent about applying these statuses to files, and 'C-x v v' behaved subtly differently for the two statuses. The combination of these differences between backends and in 'C-x v v' behavior was confusing. Now,

- in VC Directory, you can use 'C-x v v' on missing files to mark them as removed - when committing, you can include missing files in a set of files with different statuses, just like you've always been able to include removed files.

In addition, the Git backend has been fixed to display missing files as 'missing' instead of incorrectly subsuming them to the 'removed' status.

There is still some further work to do to rationalize VC's handling of file removal.

New user option 'vc-dir-auto-hide-up-to-date'.

If you customize this option to 'revert', the 'g' command to refresh the VC Directory buffer also has the effect of the 'x' command. That is, typing 'g' refreshes the buffer and also hides items in the 'up-to-date' and 'ignored' states. If you customize this option to any other non-nil value, then in addition, hide items whenever their state would change to 'up-to-date' or 'ignored'.

New user option 'vc-dir-save-some-buffers-on-revert'.

If you customize this option to non-nil, Emacs will offer to save relevant buffers before generating the contents of a VC Directory buffer (like the third-party package Magit does with its status buffer).

New commands to report incoming and outgoing diffs.

'vc-root-diff-incoming' and 'vc-root-diff-outgoing' report diffs of all the changes that would be pulled and would be pushed, respectively. They are the diff analogues of the existing commands

'vc-root-log-incoming' and 'vc-root-log-outgoing'.

In particular, 'vc-root-diff-outgoing' is useful as a way to preview your push and ensure that all and only the changes you intended to include were committed and will be pushed.

'vc-diff-incoming' and 'vc-diff-outgoing' are similar but limited to the current VC fileset.

New commands to report information about unintegrated changes.

'C-x v T =' ('vc-diff-unintegrated') and 'C-x v T D' ('vc-root-diff-unintegrated') report diffs of changes since the merge base with the remote branch, including uncommitted changes. 'C-x v T l' ('vc-log-unintegrated') and 'C-x v T L' ('vc-root-log-unintegrated') show the corresponding revision logs. These are useful to view all outstanding (unmerged, unpushed) changes on the current branch. They are also available as 'T =', 'T D', 'T l' and 'T L' in VC Directory buffers.

'C-x v T R =' ('vc-diff-remote-unintegrated'), 'C-x v T R D' ('vc-root-diff-remote-unintegrated'), 'C-x v T R l' ('vc-log-remote-unintegrated') and 'C-x v T R L' ('vc-root-log-remote-unintegrated') are corresponding commands which report information about the remote versions of a topic branch.

New commands to report combined diffs of all local changes.

'C-x v E =' ('vc-diff-outgoing-and-edited') and 'C-x v E D' ('vc-root-diff-outgoing-and-edited') report combined diffs of all outgoing changes plus any uncommitted changes. They are useful to show all work that's present only locally.

New user option 'vc-use-incoming-outgoing-prefixes'.

If this is customized to non-nil, 'C-x v I' and 'C-x v O' become prefix commands, such that the new incoming and outgoing commands have global bindings:

- 'C-x v I l' is bound to 'vc-root-log-incoming' - 'C-x v I D' is bound to 'vc-root-diff-incoming' - 'C-x v O l' is bound to 'vc-root-log-outgoing' - 'C-x v O D' is bound to 'vc-root-diff-outgoing'.

New display of outgoing revisions count in VC Directory.

If there are outgoing revisions, VC Directory now includes a count of how many in its headers, to remind you to push them. You can disable this by customizing 'vc-dir-show-outgoing-count' to nil.

New user option 'vc-async-checkin' to enable async checkin operations.

Currently only supported by the Git and Mercurial backends.

New user option 'vc-display-failed-async-commands'.

If non-nil, displays the buffer with the output of the failed command when an asynchronous VC command (e.g., pulls and pushes) fails.

New 'log-edit-hook' option to display diff of changes to commit.

You can customize 'log-edit-hook' to include its new 'log-edit-maybe-show-diff' option to enable displaying a diff of the changes to be committed in a window. This is like the 'C-c C-d' command in Log Edit mode buffers, except that it does not select the "*vc-diff*" buffer's window, and so works well when added to 'log-edit-hook'.

'vc-rename-file' is now bound to 'C-x v R'.

'vc-annotate' now abbreviates the Git revision in more cases.

In Emacs 30, 'vc-annotate' gained the ability to abbreviate the Git revision in the buffer name. Now, it also abbreviates the Git revision when visiting other revisions, such as with 'vc-annotate-revision-previous-to-line'.

New buffer-local variable 'vc-buffer-overriding-fileset'.

Primarily intended for buffers not visiting files, this specifies the VC backend and VCS-managed file name or file names to which the buffer's contents corresponds. It overrides the behavior of 'vc-deduce-fileset'. This replaces and generalizes the old 'vc-annotate-parent-file'.

New buffer-local variable 'vc-buffer-revision'.

This specifies the revision to which the buffer's contents corresponds. This replaces and generalizes the old 'vc-annotate-parent-rev'.

vc-dav.el is now obsolete.

The 'log-incoming' and 'log-outgoing' functions are deprecated.

Backend authors should implement the 'incoming-revision' and 'mergebase' backend functions instead. These are jointly sufficient to support the 'C-x v I' and 'C-x v O' commands.

Marking revisions in Log View now works more like other modes.

Previously, 'm' toggled whether the current revision was marked, and didn't advance point. Now 'm' only adds marks, 'u' removes marks, and both advance point, like how marking works in Dired and VC Directory. You can get back the old behavior with something like this:

```
(with-eval-after-load 'log-view (keymap-set log-view-mode-map "m"
#'log-view-toggle-mark-entry))
```

In addition, a new command 'U' removes all marks.

New commands 'M-RET', 'M-p' and 'M-n' in Log View mode.

'M-RET' expands the current entry, if relevant, and displays its diff in another window. 'M-p' and 'M-n' move to the previous and next entries, respectively, expand them if relevant, and display their diffs.

You can use these three commands together to more easily view all the log entries and diffs of a series of revisions: use 'M-RET' on the first revision, then either 'M-n' or 'M-p' repeatedly to view the others.

New command 'w' in Log View mode.

The new command 'log-view-copy-revision-as-kill', bound to 'w' in Log View mode, copies to the kill ring the ID of the revision at point in the log entry. If there are marked revisions, it copies the IDs of those, instead.

New commands 'vc-print-change-log' and 'vc-print-root-change-log'.

These are just like 'vc-print-log' and 'vc-print-root-log' except that they have a different prefix argument that some users may prefer. With a prefix argument, these commands prompt for a branch, tag or other reference to a revision to log, and a maximum number of revisions to print. If you find this prefix argument more useful, or more mnemonic, than the prefix arguments that 'vc-print-log' and 'vc-print-root-log' already have, consider replacing the default global bindings, like this:

```
(keymap-global-set "C-x v l" #'vc-print-change-log) (keymap-global-set "C-x v L"
#'vc-print-root-change-log)
```

New command alias 'vc-restore' for 'vc-revert'.***The 'diff-restrict-view' command is disabled by default.***

This command is Diff mode's specialized 'narrow-to-region'. 'narrow-to-region' has long been disabled by default, so for consistency, 'diff-restrict-view' is now too. To enable it again, use 'enable-command'.

'C-x v !' has its own input history.***'C-x v +' for Git pulls from a configured push remote.***

If the current branch has a configured push remote, the default arguments to 'git pull' will cause a pull from the push remote. You can use 'C-u C-x v +' to preview or change the arguments.

'C-u C-x v +' and 'C-u C-x v P' for Git have an input history.

This was already in place for Mercurial.

'vc-hg-working-revision' now returns changeset hashes.

Previously, it returned local revision numbers, but hashes are more robust for how this function is typically used.

Package

No longer warn if a package has no footer line.

package.el no longer warns for packages without a "footer line", which is the line that usually appears at the very end of an Emacs Lisp file:

```
;;; FILENAME ends here
```

New optional argument to 'package-autoremove'.

An optional argument NOCONFIRM has been added to 'package-autoremove'. If it is non-nil, or when invoked with a prefix argument, 'package-autoremove' will not prompt the user for confirmation before removing packages.

New prefix argument for 'package-install-selected-packages'.

When invoked with a prefix argument, 'package-install-selected-packages' will not prompt the user for confirmation before installing packages.

'package-refresh-contents' runs asynchronously.

Refreshing the package index will no longer block when invoked interactively.

'package-upgrade' no longer accepts a string argument.

When called from Lisp, it now only accepts a symbol.

'package-install-from-buffer' respects files marked by DireD.

When invoking the command in a DireD buffer with marked files, the command will only copy those files.

'package-isolate' can now also install packages.

If a package is missing, 'package-isolate' will fetch the missing tarballs and prepare them to be activated in the sub-process.

package-x.el is now obsolete.

The command 'package-vc-install-from-checkout' is now obsolete.

Please use the User Lisp directory instead, see Info node "(emacs) User Lisp Directory" for more details on how to use. This also means that combining the 'use-package' keywords ':vc' and ':load-path' is obsolete.

Package menu now highlights packages marked for installation or deletion.

Package menu now displays the total number of the package type.

Package menu now displays in the mode line the total number of packages installed, total number of packages from all the package archives, total number of packages to upgrade and total number of new packages available.

New functions to query builtin package information.

`'package-versioned-builtin-packages'` returns a list of symbols of built-in packages; `'package-builtin-package-version'` returns the version-list of a given package symbol. These functions provide public interfaces for external tools to query information about built-in packages.

Uninstalling a package now removes its directory from 'load-path'.

Packages can be reviewed before installation or upgrade.

The user option `'package-review-policy'` can configure which packages the user should be allowed to review before any processing takes place. The package review can include reading the downloaded source code, presenting a diff between the downloaded code and a previous installation or displaying a ChangeLog.

New command 'package-autosuggest'.

Using a built-in database of ELPA package suggestions, this command will install viable packages if no specific major mode is available for the contents of the current buffer.

New minor mode 'package-autosuggest-mode'.

When enabled, this displays indications about the availability of add-on ELPA packages for the current buffer and suggestions for installing those packages. The default is to show a button on the mode line that can be used to install such packages, but you can customise the presentation style of these suggestions using `'package-autosuggest-style'`.

New user option 'package-retention-policy'.

This user option controls what previous packages versions to keep on upgrade. By default, this is set to nil, to keep the previous behavior.

Packages are now checked for recursive dependencies before installing.

If a package has dependencies not available on any of the archives in `'package-archives'`, it will appear as unavailable, with the reasons stated in its description.

Rcirc

Authentication via NickServ can access passwords with auth-source.el.

For details, consult `'rcirc-authinfo'`.

Xref

Xref commands that jump to some location use 'display-buffer'.

The commands that jump to some location use `'display-buffer'` and specify the category `'xref-jump'`. As a result, you can customize how the destination window is chosen using `'display-buffer-alist'`. Example:

```
(setq display-buffer-alist '(((category . xref-jump) (display-buffer-reuse-window
display-buffer-use-some-window) (some-window . mru))))
```

New minor mode 'xref-mouse-mode'.

This minor mode binds `'xref-find-definitions-at-mouse'` to `'C-<down-mouse-1>'`, allowing you to "control click" to jump to a definition, following the convention from other editors. The global minor mode `'global-xref-mouse-mode'` will enable this in all buffers.

New command 'xref-change-to-xref-edit-mode'.

It is bound to `'e'` and it switches an Xref buffer into an "editable" mode like similar features in Occur and Grep buffers.

Revert

Variable 'revert-buffer-in-progress' has been renamed.

The old name, `'revert-buffer-in-progress-p'`, is kept as obsolete variable alias.

Symbol names with a trailing `'-p'` are reserved for predicates. Calling a variable like this was a mistake.

Autorevert

New variable 'inhibit-auto-revert-buffers'.

While a buffer is member of this variable, a list of buffers, auto-reverting of this buffer is suppressed.

New macro 'inhibit-auto-revert'.

This macro adds the current buffer to `'inhibit-auto-revert-buffers'`, runs its body, and removes the current buffer from `'inhibit-auto-revert-buffers'` afterwards.

New variable 'auto-revert-buffer-in-progress'.

`'auto-revert-buffer'` binds this variable to a non-nil value while it is working. This can be used by major mode `'revert-buffer-function'` implementations to suppress messages in Auto Revert modes, for example.

Strokes

'strokes-mode' no longer demands the presence of a mouse.

`'strokes-mode'` now permits itself to be enabled if no mouse is connected, to facilitate enabling `'strokes-mode'` in sessions where the availability of a mouse device varies during execution (as is frequently observed on Android).

Yank Media

'yank-media' now auto-selects the most preferred MIME type.

Major-mode authors can customize the variables `'yank-media-autoselect-function'` and/or `'yank-media-preferred-types'` to change the selection rules.

Remember

Remember mode is now a minor mode.

The `'remember'` command enables the major mode set in `'remember-initial-major-mode'` and then the `'remember-mode'` minor mode in the `'remember-buffer'`. This allows users to customize the major mode used to write notes.

New handler that appends remember data in directory.

The `'remember-append-in-data-directory'` handler appends remember data in a file, that file being chosen by the user through the minibuffer.

New prefix map for remember commands.

Meant to be given a global binding convenient to the user. Example:

```
(keymap-global-set "C-c r" 'remember-prefix-map)
```

Speedbar

New commands for Speedbar.

- `'speedbar-window-mode'` opens Speedbar in a window instead of a frame. - `'speedbar-window'` is an alias for `'speedbar-window-mode'`.

New user options for Speedbar.

- `'speedbar-prefer-window'` tells `'speedbar'` to open a side window instead of a frame. - `'speedbar-window-dedicated-window'` defines whether `'speedbar'` is displayed in a dedicated window. - `'speedbar-window-default-width'` defines the initial width of the `'speedbar-window'`. - `'speedbar-window-max-width'` defines the maximum width of the `'speedbar-window'` when it is closed and then restored.

'speedbar-easymenu-definition-trailer' is now a function.

Icomplete

Change in meaning of 'icomplete-show-matches-on-no-input' (again).

For Emacs 28 to Emacs 30, when `'icomplete-show-matches-on-no-input'` was non-nil, `'RET'` had special behavior when the minibuffer's contents was equal to the initial input it had right after minibuffer activation. In that case, `'RET'` would choose the first completion candidate, if there was one, instead of the minibuffer's default value. `'RET'` has now returned to selecting the default value in this case; you can use `'C-j'` to choose the completion under point instead. You can opt back in to the special behavior of `'RET'` like this:

```
(keymap-set icomplete-minibuffer-map "<remap> <minibuffer-complete-and-exit>"
#'icomplete-ret)
```

New user options for 'icomplete-vertical-mode'.

New user options have been added to enhance 'icomplete-vertical-mode':

- 'icomplete-vertical-in-buffer-adjust-list' aligns in-buffer completion to the original cursor column.
- 'icomplete-vertical-render-prefix-indicator' adds a prefix indicator to completion candidates.
- 'icomplete-vertical-selected-prefix-indicator' specifies the prefix string for the selected candidate.
- 'icomplete-vertical-unselected-prefix-indicator' specifies the prefix string for unselected candidates.

New faces for 'icomplete-vertical-mode'.

New faces have been added to 'icomplete-vertical-mode':

- 'icomplete-vertical-selected-prefix-indicator-face' controls the appearance of the selected candidate prefix.
- 'icomplete-vertical-unselected-prefix-indicator-face' controls the appearance of unselected candidate prefixes.

Custom

New function 'custom-initialize-after-file-load'.

Useful to delay initialization to the end of the file, so it can use functions defined later than the variable, as is common for minor modes. 'define-globalized-minor-mode' now automatically uses it if the ':init-value' is non-nil.

New major mode 'Custom-dirlocals-mode'.

This is intended for customizing directory-local variables in the current directory's ".dir-locals.el" file.

New keybinding 'C-c C-k' for 'Custom-reset-standard'.

New command 'Custom-goto-first-choice' (bound to 'C-c C-i').

When first opening the customization interface for a user option, you can use this command as a shortcut to jump to the first actionable button or field (for instance an on/off button for boolean options, or a text field for other values).

Pulse

New function 'pulse-faces'.

This function pulses a specified list of faces. The pulse duration is determined by the new user option 'pulse-face-duration'.

Miscellaneous

The first client frame now shows warnings from daemon startup.

When there are warnings emitted during Emacs startup, usually due to problems in your initialization file, these are shown in a `"*Warnings*"` buffer. Until now such warnings were not made visible in the case that Emacs was started as a daemon. Now the first frame after daemon startup will show the `"*Warnings*"` buffer. So for example, starting Emacs with a command like `'emacsclient -a"" -c'` will now show `"*Warnings*"` just like a plain invocation of `'emacs'` would.

'tooltip-mode' now shows tooltips after delay on TTY frames.

Display of tooltips on text-only terminals now happens after `'tooltip-delay'` as it does on GUI terminals. To get back the old behavior, customize the value of `'tooltip-delay'` to zero.

New user option 'follow-mode-prefix-key'.

This user option replaces `'follow-mode-prefix'`, which had to be set before loading Follow mode. This new option allows you to change the prefix even after it was loaded, using `'customize-option'` or `'setopt'`.

cdl.el is now obsolete.

Use `'shell-command'` and `'shell-command-on-region'` instead.

echistory.el is now obsolete.

hashcash.el is now obsolete.

It is believed to no longer be useful as a method to fight spam.

kermi.el is now obsolete.

New user option 'ns-click-through' on Nextstep (GNUstep/Mac OS).

This controls whether activation clicks are passed through to Emacs commands. When nil, clicking on an inactive Emacs frame will only activate it. When t (the default), the click will both activate the frame and be interpreted as a command.

New user option 'global-hl-line-buffers'.

This specifies in which buffers should the `'global-hl-line-mode'` be switched on. The default is all buffers except the minibuffer and the buffers like the completions buffer that enable `'cursor-face-highlight-mode'`.

New value 'window' for the user option 'global-hl-line-sticky-flag'.

Unlike the value t that highlights the line with buffer's point, it highlights the line with window's point. Also it uses the new face `'hl-line-nonselected'` for highlighting the line with window's point in non-selected windows.

New user option 'display-fill-column-indicator-warning'.

Customize it to a non-nil value to have the fill-column indicators change their face if the current line exceeds the `'fill-column'`. The new face `'display-fill-column-indicator-warning-face'` is used to highlight the fill-column

indicators. By default, this is disabled.

New function 'flash-face-bell-function'.

This function flashes a face briefly. It is intended to be used in 'ring-bell-function'.

New function 'flash-echo-area-bell-function'.

This function flashes the current echo area briefly. It is intended to be used in 'ring-bell-function'.

New user option 'pulse-face-duration'.

This option controls the flash duration for 'flash-face-bell-function' and 'flash-echo-area-bell-function'.

New user option 'flash-face-faces'.

This option tells 'flash-face-bell-function' which faces should flash.

New user option 'flash-face-attributes'.

This option tells 'flash-face-bell-function' and 'flash-echo-area-bell-function' which face attributes should be used for flash.

'face-all-attributes' now accepts an optional argument INHERIT.

It has the same meaning as the INHERIT argument to 'face-attribute', which already takes this argument for a single attribute. This is useful when you want the face attributes to be absolute and not 'unspecified'.

New user option 'ffap-prefer-remote-file'.

If non-nil, FFAP always finds remote files in buffers with remote 'default-directory'. If nil, FFAP finds local files first for absolute file names in above buffers. The default is nil.

New theme 'newcomers-presets'.

This new theme configures user options and minor modes that might interest new users, but would otherwise be too invasive to enable by default. Recall that an Emacs theme is a generic collection of settings, that doesn't have to affect the appearance. See the Info node "(emacs) Newcomers Theme" for more information.

Edebug

New command 'edebug-bounce-to-previous-value' (bound to 'P').

This command temporarily displays the outside current buffer with the outside point corresponding to the previous value, where the previous value is what Edebug has evaluated before its last stop point or what the user has evaluated in the context outside of Edebug.

This replaces the binding of command `'edebug-view-outside'` to `'P'`, which is still available on `'v'`.

Flymake

Enhanced `'flymake-show-diagnostics-at-end-of-line'`.

The new value `'fancy'` allowed for this user option will attempt to layout diagnostics below the affected line using unicode graphics to point to diagnostic locus.

Enhanced `'flymake-show-buffer-diagnostics'`.

The command `'flymake-show-buffer-diagnostics'` is now capable of highlighting a nearby diagnostic in the resulting listing. Additionally, it is bound to mouse clicks on fringe and margin indicators, operating on the diagnostics of the corresponding line. The user may bind it in other situations such as the diagnostic overlay map.

More powerful `'flymake-make-diagnostic'` API.

Flymake backends can now specify origin and code attributes, allowing Flymake and other extensions to segregate diagnostics based on this extended information.

New user option `'flymake-diagnostic-format-alist'`.

This provides fine-grained control over diagnostic formatting across different contexts, allowing you to specify which components (origin, code, message or one-liner message) appear in each output destination.

Dynamic column sizing in diagnostic listings.

The tabulated listings produced by `'flymake-show-buffer-diagnostics'` and `'flymake-show-project-diagnostics'` now automatically adjust their column widths based on content, optimizing display space and readability.

New value `'auto'` of user option `'flymake-indicator-type'`.

This value (set by default) tries to use fringes if possible, otherwise falls back to margins.

New user option `'elisp-flymake-byte-compile-executable'`.

This allows customizing the Emacs executable used for Flymake byte compilation in `'emacs-lisp-mode'`. This option should be set when editing Lisp code which will run with a different Emacs version than the running Emacs, such as code from an older or newer version of Emacs. This will provide more accurate warnings from byte compilation.

SQLite

SQLite databases can now be opened in read-only mode.

The new optional argument `READONLY` of function `'sqlite-open'` allows to open an existing database only for reading.

'sqlite-open' now recognizes 'file://' URIs as well as file names.

The `'file://'` URIs are supported by default. In the unusual case that a normal file name starts with `"file:"`, you can disable the URI recognition by invoking `'sqlite-open'` with the new optional argument `DISABLE-URI non-nil`.

'sqlite-close' now does nothing if the connection is already closed.

GUD

'pdb', 'perldb', and 'guiler' suggest debugging the current file via 'M-n'.

When starting these debuggers (e.g., `'pdb'`) while visiting a file, pressing `'M-n'` in the command prompt suggests a command line including the file name, using the minibuffer's `"future history"`.

Calendar

New command 'calendar-recenter'.

This command recenters the month of the date at point. By default, it is bound to `'C-l'` in the calendar buffer.

Mouse wheel bindings for scrolling the calendar.

You can now use the mouse wheel to scroll the calendar by 3 months. With the shift modifier, it scrolls by one month. With the meta modifier, it scrolls by year.

Simpler key bindings for navigation in calendar by months and by years.

The month and year navigation key bindings `'M-}'`, `'M-{'`, `'C-x ]'` and `'C-x ['` now have the alternative keys `'}'`, `'{'`, `']'` and `'['`.

Avoid modifying Calendar's user options.

The user options `'calendar-mark-holidays-flag'` and `'calendar-mark-diary-entries-flag'` are not modified anymore when changing the marking state in the calendar buffer.

New library for iCalendar data.

A new library has been added to the calendar for handling iCalendar (RFC 5545) data. The library is designed for reuse in other parts of Emacs and in third-party packages. Package authors can find the new library in the Emacs distribution under `"lisp/calendar/icalendar-*.el"`.

Most of the functions and variables in the older `icalendar.el` have been marked obsolete and now suggest appropriate replacements from the new library. `diary-icalendar.el` provides replacements for the diary-related features from `icalendar.el`; see below.

Diary

New user option 'diary-date-insertion-form'.

This user option determines how dates are inserted into the diary by Lisp functions. Its value is a pseudo-pattern of the same type as in `'diary-date-forms'`. It is used by `'diary-insert-entry'` when inserting entries from the calendar, or when importing them from other formats.

New library `diary-icalendar.el`.

This library reimplements features previously provided by `icalendar.el`: import from iCalendar format to the diary, and export from the diary to iCalendar. It also adds the ability to include iCalendar files in the diary and display and mark their contents in the calendar without importing them to the diary file. The library uses the new iCalendar library (see above) and makes diary import and export more customizable.

New commands to display more months in the calendar.

`'calendar-show-more-months'` and `'calendar-show-fewer-months'` display more or fewer months in the calendar, respectively. The calendar shows at most 12 months and at least 3 months.

'calendar-scroll-left-three-months' and its variants are obsolete aliases.

Because the calendar can display more than three months now, commands `'calendar-scroll-left-three-months'` and `'calendar-scroll-right-three-months'` have been renamed to `'calendar-scroll-calendar-left'` and `'calendar-scroll-calendar-right'`. The old names are kept as obsolete aliases.

Calc

New user option `'calc-string-maximum-character'`.

Previously, the `'calc-display-strings'`, `'string'`, and `'bstring'` functions only considered integer vectors whose elements are all in the Latin-1 range 0-255. This hard-coded maximum is replaced by

`'calc-string-maximum-character'`, and setting it to a higher value allows the display of matching vectors as Unicode strings. The default value is 0xFF or 255 to preserve the existing behavior.

New user option `'calc-inhibit-startup-message'`.

If it is non-nil, inhibit Calc from printing its startup message. The default value is nil to preserve the existing behavior.

Time

New user option `'world-clock-sort-order'`.

This option controls the order of timezone entries in the `'world-clock'`. By default, no sorting is done, and entries appear in the same order as in `'world-clock-list'`. Any format understood by `'format-time-string'` can be used to specify a key for the sort order, which is updated upon each refresh. The sort direction can be controlled by

using a cons cell of a format string and a boolean. Alternatively, a sorting function can be provided directly.

New user option 'display-time-help-echo-format'.

This option controls the format of the help echo when hovering over the time.

Fill

New variable 'fill-region-as-paragraph-function'.

The new variable `'fill-region-as-paragraph-function'` provides a way to override how functions like `'fill-paragraph'` and `'fill-region'` fill text. Major modes can bind this variable to a function that fits their needs. It defaults to `'fill-region-as-paragraph-default'`.

'report-emacs-bug' now checks whether the bug report is about Org.

The command `'report-emacs-bug'` looks in the report text for symbols that indicate problems in Org, and if found, will ask whether the bug report is actually about Org (in which case users should use the Org-specific command for reporting bugs).

The elint.el package is now obsolete.

Use the byte-compiler instead; it provides more and more useful warnings.

Newsticker

New user option 'newsticker-hide-old-feed-header'.

It controls whether to automatically hide the header of feeds whose items are all old or obsolete in the plainview `"*newsticker*" buffer`. This is only visually interesting if the content of those feeds are also hidden (see `'newsticker-hide-old-items-in-newsticker-buffer'` and `'newsticker-show-descriptions-of-new-items'`).

New commands to hide and show headers of old newsticker feeds.

The new commands `'newsticker-hide-old-feed-header'` and `'newsticker-show-old-feed-header'`, bound to `'h h'` and `'s h'` keys, respectively, hide and show the headers of feeds whose items are all old or obsolete.

CPerl mode

Syntax of Perl up to version 5.42 is supported.

CPerl mode creates imenu entries for `":writer"` generated accessors and recognizes the new functions `"all"` and `"any"`. See <https://perldoc.perl.org/5.42.0/perldelta> for details.

Zone

Zone can scramble multiple windows across multiple frames; it may also reorganize frames to be a single window. As before, when a key or mouse event occurs, all of the frames and windows are restored to their original state. This is controlled by three new user options which control the use of frames and windows beyond the currently active ones. It identifies suitable buffers for zoning out so that potentially important buffer contents are not exposed.

New user option 'zone-delete-other-windows'.

When non-nil, the frame is made into a single full frame window to hold the zoned buffer. If all frames were to be used ('zone-all-frames' set to non-nil), then all frames are converted to single window frames.

New user option 'zone-all-frames'.

When non-nil, Zone will appear on all visible frames. While the buffer scrambling will appear on each frame, it will be the same buffer so they will all behave the same way.

New user option 'zone-all-windows-in-frame'.

When non-nil, the zoned buffer will be mapped to all of the windows present on the frame. If the option is nil, then only the selected window will show the zoned buffer. Note, however, that each window holding the zoned buffer is showing the same zoned buffer.

New variable 'zone-ignored-buffers'.

This variable is a list of criteria for excluding a buffer from consideration as the source of zoning. The list has entries that are tested against each buffer until a suitable one is found. The criteria can be a symbol that ends in '-mode' which excludes buffers that are in a mode derived from the specified mode. It may also be a function-bound symbol or lambda expression that is called with a buffer that returns a non-nil value if it should not be the Zone source. Finally, an entry can also be a regular expression that must not match the buffer's name.

Initially, the list excludes buffers in 'special-mode', in 'image-mode', contains an encrypted file, is an empty buffer, is a hidden buffer, or is the "**scratch**" buffer. If it cannot locate any acceptable buffers, it will begrudgingly use the scratch buffer.

Abbrev mode

You can now enable Abbrev mode by default using Easy Customization.

Customize the user option 'abbrev-mode' to non-nil to enable Abbrev mode by default in all buffers.

Antlr mode

Variable 'antlr-tool-version' is no user option anymore.

If becomes buffer-local and has the symbol `'antlr-v2'`, `'antlr-v3'` or `'antlr-v4'` as value. The value determines which of the tool-version dependent customization options are used. For example, the command `'antlr-run-tool'` uses the option `'antlr-v3-tool-command'` (with default value `"java org.antlr.Tool"`) when `'antlr-tool-version'` has the value `'antlr-v3'`.

Command `'antlr-mode'` now also works on ANTLR v3 or v4 grammars.

If the variable `'antlr-tool-version'` is not set locally, e.g., by the file's local variables specs, the command sets its local value to `'antlr-v2'` if a keyword `"class"` or `"header"` appears at the beginning of the source, or to `'antlr-v3'` otherwise.

New command `'antlr-v4-mode'` is a derived mode of `'antlr-mode'`.

It sets `'antlr-tool-version'` to value `'antlr-v4'`, and is automatically used for files with extension `".g4"`.

The variable `'antlr-language'` is now used more generally.

The variable has a symbol as value which determines which of the language-dependent customization options are used. These options influence font-locking and indentation commands. Its value is usually set according to the grammar option `"language"`, see the default values of `'antlr-v2-language-list'`, `'antlr-v3-language-list'` and `'antlr-v4-language-list'`. This mode now supports C, Delphi, JavaScript, ObjC, Python and Ruby, additional to Java and Cpp.

New user option `'antlr-run-tool-on-buffer-file'`.

Command `'antlr-run-tool'` now usually runs on the file for the current buffer. Customize this user option to nil to get the previous behavior back.

Hi Lock

Use active region for default values in more functions.

If an active region exists, the commands `'hi-lock-line-face-buffer'` and `'hi-lock-face-phrase-buffer'` now use its contents as their default value. Previously, only `'hi-lock-face-buffer'` supported this.

Shadowfile

`'shadow-info-buffer'` and `'shadow-todo-buffer'` use ephemeral buffer names now.

Display Battery mode

UPower battery status can update automatically without polling.

On systems where the user option `'battery-status-function'` is set to `'battery-upower'`, it is now possible to get battery status updates on the mode line without polling for changes every `'battery-update-interval'` seconds. Setting this user option to nil means the mode line will update only when the battery power state, percentage, or presence in the bay changes.

Etags Regen mode

Tags table is not created during completion anymore.

Previously, when there was no tags table loaded and the default completion function was called, `'etags-regen-mode'` ensured that tags were created. This has been disabled, and the new user option `'etags-regen-create-on-completion'` can be used to enable it again.

New Modes and Packages in Emacs 31.1

New major mode `'icalendar-mode'`.

A major mode for displaying and editing iCalendar (RFC 5545) data. This mode handles line unfolding and fontification, including highlighting syntax errors in invalid data.

New minor mode `'delete-trailing-whitespace-mode'`.

A simple buffer-local mode that runs `'delete-trailing-whitespace'` before saving the buffer.

New major mode `'conf-npmrc-mode'`.

A major mode based on `'conf-mode'` for editing `".npmrc"` files.

New major modes based on the tree-sitter library

New major mode `'mhtml-ts-mode'`.

An optional major mode based on the tree-sitter library for editing HTML files. This mode handles indentation, fontification, and commenting for embedded JavaScript and CSS.

New major mode `'go-work-ts-mode'`.

A major mode based on the tree-sitter library for editing `"go.work"` files. If tree-sitter is properly set-up by the user, it can be enabled for files named `"go.work"`.

New package `'lua-mode'`.

The `'lua-mode'` package from NonGNU ELPA is now included in Emacs.

New library `'timeout'`.

This library provides functions to throttle or debounce Emacs Lisp functions. This is useful for corraling overeager code that is slow and blocks Emacs, or does not provide ways to limit how often it runs.

New mode `'system-taskbar-mode'`.

This is a global minor mode and companion functions that integrate Emacs with system GUI taskbars (also called docks or launchers or something similar) to display a taskbar

icon `"badge"` overlay, a progress bar report overlay, alert the user that an Emacs session needs attention, often by flashing or bouncing the Emacs application icon. Supported capable systems are GNU/Linux via D-Bus, macOS/GNUstep 10.5+, MS-Windows 7+.

On GNU/Linux systems, shell extensions or similar helpers such as `"dash-to-dock"` may be required. See <https://extensions.gnome.org/extension/307/dash-to-dock/> and <https://wiki.ubuntu.com/Unity/LauncherAPI>.

New package `'system-sleep'`.

This package provides platform-neutral interfaces to block your system from entering idle sleep and a hook to process pre-sleep and post-wake events. You can use this to avoid the system entering an idle sleep state and interrupting a long-running process due to lack of user activity. The sleep event hook lets you, for example, close external connections or serial ports before sleeping, and reestablish them when the system wakes up.

Supported capable systems are GNU/Linux via D-Bus (sleep blocking and sleep events require the `org.freedesktop.login1` service, display sleep blocking requires `org.freedesktop.Screensaver` service), macOS (sleep/display blocking requires version 10.9 or later, sleep events are supported on all versions), MS-Windows (sleep blocking is supported on all versions, sleep events require Windows 8 or later).

Incompatible Lisp Changes in Emacs 31.1

Boundaries of `'cursor-sensor-functions'` now obey stickiness.

`'cursor-sensor-mode'` now uses `'get-pos-property'` to decide whether a boundary is considered as inside or outside.

This means that by default, the boundaries have changed: the end position of a stretch of a `'cursor-sensor-functions'` text property used to be considered outside of the stretch whereas it is now considered as inside. You can recover the previous behavior by controlling the stickiness, for example with a call like:

```
(add-text-properties BEG END '(cursor-sensor-functions (MY-FUNCTION) rear-nonsticky
(cursor-sensor-functions)))
```

`'makunbound'` on a variable alias undoes the alias.

Previously, it had the effect of applying the `'makunbound'` on the target of the alias (which can fail on some builtin variables).

`'F00-ts-mode-indent-offset'` renamed to `'F00-ts-indent-offset'`.

When the new TS modes were introduced, a mistake was made where those modes used `'F00-mode-indent-offset'` instead of the previous conventional `'F00-indent-offset'`. The

following names are the new names: 'toml-ts-indent-offset', 'mhtml-ts-js-css-indent-offset', 'html-ts-indent-offset', 'typescript-ts-indent-offset', 'php-ts-indent-offset', 'php-ts-html-indent-offset', 'json-ts-indent-offset', 'java-ts-indent-offset', 'go-ts-indent-offset', 'csharp-ts-indent-offset', 'cmake-ts-indent-offset', 'c-ts-indent-offset'.

String mutation has been restricted further.

'aset' on unibyte strings now requires the new character to be a single byte (0-255). On multibyte strings the new character and the character being replaced must both be ASCII (0-127).

These rules ensure that mutation will never transform a unibyte string to multibyte, and that the size of a string in bytes (as reported by 'string-bytes') never changes. They also allow strings to be represented more efficiently in the future.

Other functions that use 'aset' to modify string data, such as 'subst-char-in-string' with a non-nil INPLACE argument, will signal an error if called with arguments that would violate these rules.

More program constants are combined by the compiler.

The compiler now unifies more constants that are 'equal' for better code generation. This does not affect correct programs but may expose some coding mistakes. For example,

```
(eq (cdr '(1 2 3)) '(2 3)))
```

may return either nil or t.

Nested backquotes are not supported any more in Pcase patterns.

The obsolete variable 'redisplay-dont-pause' has been removed.

The 'rx' category name 'chinese-two-byte' must now be spelled correctly.

An old alternative name (without the first 'e') has been removed.

'text-property-default-nonsticky' is now buffer-local.

This variable now becomes buffer-local when set. Use 'setq-default' in (the unlikely) case you want to change the global value.

All the digit characters now have the 'digit' category.

All the characters whose Unicode general-category is Nd now have the 'digit' category, whose mnemonic is '6'. This includes both ASCII and non-ASCII digit characters.

All the symbol characters now have the 'symbol' category.

All the characters that belong to the `'symbol'` script (according to `'char-script-table'`) now have the `'symbol'` category, whose mnemonic is `'5'`.

Some libraries obsolete since Emacs 24.4 and 24.5 have been removed:

`cc-compat.el`, `info-edit.el`, `meese.el`, `otodo-mode.el`, `rcompile.el`, `sup-mouse.el`, `terminal.el`, `vi.el`, `vip.el`, `ws-mode.el`, and `yow.el`.

`'if-let'` and `'when-let'` are now obsolete.

Use `'if-let*'`, `'when-let*'` and `'and-let*'` instead.

This effectively obsoletes the old `'(if-let (SYMBOL SOMETHING) ...)'` single binding syntax, which we'd kept only for backwards compatibility.

The Eshell `'pwd'` command now expands the directory name on all systems.

This ensures that user directories are properly expanded to their full name. Previously, Eshell only did this for MS-Windows systems. To restore the old behavior, you can set `'eshell-pwd-convert-function'` to `'identity'`.

The rx `'eval'` form now uses the current Elisp dialect for evaluation.

Previously, its argument was always evaluated using dynamic binding.

Unused block comment variables have been removed.

The unused variables `'block-comment-start'` and `'block-comment-end'`, which never actually had any effect when set by major modes, have been removed.

`'delete-frame'` now needs non-nil FORCE argument to delete daemon frame.

The initial terminal frame of an Emacs process running as daemon can be deleted via `'delete-frame'` if and only if its optional FORCE argument is non-nil.

`'date-to-time'` no longer accepts malformed times with time zone like "EDT".

Time strings like `"2025-06-04T13:21:00 EDT"` are not in valid ISO 8601 time format, and `'date-to-time'` now signals an error for them. Use a numerical time-zone specification, like `"2025-06-04T13:21:00-0400"`, instead, which gives the time offset as `+/-hh` or `+/-hh:mm`. A designator `"Z"` for UTC time is also supported. Less formal space-separated time formats, like `"2025-06-04 13:21:00 EDT"`, without the ISO 8601 `"T"` separator, are also supported.

The obsolete variable `'load-convert-to-unibyte'` has been removed.

The experimental variable `'binary-as-unsigned'` has been removed.

Instead of `'(let ((binary-as-unsigned t)) (format "%x" N))'` you can use `'(format "%x" (logand N MASK))'` where MASK is for the desired word size, e.g., `#x3fffffffffffffff` for typical Emacs fixnums.

The 'exec-path' variable now uses same default PATH as other programs.

That is, if the PATH environment variable is unset or empty, 'exec-path' now acts as if PATH is the system default, which is `"/bin:/usr/bin"` on GNU/Linux systems.

New variable 'tty-cursor-movement-use-TAB-BS'.

The display optimization where the combination 'TAB' characters + 'BACKSPACE' is used to move to a position on a TTY frame is now disabled by default and controlled by this variable; it can be set to non-nil to keep the old behavior. This change is to accommodate screen readers.

'next-completion' and 'previous-completion' now use 'completions-format'.

Previously, these commands only took horizontal format into account; now, they call either `'{next,previous}-line-completion'` or the new commands `'{next,previous}-column-completion'`, depending on the value of `'completions-format'`. The latter two commands improve and extend the previous implementations of `'{next,previous}-completion'`, which better reflect that they only take the (default) horizontal completions format into account. Any external code using `'{next,previous}-completion'`, that assumes the previous implementation, must be adjusted accordingly; see `'minibuffer-next-completion'` for an example of such an adjustment in Emacs core.

A thread's current buffer can now be killed.

We introduce a new attribute for threads called `"buffer disposition"`. See the new argument in `'make-thread'` as well as the `'thread-buffer-disposition'` and `'thread-set-buffer-disposition'` functions. The default value allows the thread's current buffer to be killed by another thread. This does not apply to the main thread's buffer.

Defining or modifying a face so that it inherits from itself signals error.

Calling any function that defines or modifies a face in a way that causes cyclical inheritance (i.e., the face inherits from itself, either directly or indirectly) now signals an error. Previously, Lisp programs could get away with this, and the problem would either be detected at display time or even cause Emacs to hang trying to display such a face. Affected APIs include `'defface'`, `'set-face-attribute'`, their callers, and other similar functions.

Original behavior of 'overlays-in' and 'overlays-at' has been restored.

Before Emacs 28.1, the list of overlays returned by these two functions included overlays outside of the current narrowing of the buffer, and there wasn't a special exception for including empty overlays at end of accessible portion of the buffer. This behavior has been restored, and the special behavior for empty overlays is again reserved only to the actual end of buffer, disregarding the narrowing. As result, `'remove-overlays'` can now again remove overlays outside of the narrowing, as it did

before Emacs 28.1.

'help-setup-xref' now re-enables the major mode of the Help buffer.

As a result, in many cases the buffer will be read-only afterwards. This should not cause any trouble as long as the actual buffer modification takes place inside 'with-help-window' or 'with-output-to-temp-buffer' after the call to 'help-setup-xref'.

Xref commands don't automatically suggest to visit a tags table anymore.

When no tags file is loaded, symbol completion just won't provide any suggestions. So the 'M-?' command now works without a tags table. And the 'M-.' command will show a message describing the several built-in options that will provide an Xref backend when used.

Calling 'debug' in batch sessions no longer kills Emacs.

If you want Emacs to exit, your program will now have to call 'kill-emacs' explicitly.

Lisp Changes in Emacs 31.1

The API to manipulate error descriptors has been improved.

There are new functions: 'error-type-p', 'error-type', 'error-has-type-p', and 'error-slot-value'. And you can now do '(signal err)' instead of '(signal (car err) (cdr err))', which is not only more concise but also preserves the 'equality of the error descriptor.

'secure-hash' now supports generating SHA-3 message digests.

The list returned by 'secure-hash-algorithms' now contains the symbols 'sha3-224', 'sha3-256', 'sha3-384', and 'sha3-512'. These symbols can be used as the ALGORITHM argument of 'secure-hash' to generate SHA-3 hashes.

New function 'garbage-collect-heapsize'.

Same as 'garbage-collect' but just returns the info from the last GC without performing a collection.

Improve 'replace-region-contents' to accept more forms of sources.

It has been promoted from subr-x.el to the C code. You can now directly pass it a string or a buffer rather than a function. Actually passing it a function is now deprecated.

New function 'char-displayable-on-frame-p'.

'char-displayable-on-frame-p' returns non-nil if Emacs ought to be able to display its char argument on a given frame. This new function, unlike 'char-displayable-p', does not check whether the character can be encoded by the underlying terminal.

New function `'frame-initial-p'`.

This predicate returns non-nil if a given frame or terminal is or holds, respectively, the initial text frame that is used internally during daemon mode, batch mode, and the early stages of startup. Interactive and graphical programs, for instance, can use this predicate to avoid operating on the initial frame, which is never displayed.

New macros `'static-when'` and `'static-unless'`.

Like `'static-if'`, these macros evaluate their condition at macro-expansion time and are useful for writing code that can work across different Emacs versions.

New feature to speed up repeated lookup of Lisp files in `'load-path'`.

If the new variable `'load-path-filter-function'` is set to the new function `'load-path-filter-cache-directory-files'`, calling `'load'` will cache the directories it scans and their files, and the following lookups should be faster.

`'let-alist'` supports indexing into lists.

The macro `'let-alist'` now interprets symbols containing numbers as list indices. For example, `'.key.0'` looks up `'key'` in the alist and then returns its first element.

Lexical binding

You can change the default value of `'lexical-binding'`.

While the default is still the use of dynamic binding dialect of Elisp in those places that don't explicitly set `'lexical-binding'` you can change it globally with:

```
(set-default-toplevel-value 'lexical-binding t)
```

Loading a file displays a warning if there is no `'lexical-binding'` cookie.

Files loaded from `'-x'` and `'--script'` now use lexical binding.

If you don't have time to adapt your script's code to the lexical binding dialect (see `"(elisp) Converting to Lexical Binding"`), you can wrap your code in:

```
#!/usr/bin/env -S emacs --batch --script (eval '(progn YOUR CODE HERE))
```

New function `'set-local'`.

This is the buffer-local equivalent of the function `'set'`.

New macro `'setopt-local'`.

This is the buffer-local version of `'setopt'` for user options rather than plain variables and uses `'custom-set'`/`'set-local'` to set variable values. A new argument, `BUFFER-LOCAL`, is passed to `'custom-set'` functions to indicate the buffer-local context.

New macros `'incf'` and `'decf'`.

They increment or decrement the value stored in a variable (a symbol), or in a generalized variable.

New functions 'plusp' and 'minusp'.

They return non-nil if a number is positive or negative, respectively, and signal an error if they are given a non-number.

New functions 'oddp' and 'evenp'.

They return non-nil if an integer is odd or even, respectively, and signal an error if they are given a non-integer.

New functions 'drop-while' and 'take-while'.

These work like 'drop' and 'take' but use a predicate instead of counting.

New function 'all' and function alias 'any'.

These return non-nil for lists where all and any elements, respectively, satisfy a given predicate.

'equal' now compares circular lists without raising an error.

Comparing very deeply nested objects will still fail, but 'equal' will no longer signal the 'circular-list' error.

The 'defvar-local' macro second argument is now optional.

This means that you can now call it with just one argument, like 'defvar', to declare a variable both special, and buffer-local.

The rx atom 'any' is obsolete and will cause warning when used.

When used as an atom, 'any' is an old alias for 'not-newline' but is often mistakenly used where 'anychar' was intended. Note that the construct '(any ...)' is unrelated and not obsolete.

ERT

Some experimental ERT macros are now considered stable.

The following macros, previously only available in the experimental 'ert-x' module, are now considered stable and have been moved to 'ert':

- 'ert-with-test-buffer' - 'ert-with-buffer-selected' - 'ert-with-buffer-renamed'

See "(ert) Helper Functions" node in the ERT manual for more information.

New function 'ert-play-keys'.

Previously, 'ert-simulate-keys' could be used for sending keys to input functions such as 'read-from-minibuffer', but not for other interactive input such as starting

key-mapped commands.

Show executed tests from erts files via the ERT results buffer.

For tests that call `'ert-test-erts-file'`, the ERT results buffer now allows you to list the tests defined in the referenced erts files that have been executed by the test at point. See `"(ert) Running Tests Interactively"` for more information.

Time & Date

'seconds-to-string' supports new formatting options.

Optional arguments are provided to produce human-readable time-duration strings in a variety of formats, for example `"6 months 3 weeks"` or `"5m 52.5s"`.

New function 'hash-table-contains-p'.

This function returns non-nil if a given key is present in a hash table.

The function 'purecopy' is now an obsolete alias for 'identity'.

New function 'native-compile-directory'.

This function natively compiles all Lisp files in a directory and in its sub-directories, recursively, which were not already natively compiled.

New function 'color-blend'.

This function takes two RGB lists and optional ALPHA and returns an RGB list whose elements are blended in linear space proportional to ALPHA.

New function 'dom-inner-text'.

This function gets all the text within a DOM node recursively, returning it as a concatenated string. It replaces the now-obsolete functions `'dom-text'` and `'dom-texts'`.

The 'defcustom' `':local'` keyword can now be 'permanent-only'.

This means that the variable's `'permanent-local'` property is set to t, without marking it as automatically buffer-local.

The obsolete face attribute `':reverse-video'` has been removed.

Use `':inverse-video'` instead.

D-Bus

Support interactive D-Bus authorization.

A new `':authorizable t'` parameter has been added to `'dbus-call-method'` and `'dbus-call-method-asynchronously'` to allow the user to interactively authorize the invoked D-Bus method (for example via polkit).

Support D-Bus file descriptor manipulation.

A new `:keep-fd` parameter has been added to `'dbus-call-method'` and `'dbus-call-method-asynchronously'` to instruct D-Bus to keep a file descriptor, which has been returned by a method call, internally. The functions `'dbus--fd-open'`, `'dbus--fd-close'` and `'dbus--registered-fds'` implement managing these file descriptors. See the Info node `"(dbus) File Descriptors"` for details.

The customization group 'wp' has been removed.

It has been obsolete since Emacs 26.1. Use the group `'text'` instead.

New optional BUFFER argument for 'string-pixel-width'.

If supplied, `'string-pixel-width'` will use any face remappings from BUFFER when computing the string's width.

New function 'truncate-string-pixelwise'.

This function truncates a string to the specified maximum number of pixels rather than by characters, as in `'truncate-string-to-width'`, and respects face remappings if BUFFER is specified. You can also specify an optional ellipsis string to append, similar to `'truncate-string-to-width'`.

New macro 'with-work-buffer'.

This macro is similar to the already existing macro `'with-temp-buffer'`, except that it does not allocate a new temporary buffer on each call, but tries to reuse those previously allocated (up to a number defined by the new variable `'work-buffer-limit'`, which defaults to 10).

'date-to-time' now defaults to local time.

The function now assumes local time instead of Universal Time when its argument lacks explicit time zone information. This has been the de-facto behavior since Emacs 24 although documentation said otherwise. Also, the fallback on `'timezone-make-date-arpa-standard'` has been removed because its supported date styles can be handled by `'parse-time-string'`. To restore the previously documented behavior, specify `" +0000 "` or `"Z"` as the time zone in the argument.

The 'min-width' property is now supported for overlays as well.

This `'display'` property was previously supported only as text property. Now overlays can also have this property, with the same effect for the text `"covered"` by the overlay.

New function 'remove-display-text-property'.

This function removes a display property from the specified region of text, preserving any other display properties already set for that region.

New macro `'cond*`.

The new macro `'cond*` is an alternative to `'cond` and `'pcase`. Like them, it can be used to define several clauses, each one with its own condition; the first clause that matches will cause its body to be evaluated. `'cond*` can use Pcase's pattern matching syntax and also provides another pattern matching syntax that is different from that of `'pcase`, which some users might find less cryptic. See the Info node `"(elisp) cond* Macro"` for details.

New function `'shell-command-do-open`.

This lets a Lisp program access the core functionality of the `'dired-do-open` command. It opens a file or files using an external program, choosing the program according to the operating system's conventions.

`'make-vtable` can create an empty vtable.

It is now possible to create a vtable without data, by leaving the `':objects` list empty, or by providing an `':objects-function` that (initially) produces no data. In such a case, it is necessary to provide a `':columns` spec, so that the number of columns and their widths can be determined. Column widths can be set explicitly, or they will be calculated based on the window width.

New symbol property `'repeat-continue` for `'repeat-mode`.

A command with the `'repeat-continue` symbol property, which can be a list of keymaps or t, will continue an already active repeating sequence for a keymap in that list (resp. all keymaps). The new property does not affect whether the command starts a repeating sequence, which remains governed by the `'repeat-map` property. `'defvar-keymap` supports a new keyword `':continue`, a list of commands, and adds the keymap to the `'repeat-continue` property of each command in that list. The `'use-package` and `'bind-keys` macros support a similar keyword `':continue-only`.

New function `'completion-table-with-metadata`.

It offers a more concise way to create a completion table with metadata.

`'all-completions` and `'unintern` no longer support old calling conventions.

New symbol property `'find-function-type-alist` used by `'find-function` etc.

Macros that define an object in a way that makes the object's name and the macro call site defining the object hard to associate can add an entry to the property `'find-function-type-alist` on the object's name to provide instructions for finding the definition.

New convenience function `'find-function-update-type-alist` offers a concise way to update a symbol's `'find-function-type-alist` property.

New function variable `'comment-setup-function` for multi-language modes.

It can set comment-related variables such as `'comment-start'` depending on the language under point.

'inhibit-message' can now inhibit clearing of the echo area.

Binding `'inhibit-message'` to a non-nil value will now suppress both the display of messages and the clearing of the echo area, such as caused by calling `'message'` with a nil argument.

'minibuffer-message' no longer blocks while displaying message.

`'minibuffer-message'` now uses a timer to clear the message printed to the minibuffer, instead of waiting with `'sit-for'` and then clearing it. This makes `'minibuffer-message'` usable in Lisp programs which want to print a message and then continue to perform work.

Special Events

New primitive `'insert-special-event'`.

This function inserts the special EVENT into the input event queue.

New event type `'sleep-event'`.

This event is sent when the device running Emacs enters or leaves the sleep state.

Function aliases obsolete since Emacs 23.2 have been removed:

`'advertised-undo'`, `'advertised-widget-backward'`, and `'dired-advertised-find-file'`.

New functions to get and set top-level buffer-local values.

`'buffer-local-toplevel-value'` and `'set-buffer-local-toplevel-value'` get and set the top-level buffer-local value of a variable. A top-level value is the one that variable has outside of any let-bindings.

New function `'exec-suffixes'`.

This function by default returns the value of the corresponding user option, but can optionally return the equivalent of `'exec-suffixes'` from a remote host. It must be used in conjunction with the function `'exec-path'`.

'read-directory-name' now accepts an optional PREDICATE argument.

JSON parse error line and column are now obsolete.

The column number is no longer available; the line number will be removed in next Emacs release.

'defvar-keymap' can now take a `':prefix t'` option.

This is an abbreviation for using the name of the keymap as the prefix command name. E.g., `'(defvar-keymap foo-map :prefix t)'` is equivalent to `'(defvar-keymap foo-map :prefix 'foo-map)'`.

New 'R' code letter for 'interactive' forms.

This specifies the beginning and end of an active region, and nil twice if the region is inactive. The interactive specification

```
(interactive "R")
```

is equivalent to

```
(interactive (list (use-region-beginning) (use-region-end)))
```

Toolkit

The Emacs PGTK toolkit respects dark and light modes.

Emacs when built with the pure GTK toolkit now respects desktop dark and light modes for drawing a GTK toolbar and widgets, automatically toggling between them.

'toolkit-theme-set-functions' called when the toolkit theme is set for Emacs.

When the theme is set on PGTK, Android, or MS-Windows systems,

`'toolkit-theme-set-functions'` is called. The result is stored in the variable `'toolkit-theme'` as either symbol `'dark'` or `'light'`, but may be extended to encompass other toolkit-specific symbols in the future.

Progress reporter context.

`'make-progress-reporter'` now accepts the optional argument `CONTEXT`, which if it is the symbol `'async'`, inhibits updates in the echo area when it is busy. This is useful, for example, if you want to monitor progress of an inherently asynchronous command such as `'compile'`.

Binary format specifications '%b' and '%B' added.

These produce the binary representation of a number. `'%#b'` and `'%#B'` prefix the bits with `'0b'` and `'0B'`, respectively.

'pp-eval-expression' can now insert results into the current buffer.

With a prefix argument, `'pp-eval-expression'` inserts the result into the current buffer, just like `'eval-expression'` already did.

New function 'multiple-command-partition-arguments'.

This function partitions a list of command arguments that might be arbitrarily long. It can be used in cases in which it is known to be safe to run the command multiple times on subsequent partitions of the list of arguments. The variable

`'command-line-max-length'` controls the partitioning.

New function `'ensure-proper-list'`.

This function is a variation on `'ensure-list'` that checks if an object is a proper list, in which case the list will be returned as is, otherwise the function will return the object wrapped in a singleton list.

In batch mode, `'C-c'` (i.e. SIGINT) can either `'quit'` or kill Emacs.

By default it kills Emacs, as before, but `'kill-emacs-on-sigint'` can be set to nil to change that. The response to SIGINT in interactive sessions is unaffected, e.g., in a normal GUI session it still kills Emacs whereas in a terminal it causes `'quit'` since it is used for `'C-g'`.

New `':interactive-only'` way to add an advice.

While it is marginally more efficient than `':after'` or `':before'`, the main purpose is to make the intention more obvious when the advice modifies only the interactive form and not the actual behavior of the function.

Changes in Emacs 31.1 on Non-Free Operating Systems

Support macOS Accessibility Zoom focus tracking.

This is an important change for visually-impaired users. If macOS Accessibility Zoom is enabled via (System Settings, Accessibility, Zoom...) with keyboard focus tracking (Advanced...), Zoom is informed of updated cursor positions during each redisplay cycle.

New macOS function `'ns-process-is-accessibility-trusted'`.

This function returns t if the macOS Accessibility Framework trusts Emacs. This is a necessary condition for Accessibility Zoom and other accessibility features. Enable Emacs via (System Settings, Privacy & Security, Accessibility...) and add the Emacs.app installed directory to the enabled application list.

Process execution has been optimized on Android.

The run-time performance of subprocesses on recent Android releases, where a userspace executable loader is required, has been optimized on systems featuring Linux 3.5.0 and above.

It is now possible to read GUI events from non-main Lisp threads on Android.

Put differently, this enables input events to be read and recursive editing sessions to be started from non-main threads. The only platform where this remains unsupported is Nextstep (GNUstep or Mac OS).

`'desktop-restore-frames'` has been disabled by default on Android systems.

Restrictions imposed on clients by the window manager on these systems are too prohibitive and don't allow restoring frame configurations. (For the same reason many window management facilities are also

not implemented by Emacs.)

Emacs responds to runtime display configuration changes on Android.

The upshot of this is that Emacs will adapt to display resolution / layout changes applied while an Emacs session is active, which is possible on some recently released devices.

'NSSpeechRecognitionUsageDescription' now included in "Info.plist" (macOS).

Should Emacs (or any built-in shell) invoke a process using macOS speech recognition APIs, the relevant permission dialog is now displayed, thus allowing Emacs users access to speech recognition utilities.

Note: Accepting this permission allows the use of system APIs, which may send user data to Apple's speech recognition servers.

Re-introduced dictation, lost in Emacs 30 (macOS).

We lost macOS dictation in Emacs 30 when migrating to NSTextInputClient. Implemented `'selectedRange'` in `'nsterm.m'` to enable in new subsystem. You may notice a slight change in dictation UI provided by macOS.

On Mac OS X, stipples now render with color.

Emacs on MS-Windows now supports GUI dialogs and message boxes better.

In particular, it is now possible to show text with embedded newlines in a dialog popped by `'message-box'`. This is supported on Windows Vista and later versions.

Emacs on MS-Windows now supports drag-n-drop of text into a buffer.

This is in addition to drag-n-drop of files, that was already supported. As on X, the user options `'dnd-scroll-margin'` and `'dnd-indicate-insertion-point'` can be used to customize the process.

Emacs on MS-Windows now supports color fonts.

On Windows 8.1 and later versions, Emacs now uses DirectWrite to draw text, which supports color fonts. This can be disabled by setting the variable `'w32-inhibit-dwrite'` to t. Also see `'w32-dwrite-available'` and `'w32-dwrite-reinit'` to check availability and to configure the DirectWrite rendering parameters.

To show color Emojis in Emacs, customize the default fontset to use a color Emoji font installed on your system for the `'emoji'` script.

Emacs on MS-Windows now supports 'yank-media'.

This command inserts clipboard data of different formats into the current buffer, if the major mode supports it.

Emacs on MS-Windows now supports up to 1024 sub-processes.

Changes in implementation of monitoring sub-processes allow Emacs on MS-Windows to start up to 1024 sub-processes, similar to GNU/Linux and other free systems.

Images on MS-Windows now support the ':transform-smoothing' flag.

Transformed images are smoothed using the bilinear interpolation by means of the GDI+ library.

Emacs on MS-Windows is now capable of exporting frame screenshots to files.

The new primitive `'w32-export-frame'` can be used to export a screenshot of a specified frame to an image file in one of the supported image formats, such as JPEG or PNG.

Emacs on MS-Windows now supports the ':data' keyword for 'play-sound'.

In addition to `':file FILE'` for playing a sound from a file, `':data DATA'` can now be used to play a sound from memory.

New primitive 'w32-sound-volume'.

This primitive allows getting and setting the volume of the system's default audio device (or the `"optimal device"`, if there are several devices).

The MS-DOS port of Emacs now supports more recent releases of GCC and Binutils.

Accordingly, we have revised our recommendations for a suitable DJGPP toolchain to GCC 14.2.0 and Binutils 2.35.1 in lieu of GCC 3.4.x and Binutils 2.26.

Windows Terminal can now display 256 and 24-bit RGB color.

Previously, terminal sessions on Windows supported display of 16 colors. There is now support for 8-bit (256 color) and 24-bit RGB (true color). The new mechanism will be enabled automatically when supported. It defaults to 24-bit RGB color, but can be set to 8, 16, `'8bit'`, or `'24bit'` by passing the `'--color'` flag or setting the `'tty-color-mode'` frame parameter. Use of the new mechanism is controlled by an internal variable that can be set and inspected via the functions `'w32-use-virtual-terminal'` and `'w32-use-virtual-terminal-p'` (respectively). See the manual entry `"(emacs) Windows Misc"` for more details.

This file is part of GNU Emacs.

GNU Emacs is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNU Emacs is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNU Emacs. If not, see <<https://www.gnu.org/licenses/>>.

Local variables: coding: utf-8 mode: outline mode: emacs-news paragraph-separate: "[]" end:

Emacs 31.0.90 – NEWS

GNU Emacs NEWS -- history of user-visible changes.

Copyright (C) 2022-2026 Free Software Foundation, Inc. See the end of the file for license conditions.

Please send Emacs bug reports to 'bug-gnu-emacs@gnu.org'. If possible, use '`M-x report-emacs-bug`'.

This file is about changes in Emacs version 31.

See file HISTORY for a list of GNU Emacs versions and release dates. See files NEWS.30, NEWS.29, ..., NEWS.18, and NEWS.1-17 for changes in older Emacs versions.

You can narrow news to a specific version by calling '`view-emacs-news`' with a prefix argument or by typing '`C-u C-h C-n`'.

Temporary note: +++ indicates that all relevant manuals in doc/ have been updated. --- means no change in the manuals is needed. When you add a new item, use the appropriate mark if you are sure it applies, and please also update docstrings as needed.

Installation Changes in Emacs 31.1

Unexec dumper removed.

The traditional unexec dumper, deprecated since Emacs 27, has been removed.

Emacs's old 'ctags' program is no longer built or installed.

You are encouraged to use Universal Ctags <<https://ctags.io/>> instead. For now, to get the old 'ctags' behavior you can run '`etags --ctags`' or use a shell script named '`ctags`' that runs '`etags --ctags "$@"`'.

Changed GCC default options on 32-bit x86 systems.

When using GCC 4 or later to build Emacs on 32-bit x86 systems, '`configure`' now defaults to using the GCC options '`-mfpmath=sse`' (if the host system supports SSE2) or

'`--fno-tree-sra`' (if not). These GCC options work around GCC bug 58416, which can cause Emacs to behave incorrectly in rare cases.

New configure option '`--with-systemduserunitdir`'.

This allows to specify the directory where the user unit file for systemd is installed; default is '`${prefix}/usr/lib/systemd/user`'.

Startup Changes in Emacs 31.1

In compatible terminals, '`xterm-mouse-mode`' is turned on by default.

For these terminals the mouse will work by default. A compatible terminal is one that supports Emacs setting and getting the OS selection data (a.k.a. the clipboard) and mouse button and motion events. With

'`xterm-mouse-mode`' enabled, you must use Emacs keybindings to copy to the OS selection instead of terminal-specific keybindings.

You can keep the old behavior by putting '`(xterm-mouse-mode -1)`' in your init file.

`site-start.el` is now loaded before the user's early init file.

Previously, the order was `early-init.el`, `site-start.el` and then the user's regular init file, but now `site-start.el` comes first. This allows site administrators to customize things that can normally only be done from `early-init.el`, such as adding to '`package-directory-list`'.

Emacs prepares a User Lisp directory by default.

If you have a subdirectory "`user-lisp/`" in your Emacs configuration directory, then Lisp files in it and its subdirectories will now be recursively byte-compiled, scraped for autoload cookies and ensured to be in '`load-path`' by default. You can disable the feature by setting '`user-lisp-auto-scrape`' to nil, or set the '`user-lisp-directory`' user option to process any other directory on your system. You can also invoke the '`prepare-user-lisp`' command manually at any time. See the Info node "`(emacs) User Lisp Directory`" for more details.

Changes in Emacs 31.1

'`line-spacing`' now supports specifying spacing above the line.

Previously, only spacing below the line could be specified. The user option can now be set to a cons cell to specify spacing both above and below the line, which allows for vertically centering text.

New face '`margin`' for the window margin display.

A new basic face `'margin'` is used by default for text displayed in the left and right margin areas, which are used by various packages for per-line annotations. Its background defaults to the frame default, so existing behavior is unchanged for users who do not customize this new face.

Display strings shown in the margins now inherit the unspecified face attributes from the `'margin'` face, if the string itself does not fully specify its face. If your code relied on the face of the underlying buffer text to serve as default for the unspecified face attributes of strings displayed in the margin, you must now set those face attributes in the margin string itself via `'propertize'`.

`'prettify-symbols-mode'` attempts to ignore undisplayable characters.

Previously, such characters would be rendered as, e.g., white boxes.

`'standard-display-table'` now has more extra slots.

`'standard-display-table'` has been extended to allow specifying glyphs that are used for borders around child frames and menu separators on TTY frames.

Call the command `'standard-display-unicode-special-glyphs'` to set up the `'standard-display-table'`'s extra slots with Unicode characters. Please see the documentation of that command to see which slots of the display table it changes.

Child frames are now supported on TTY frames.

This supports use-cases like Posframe, Corfu, and child frames acting like tooltips.

To enable tooltips on TTY frames, call `'tty-tip-mode'`.

The presence of child frame support on TTY frames can be checked with `'(featurep 'tty-child-frames)'`.

Recent versions of Posframe and Corfu are known to use child frames on TTYs if they are supported.

Several font-lock face variables are now obsolete.

The following variables are now obsolete: `'font-lock-builtin-face'`, `'font-lock-comment-delimiter-face'`, `'font-lock-comment-face'`, `'font-lock-constant-face'`, `'font-lock-doc-face'`, `'font-lock-doc-markup-face'`, `'font-lock-function-name-face'`, `'font-lock-keyword-face'`, `'font-lock-negation-char-face'`, `'font-lock-preprocessor-face'`, `'font-lock-string-face'`, `'font-lock-type-face'`, `'font-lock-variable-name-face'`, and `'font-lock-warning-face'`.

These variables contributed both to confusion about the relation between faces and variables, and to inconsistency when major mode authors used one or the other (sometimes interchangeably). We always recommended using faces directly, and not

creating variables going by the same name.

If you have customized these variables, you should now customize the corresponding faces instead, using something like:

```
M-x customize-face RET font-lock-string-face RET
```

If you have been using these variables in Lisp code (for example, in font-lock rules), simply quote the symbol, to use the face directly instead of its now-obsolete variable.

New char-table 'special-mirror-table' for mirroring special glyphs.

This char-table is used to mirror special glyphs (truncation and continuation) when the user has defined an alternative representation for those characters via display tables.

find-func.el commands now have history enabled.

The 'find-function', 'find-library', 'find-face-definition', and 'find-variable' commands now allow retrieving previous input using the usual minibuffer history commands. Each command has a separate history.

New minor mode 'find-function-mode' replaces 'find-function-setup-keys'.

The new minor mode defines the keys at a higher precedence level than the old function, one more usual for a minor mode. To restore the old behavior, customize 'find-function-mode-lower-precedence' to non-nil.

'find-function' can now find 'cl-defmethod' invocations inside macros.

New minor mode 'prettify-special-glyphs-mode'.

The new minor mode prettifies the special character glyphs (truncation and continuation) on TTY frames (and GUI frames without fringes). You can customize the associated new face 'special-glyphs'.

Minibuffer and Completions

Support for immediate display of the "*Completions*" buffer.

Whenever a minibuffer with completion is opened, the "*Completions*" buffer will now be displayed immediately if the completion property 'eager-display', set by the completion table, is non-nil. This property can be overridden for different completion categories by customizing 'completion-category-overrides'. Alternatively, the new user option 'completion-eager-display' can be set to t to force eager display of "*Completions*" for all minibuffers, or nil to suppress this for all minibuffers.

Support for completion category inheritance.

You can now define completion categories that inherit properties from existing categories, using the new function 'define-completion-category'.

Support for updating `"*Completions"` as you type.

If the `"*Completions"` buffer is displayed, it will now be updated as you type if the completion property `'eager-update'`, set by the completion table, is non-nil. This property can be overridden for different completion categories by customizing `'completion-category-overrides'`. Alternatively, the new user option `'completion-eager-update'` can be set to `t` to make `"*Completions"` always be updated as you type, or `nil` to suppress this always. Note that for large or inefficient completion tables this can slow down typing.

New optional value of `'minibuffer-visible-completions'`.

If the value of this option is `'up-down'`, only the `'<up>'` and `'<down>'` arrow keys move point between candidates shown in the `"*Completions"` buffer display, while `'<right>'` and `'<left>'` arrows move point in the minibuffer window.

`'RET'` chooses the completion selected with `'M-<up>'/M-<down>'`.

If a completion candidate is selected with `'M-<up>'` or `'M-<down>'`, hitting `'RET'` will exit completion with that as the result. This works both in minibuffer completion and in-buffer completion. This supersedes `'minibuffer-completion-auto-choose'`, which previously provided similar behavior; that variable is now `nil` by default.

New user option `'completion-pcm-leading-wildcard'`.

This option configures how the partial-completion style does completion. It defaults to `nil`, which preserves the existing behavior. When it is set to `t`, the partial-completion style behaves more like the substring style, in that a string being completed can match against a candidate anywhere in the candidate string.

`'completion-styles'` now can contain lists of bindings.

In addition to being a symbol naming a completion style, an element of `'completion-styles'` can now be a list of the form `'(STYLE ((VARIABLE VALUE) ...))'` where `STYLE` is a symbol naming a completion style. `VARIABLE` will be bound to `VALUE` (without evaluating it) while the style is executing. This allows multiple

references to the same style with different values for completion-affecting variables like `'completion-pcm-leading-wildcard'` or `'completion-ignore-case'`. This also applies for the styles configuration in `'completion-category-overrides'` and `'completion-category-defaults'`.

Navigating `"*Completions"` now accommodates `'completions-format'`.

When `'completions-format'` is set to `'vertical'`, typing `'n'`, `'TAB'` or `'M-<down>'` in the `"*Completions"` buffer (the latter also in the minibuffer) now moves point to the completion candidate in the next line in the current column, and wraps to the next column when typed on the last completion candidate of the current column. Likewise, typing `'p'`, `'S-TAB'` or `'M-<up>'` moves point to the completion candidate in the previous line or wraps to the previous column. Previously, these keys ignored the vertical format, i.e., moved point only to the item in the same line of the next or previous

column, in accordance with the default horizontal format. In vertical format, typing '`<left>`' and '`<right>`' in the `"*Completions"` buffer (and when '`minibuffer-visible-completions`' is non-nil, also in the minibuffer) moves point only within the current line, analogously to how, in horizontal format, '`<down>`' and '`<up>`' move point only within the current column.

Selected completion candidate is preserved across `"*Completions"` updates.

When point is on a completion candidate in the `"*Completions"` buffer (because of '`minibuffer-next-completion`' or for any other reason), point will still be on that candidate after `"*Completions"` is updated with a new list of completions. The candidate is automatically deselected when the `"*Completions"` buffer is hidden.

`"*Completions"` is now displayed faster when there are many candidates.

As before, if there are more completion candidates than can be displayed in the current frame, only a subset of the candidates is displayed. This process is now faster: only that subset of the candidates is actually inserted into `"*Completions"` until you run a command which interacts with the text of the `"*Completions"` buffer. This optimization only applies when '`completions-format`' is '`horizontal`' or '`one-column`'.

New user option '`crm-prompt`' for '`completing-read-multiple`'.

This option configures the prompt format of '`completing-read-multiple`'. By default, the prompt indicates to the user that the completion command accepts a comma-separated list. The prompt format can include the separator description and the separator string, which are both stored as text properties of the '`crm-separator`' regular expression.

New user option '`completion-preview-sort-function`'.

This option controls how Completion Preview mode sorts completion candidates. If you use this mode together with an in-buffer completion popup interface, such as the interfaces that the GNU ELPA packages Corfu and Company provide, you can set this option to the same sort function that your popup interface uses for a more integrated experience.

(`'completion-preview-sort-function`' was already present in Emacs 30.1, but as a plain Lisp variable, not a user option.)

New user option '`completion-preview-inhibit-functions`'.

This option provides fine-grained control over Completion Preview mode activation. You can use it to specify arbitrary conditions in which to inhibit the mode's operation.

New mode '`minibuffer-nonselected-mode`'.

This mode, enabled by default, directs the attention to the active minibuffer window using the '`minibuffer-nonselected`' face in case when the minibuffer window is no longer selected, but the minibuffer is still waiting for input.

`'read-multiple-choice`' now uses the minibuffer to read a character.

It still can use `'read-key'` when the variable `'read-char-choice-use-read-key'` is non-nil.

'map-y-or-n-p' now uses the minibuffer to read a character.

It still can use `'read-key'` when the variable `'y-or-n-p-use-read-key'` is non-nil.

'flex' completion style rewritten to be faster and more accurate.

Completion and highlighting use a new superior algorithm. For example, pattern `"scope"` now ranks `'elisp-scope-*` functions well above `'dos-codepage'` and `'test-completion'`. Pattern `"botwin"` finds `'menu-bar-bottom-window-divider'` before `'ibuffer-other-window'`.

Mouse

New mode 'mouse-shift-adjust-mode' extends selection with 'S-<mouse-1>'.

When enabled, you can use the left mouse button with the `'<Shift>'` modifier to extend the boundaries of the active region by dragging the mouse pointer.

'context-menu-mode' now includes a "Send to..." menu item.

The menu item enables sending current file(s) or region text to external (non-Emacs) applications or services. See `send-to.el` for customizations.

The mouse now drags lines in character increments again.

Dragging a horizontal or vertical line like the mode line or the lines dividing side-by-side windows now by default happens in increments of the corresponding frame's character size again. This is the behavior described in the manual and was the default behavior before `'window-resize-pixelwise'` was added for Emacs 24.1. To drag in pixel increments as with Emacs 24 through Emacs 30 you now have to set `'window-resize-pixelwise'` to `t`.

Windows

New commands to modify window layouts.

- `'C-x w t'` and `'C-x w r <left>/<right>'` rotate the window layout. - `'C-x w o <left>/<right>'` rotate the windows within the current layout. - `'C-x w f <left>/<right>/<up>/<down>'` flip window layouts.

By default, these commands operate on the selected frame's root window. With a prefix argument, they operate on the selected window's parent.

Windmove commands now move to skipped windows if invoked twice in a row.

The new user option `'windmove-allow-repeated-command-override'` controls this behavior: if it is non-nil, invoking the same windmove command twice overrides the `'no-other-window'` property, allowing navigation to windows that would normally be skipped. The default is `t`; customize it to `nil` if you want the old behavior.

New hook 'window-deletable-functions'.

This abnormal hook gives its client a way to save a window from getting deleted implicitly by functions like `'kill-buffer'`, `'bury-buffer'` and `'quit-restore-window'`.

Buffer-local window change functions run in their buffers now.

Running the buffer-local version of each of the abnormal hooks `'window-buffer-change-functions'`, `'window-size-change-functions'`, `'window-selection-change-functions'` and `'window-state-change-functions'` will make the respective buffer temporarily current for running the hook.

'window-buffer-change-functions' is run for removed buffers too.

The buffer-local version of `'window-buffer-change-functions'` may be run twice now: Once for the buffer removed from the respective window and once for the buffer now shown in that window.

New user option 'quit-window-kill-buffer'.

This option specifies whether `'quit-window'` should preferably kill or bury the buffer shown by the window to quit. The default is nil. Customize it to t to always kill the buffer; customize to a list of major modes to kill if the buffer's major mode is one of those.

New user option 'kill-buffer-quit-windows'.

This option has `'kill-buffer'` call `'quit-restore-window'` to handle the further destiny of any window showing the buffer to be killed.

'split-window' can optionally resurrect deleted windows.

A new optional argument REFER of `'split-window'` makes it possible to, instead of making a new window object, reuse an existing, deleted one. This can be used to preserve the identity of windows when swapping or transposing them.

New window parameter 'quit-restore-prev'.

This parameter is set up by `'display-buffer'` when it detects that the window used already has a `'quit-restore'` parameter. Its presence gives `'quit-restore-window'` a way to undo a sequence of buffer display operations more intuitively.

'quit-restore-window' handles new values for BURY-OR-KILL argument.

The values `'killing'` and `'burying'` are like `'kill'` and `'bury'` but assume that the actual killing or burying of the buffer is done by the caller.

New user option 'quit-restore-window-no-switch'.

With this option set, `'quit-restore-window'` will delete its window more aggressively rather than switching to some other buffer in it.

The user option 'display-comint-buffer-action' has been removed.

It has been obsolete since Emacs 30.1. Use `'(category . comint)'` instead. Another user option `'display-tex-shell-buffer-action'` has been removed too for which you can use

```
'(category . tex-shell)'.
```

New user option 'split-window-preferred-direction'.

Functions called by `'display-buffer'` split the selected window when they need to create a new window. A window can be split either vertically, one below the other, or horizontally, side by side. This new option determines which direction will be tried first, when both directions are possible according to the values of `'split-width-threshold'` and `'split-height-threshold'`. The default value is `'longest'`, which means to prefer to split horizontally if the window's frame is a `"landscape"` frame, and vertically if it is a `"portrait"` frame. (A frame is considered to be `"portrait"` if its vertical dimension in pixels is greater or equal to its horizontal dimension, otherwise it is considered to be `"landscape"`.) Previous versions of Emacs always tried to split vertically first, so to get previous behavior, you can customize this option to `'vertical'`. The value `'horizontal'` always prefers the horizontal split.

The default value of 'split-width-threshold' is changed from 160 to 150.

After splitting it is still comfortable to read text filled to 75 columns.

New optional argument INDIRECT for 'get-buffer-window-list'.

With this argument non-nil, `'get-buffer-window-list'` will include in the return value windows whose buffers share their text with BUFFER-OR-NAME.

New 'display-buffer' action alist entry 'reuse-indirect'.

With such an entry, `'display-buffer-reuse-window'` may also choose a window whose buffer shares text with the buffer to display.

New variable 'window-state-normalize-buffer-name'.

When bound to non-nil, `'window-state-get'` will normalize `'uniquify'` managed buffer names by removing `'uniquify'` prefixes and suffixes. This helps to restore window buffers across Emacs sessions.

New action alist entry 'this-command' for 'display-buffer'.

You can use this in `'display-buffer-alist'` to match buffers displayed during the execution of particular commands.

New command 'other-window-backward' ('C-x 0').

This moves in the opposite direction of `'other-window'` and is for its default keybinding consistent with `'repeat-mode'`.

New functions 'combine-windows' and 'uncombine-window'.

`'combine-windows'` is useful to make a new parent window for several adjacent windows and subsequently operate on that parent. `'uncombine-window'` can then be used to restore the window configuration to the state it had before running `'combine-windows'`.

New function 'window-cursor-info'.

This function returns a vector of pixel-level information about the physical cursor in a given window, including its type, coordinates, dimensions, and ascent.

Frames

New function 'frame-deletable-p'.

If this function returns nil, the following call to `'delete-frame'` might fail to delete its argument FRAME or might signal an error. It is therefore advisable to use this function as part of a condition that determines whether to call `'delete-frame'`.

New function 'frame-use-time'.

This function is the frame equivalent of the function `'window-use-time'` for a window. The result is the `'window-use-time'` of a frame's most recently used window.

New functions 'get-mru-frames' and 'get-mru-frame'.

`'get-mru-frames'` returns a list of frames sorted by their most recent use time from among all frames, or those visible or iconified on the same terminal as the selected frame. Child frames can be excluded. A single frame can be excluded, for example, the selected frame. `'get-mru-frame'` returns the single most recently used frame.

After deleting, 'delete-frame' now selects the most recently used frame.

Previously, after deleting a specified frame, `'delete-frame'` would select the oldest visible frame on the same terminal. To revert to the old behavior, set the new user option `'delete-frame-choose-selected'` to nil.

New value 'force' for user option 'frame-inhibit-implied-resize'.

This will inhibit implied resizing while a new frame is made and can be useful on tiling window managers where the initial frame size should be specified by external means.

New user option 'alter-fullscreen-frames'.

This option is useful to maintain a consistent state when attempting to resize fullscreen frames. It defaults to `'inhibit'` on NS builds which means that a fullscreen frame would not change size. It defaults to nil everywhere else which means that the window manager is supposed to either resize the frame and change the fullscreen status accordingly or keep the frame size unchanged. The value `t` means to first reset the fullscreen status and then resize the frame.

New functions to set frame size and position in one compound step.

`'set-frame-size-and-position'` sets the new size and position of a frame in one compound step. Both, size and position, can be specified as with the corresponding frame parameters `'width'`, `'height'`, `'left'` and `'top'`.

`'set-frame-size-and-position-pixelwise'` is similar but has a more restricted set of values for specifying size and position.

New commands 'split-frame' and 'merge-frames'.

'split-frame' moves a specified number of windows from an existing frame to a newly-created frame. 'merge-frames' merges all windows from two frames into one of these frames and deletes the other one.

Frames can now be renamed to "F<number>" on text terminals.

Unlike with other frame names, an attempt to rename to "F<number>" throws an error when a frame of that name already exists.

New frame parameters 'cloned-from' and 'undeleted'.

The frame parameter 'cloned-from' is set to the frame from which the new frame is cloned using the command 'clone-frame'.

The frame parameter 'undeleted' is set to t when a frame is undeleted using the command 'undelete-frame'.

These are useful if you need to detect a cloned frame or undeleted frame in hooks like 'after-make-frame-functions' and 'server-after-make-frame-hook'.

Frames now have unique ids and the new function 'frame-id'.

Each non-tooltip frame is assigned a unique integer id. This allows you to unambiguously identify frames even if they share the same name or title. When 'undelete-frame-mode' is enabled, each deleted frame's id is stored for resurrection. The function 'frame-id' returns a frame's id (in C, use the frame struct member 'id').

New commands 'select-frame-by-id', 'undelete-frame-by-id'.

The command 'select-frame-by-id' selects a frame by ID and undeletes it if deleted. The command 'undelete-frame-by-id' undeletes a frame by its ID. When called interactively, both functions prompt for an ID.

Mode Line

New definitions for mode line faces under dark background mode.

The faces 'mode-line' and 'mode-line-highlight' now have separate definitions for the dark background mode. Previously, these two faces looked the same in both the light and dark background modes. To get the previous visuals for these two faces, customize them to have the colors "grey75" and "grey40", respectively, regardless of the background mode.

New user option 'mode-line-collapse-minor-modes'.

If non-nil, minor mode lighters on the mode line are collapsed into a single button. The value could also be a list to specify minor mode lighters to hide or show. The default value is nil, which retains the previous

behavior of showing all minor mode lighters.

New user option 'mode-line-modes-delimiters'.

This option allows changing or removing the delimiters shown around the major mode and list of minor modes in the mode line. The default retains the existing behavior of inserting parentheses.

New minor mode 'mode-line-invisible-mode'.

This minor mode makes the mode line of the current buffer invisible. The command `'mode-line-invisible-mode'` toggles the visibility of the current-buffer's mode line. The default is to show the mode line of every buffer.

Standard mode line no longer specifies minimum widths.

The default values for the `'mode-line-position'` variable and `'mode-line-format'` user option no longer specify any minimum widths. If you use a proportional font for your mode line, you may need to customize the values of these variables to include minimum widths again.

Tab Bars and Tab Lines***New commands 'split-tab' and 'merge-tabs'.***

`'split-tab'` moves a specified number of windows from an existing tab to a newly-created tab. `'merge-tabs'` merges all windows from two tabs into one of these tabs and closes the other tab.

New abnormal hook 'tab-bar-auto-width-functions'.

This hook allows you to control which tab-bar tabs are auto-resized.

'mouse-face' properties are now supported on the 'tab-bar'.

`'tab-bar'` tab "buttons" are now highlighted when the mouse pointer hovers over them. You can customize the new face `'tab-bar-tab-highlight'`.

New abnormal hook 'tab-bar-post-undo-close-tab-functions'.

This hook allows you to operate on a reopened tab.

This is useful when you define custom tab parameters that may need adjustment when a tab is restored, and avoids advice.

Tabs are closed now on releasing the middle mouse button.

Previously, closing the tab-bar's tabs was on pressing the button.

New user option 'tab-bar-define-keys'.

This controls which key bindings tab-bar creates. Values are `t`, the default, which defines all keys and is backwards compatible, `'numeric'` (tab number selection only), `'tab'` (`'TAB'` and `'S-TAB'` keys only), `nil` (which defines none).

This is useful to avoid key binding conflicts, such as when folding in outline mode using `'TAB'` keys, or when a user wants to define her own tab-bar keys without first having to remove the defaults.

New variable `'tab-bar-format-tab-help-text-function'`.

This variable may be overridden with a user-provided function to customize help text for tabs displayed on the tab-bar. Help text is normally shown in the echo area or via tooltips. See the variable's docstring for arguments passed to a help-text function.

New variable `'tab-bar-truncate'`.

When non-nil, it truncates the tab bar, and therefore prevents wrapping and resizing the tab bar to more than one line.

New user option `'tab-line-define-keys'`.

When t, the default, it redefines window buffer switching keys such as `'C-x <left>'` and `'C-x <right>'` to tab-line specific variants for switching tabs.

New command `'tab-line-move-tab-forward'` (`'C-x M-<right>'`).

Together with the new command `'tab-line-move-tab-backward'` (`'C-x M-<left>'`), it can be used to move the current tab on the tab line to a different position.

New command `'tab-line-close-other-tabs'`.

It is bound to the tab's context menu item `"Close other tabs"`.

New user option `'tab-line-exclude-buffers'`.

This user option controls where `'tab-line-mode'` should not be enabled in a buffer. The value must be a condition which is passed to `'buffer-match-p'`.

New user option `'tab-line-close-modified-button-show'`.

With this user option, if non-nil (the default), the tab close button will change its appearance if the tab buffer has been modified.

New user option `'tab-line-tabs-window-buffers-filter-function'`.

This user option controls which buffers should appear in the tab line. By default, this is set to not filter the buffers.

New faces `'tab-line-active'` and `'tab-line-inactive'`.

These inherit from the `'tab-line'` face, but the faces actually used on the tab lines are now these two: the selected window uses `'tab-line-active'`, non-selected windows use `'tab-line-inactive'`.

Help

New keybinding `'C-h u'` for `'apropos-user-option'`.

IDLWAVE has been moved to GNU ELPA.

The version bundled with Emacs is out-of-date, and is now marked as obsolete. Use `'list-packages'` to install the `'idlwave'` package from GNU ELPA instead.

New faces `'header-line-active'` and `'header-line-inactive'`.

These inherit from the `'header-line'` face, but the faces actually used on the header lines are now these two: the selected window uses `'header-line-active'`, non-selected windows use `'header-line-inactive'`.

In `'customize-face'`, the "Font family" attribute now supports completion.

`'process-adaptive-read-buffering'` is now nil by default.

Setting this variable to a non-nil value reduces performance and leads to wrong results in some cases. We believe that it is no longer useful; please contact us if you still need it for some reason.

`'byte-compile-cond-use-jump-table'` is now obsolete.

Modified settings for an enabled theme now apply immediately.

Evaluating a `'custom-theme-set-faces'` or `'custom-theme-set-variables'` call for an enabled theme causes the settings to apply immediately, without a need to re-load the theme.

`'describe-variable'` now automatically says if `'setopt'` is needed.

If a user option has a defcustom `':set'` function, users will normally need to set it with `'setopt'` for it to take an effect. If the docstring doesn't already mention `'setopt'`, the `'describe-variable'` command will now add a note about this automatically.

New user option `'eldoc-help-at-pt'` to show help at point via ELDoc.

When enabled, display the `'help-at-pt-kbd-string'` via ELDoc. This setting is an alternative to `'help-at-pt-display-when-idle'`.

New user option `'native-comp-async-on-battery-power'`.

Customize this to nil to disable starting new asynchronous native compilations while AC power is not connected.

New user option `'show-paren-not-in-comments-or-strings'`.

If this option is non-nil, it tells `'show-paren-mode'` not to highlight the parens that are inside comments and strings. If set to `'all'`, `'show-paren-mode'` will never highlight parens that are inside comments or strings. If set to `'on-mismatch'`, mismatched parens inside comments and strings will not be highlighted. If set to nil (the default), highlight the parens wherever they are.

New user option 'view-lossage-auto-refresh'.

If this option is non-nil, the lossage buffer in 'view-lossage' will be refreshed automatically for each new input keystroke and command performed.

Change in SVG foreground color handling.

SVG images no longer have the 'fill' attribute set to the value of ':foreground' or the current text foreground color. The 'currentcolor' CSS attribute continues to be set as before.

This change should result in more consistent display of SVG images.

To use the ':foreground' or current text color ensure the 'fill' attribute in the SVG is set to 'currentcolor', or set the image spec's ':css' value to 'svg {fill: currentcolor;}'.

Errors signaled by 'emacsclient' connections can now enter the debugger.

If 'debug-on-error' is non-nil, errors signaled by Lisp programs executed due to 'emacsclient' connections will now enter the Lisp debugger and show the backtrace. If 'debug-on-error' is nil, these errors will be sent to 'emacsclient', as before, and will be displayed on the terminal from which 'emacsclient' was invoked.

Empty string arguments to emacsclient are no longer ignored.

Emacs previously discarded arguments to emacsclient of zero length, such as in 'emacsclient --eval "(length (pop server-eval-args-left))" ""'. These are no longer discarded.

Emacs now uses the 'setrgbf' and 'setrgbb' terminfo capabilities.

Emacs now uses 24-bit colors on terminals that support the 'setrgbf' and 'setrgbb' user-defined terminfo capabilities. These are supported by more terminals and applications than the old capabilities, 'setf24' and 'setb24', which are now obsolete.

New user option 'xterm-update-cursor' to update cursor display on TTYs.

When enabled, Emacs sends Xterm escape sequences on Xterm-compatible terminals to update the cursor's appearance. Emacs can update the cursor's shape and color. For example, if you use a purple bar cursor on graphical displays then when this option is enabled Emacs will use a purple bar cursor on compatible terminals as well. See the Info node "(emacs) Cursor Display" for more information.

New command 'copy-theme-options'.

You can use this command to copy options from a theme into your user configuration.

New user option 'multiple-terminals-merge-keyboards'.

Customizing this option to non-nil disables entering single-keyboard mode in most cases in which Emacs would by default enter that mode. This can make things work better for some cases of X forwarding; see the docstring for the new option.

Emacs now comes with Org v9.8.

See the file `"etc/ORG-NEWS"` for user-visible changes in Org.

New user option 'compilation-search-extra-path'.

`compile.el` will now use paths specified in both `'compilation-search-extra-path'` and `'compilation-search-path'`, when doing search. `'compilation-search-extra-path'` is consulted first. One possible use case of this option is to add new search paths on a per-project basis with directory-local variables.

Editing Changes in Emacs 31.1

Commands for keyboard translation.

`'key-translate'` is now interactive. It prompts for a key to translate from, and another to translate to, and sets `'keyboard-translate-table'`. The new command `'key-translate-remove'` prompts for a key/translation pair, with `'completing-read'`, and removes the translation from the translation table.

Internationalization

Emacs now supports Unicode Standard version 17.0.

New input method 'greek-polytonic'.

This input method has support for polytonic and archaic Greek characters.

New language environment and input method for Tifinagh.

The Tifinagh script is used to write the Berber languages.

New input methods for Northern Iroquoian languages.

Input methods are now implemented for Haudenosaunee languages in the Northern Iroquoian language family: `'mohawk-postfix'` (Mohawk [Kanien'kéha / Kanyen'kéha / Onkwehonwehnéha]), `'oneida-postfix'` (Oneida [On'yote'a·ká· / Onyota'a:ká: / Ukwehuwehnéha]), `'cayuga-postfix'` (Cayuga [Gayogo'ho:nqhnéha:]), `'onondaga-postfix'` (Onondaga [Onyda'gegá]), `'seneca-postfix'` (Seneca [Onödowá'ga:']), and `'tuscarora-postfix'` (Tuscarora [Skarù·rę]). Additionally, there is a general-purpose `'haudenosaunee-postfix'` input method to facilitate writing in the orthographies of the six languages simultaneously.

New input methods for languages based on Burmese.

These include: Burmese, Burmese (visual order), Shan, and Mon.

New language environment and input methods for Syriac languages.

A new language environment for languages that use the Syriac script: Classical Syriac, Aramaic, and others. Two new input methods for these languages: Syriac and Syriac (phonetic).

'visual-wrap-prefix-mode' now supports variable-pitch fonts.

When using `'visual-wrap-prefix-mode'` in buffers with variable-pitch fonts, the wrapped text will now be lined up correctly so that it is exactly below the text after the prefix on the first line.

New commands 'unix-word-rubout' and 'unix-filename-rubout'.

Unix-words are words separated by whitespace regardless of the buffer's `syntax table`. In a Unix terminal or shell, `'C-w'` kills by Unix-word. The new commands `'unix-word-rubout'` and `'unix-filename-rubout'` allow you to bind keys to operate more similarly to the terminal.

New user option 'kill-region-dwim'.

This option, if non-nil, modifies the fall-back behavior of `'kill-region'` (`'C-w'`) if no region is active, and will kill the last word instead of raising an error. Note that if you have disabled Transient Mark mode you might prefer to use `'unix-word-rubout'`, as this feature relies on there being an active region.

New user option 'delete-pair-push-mark'.

This option, if non-nil, makes `'delete-pair'` push a mark at the end of the region enclosed by the deleted delimiters. This makes it easy to act on that region. For example, we can highlight it using `'C-x C-x'`.

Electric Pair mode

Electric Pair mode can now pair multiple delimiters at once.

You can now insert or wrap text with multiple sets of parentheses and other matching delimiters at once with Electric Pair mode, by providing a prefix argument when inserting one of the delimiters.

Electric Pair mode now supports multi-character paired delimiters.

`'electric-pair-pairs'` and `'electric-pair-text-pairs'` now allow using strings for multi-character paired delimiters.

To use this, add a list to both electric pair user options: `'("/*" . "*/")'`.

You can also specify to insert an extra space after the first string pair: `'("/*" " */" t)'`.

New user option 'electric-indent-actions'.

This user option specifies a list of actions to reindent. The possible elements for this list are: `'yank'`, reindent the yanked text; `'before-save'`, indent the whole buffer before saving it.

You can now use `'M-~'` during `'C-x s'` (`'save-some-buffers'`).

Typing `'M-~'` while saving some buffers means not to save the buffer and also to mark it as unmodified. This is an alternative way to mark a buffer as unmodified which doesn't require switching to that buffer.

New minor mode `'delete-selection-local-mode'`.

This mode sets `'delete-selection-mode'` buffer-locally. This can be useful for enabling or disabling the features of `'delete-selection-mode'` based on the state of the buffer, such as for the different states of modal editing packages.

New user option `'exchange-point-and-mark-highlight-region'`.

When set to nil, this modifies `'exchange-point-and-mark'` so that it doesn't activate the mark if it is not already active. The default value is t, which retains the old behavior. This variable has no effect when Transient Mark mode is off.

You can now use `'M-s t'` to swap FROM and TO during `'query-replace'`.

Likewise during `'query-replace-regexp'`. The original binding of `'M-s'` (`'next-matching-history-element'`) is now available on `'M-s M-s'` or `'M-s s'` for query replace minibuffer input.

New commands for filling text using semantic linefeeds.

The new command `'fill-paragraph-semelf'` fills a paragraph of text using `"semantic linefeeds"`, whereby a newline is inserted after every sentence. The new command `'fill-region-as-paragraph-semelf'` fills a region of text using semantic linefeeds as if the region were a single paragraph. You can set the variable `'fill-region-as-paragraph-function'` to the value `'fill-region-as-paragraph-semelf'` to enable functions like `'fill-paragraph'` and `'fill-region'` to fill text using `"semantic linefeeds"`.

Temporary files are named differently when `'file-precious-flag'` is set.

When the user option `'file-precious-flag'` is set to a non-nil value, Emacs now names the temporary file it creates while saving buffers using the original file name with `".tmp"` appended to it. Thus, if saving the buffer fails for some reason, and the temporary file is not renamed back to the original file's name, you can easily identify which file's saving failed.

`'C-u C-x .'` clears the fill prefix.

You can now use `'C-u C-x .'` to clear the fill prefix, similarly to how you could already use `'C-u C-x C-n'` to clear the goal column.

New prefix argument for 'C-/' in Direed and Proced modes.

The Direed and Proced major modes bind mode-specific undo commands to the same keys to which 'undo' is globally bound, 'C-/', 'C-_' and 'C-x u'. These commands did not previously accept a prefix argument. Now a numeric prefix argument specifies a repeat count, just like it already did for 'undo'.

New minor mode 'center-line-mode'.

This mode keeps modified lines centered horizontally according to the value of 'fill-column', by calling 'center-line' on each non-empty line of the modified region.

New command 'unfill-paragraph'.

This is the inverse of 'M-q' ('fill-paragraph').

Changes in Specialized Modes and Packages in Emacs 31.1

Project

New command 'project-root-find-file'.

It is equivalent to running 'project-any-command' with 'find-file'.

New command 'project-customize-dirlocals'.

It is equivalent to running 'project-any-command' with 'customize-dirlocals'.

Improved prompt for 'project-switch-project'.

The prompt now displays the chosen project on which to invoke a command.

'project-prompter' values may be called with up to three arguments.

These allow callers of the value of 'project-prompter' to specify a prompt string; prompt the user to choose between a subset of all the known projects; and disallow returning arbitrary directories. See the docstring of 'project-prompter' for a full specification of these new optional arguments.

'project-current' has a new optional argument, MAYBE-PROMPT.

If 'project-current' is called with this argument non-nil, then it is passed to the 'project-prompter' to use as a prompt string. Callers can use this to indicate the reason for which or context in which Emacs should ask the user to select a project.

New command 'project-find-matching-buffer'.

It can be used when switching between projects with similar file trees (such as Git worktrees of the same repository). It supports being invoked standalone or from the 'project-switch-commands' dispatch menu.

New variable 'project-find-matching-buffer-function'.

Major modes can set this to major mode-specific functions to control how `'project-find-matching-buffer'` finds matching buffers.

New user option 'project-list-exclude'.

This user option describes projects that should always be skipped by `'project-remember-project'`.

New user option 'project-prune-zombie-projects'.

This user option controls the automatic deletion of projects from `'project-list-file'` that cannot be accessed when prompting for a project.

The value must be an alist where each element must be in the form:

(WHEN . PREDICATE)

where WHEN specifies where the deletion will be performed, and PREDICATE is a function which takes one argument, and must return non-nil if the project should be removed.

New command 'project-save-some-buffers' bound to 'C-x p C-x s'.

This is like `'C-x s'`, but only for this project's buffers.

'project-remember-project' can now be called interactively.

'project-shell' and 'project-eshell' support numeric prefix buffer naming.

They now accept numeric prefix arguments to select or create numbered shell sessions. For example, `'C-2 C-x p s'` switches to or creates a buffer named `"*name-of-project-shell<2>*"`. By comparison, a plain universal argument as in `'C-u C-x p s'` always creates a new session.

'project-switch-to-buffer' re-uniquifies buffer names while prompting.

When `'uniquify-buffer-name-style'` is non-nil, `'project-switch-to-buffer'` changes the buffer names to only make them unique within the given project, during completion. That makes some items shorter.

'project-switch-to-buffer' uses 'project-buffer' as completion category.

The category defaults are the same as for `'buffer'` but any user customizations would need to be re-added.

'project-mode-line' can now show project name only for local files.

If the value of `'project-mode-line'` is `'non-remote'`, project name and the Project menu will be shown on the mode line only for projects with local files.

The "VC-aware" project backend caches the current project and its name.

The duration for which the values are cached depends on whether it is called from `'non-essential'` context, and it is determined by variables `'project-vc-cache-timeout'` and `'project-vc-non-essential-cache-timeout'`.

Network Security Manager (NSM)

NSM warns about TLS 1.1 by default.

It has been deprecated by RFC 8996, published in 2021.

NSM warns about DHE and RSA key exchange by default.

Emacs now warns about ephemeral Diffie-Hellman key exchange, and static RSA key exchange, also when `'network-security-level'` is customized to its default `'medium'` value.

Etags

New command-line options for handling unrecognized programming languages.

The new command-line option `'--no-fallback-lang'` disables attempts to parse as Fortran or C/C++ files whose programming language `'etags'` could not determine. This allows to avoid false positives and reduces the time required to scan directories with many such files. Another new option `'--no-empty-file-entries'` disables generation of file entries in tags tables for files in which no tags were found.

Delete Selection mode

New face 'delete-selection-replacement' for the replacement text.

This comes with a change to how we track what is considered `"the replacement text"`, which should be more robust now, and is made more clear by the highlighting.

Editorconfig

'editorconfig-apply' is declared obsolete.

You can now use `'editorconfig-display-current-properties'` without having to call `'editorconfig-apply'`.

Auth Source

Non-existing or empty files are ignored in 'auth-sources'.

File-based data stores are ignored in `'auth-sources'`, if the underlying data file does not exist. This is relevant, if a new secret is stored in such a file; the first usable entry of `'auth-sources'` is selected as target. If you want also not existing files to be selected, set the user option `'auth-source-ignore-non-existing-file'` to `nil`.

'auth-sources' set to nil means using the password cache only.

Autoinsert

New condition for 'auto-insert-alist'.

`'auto-insert-alist'` now also allows to have a predicate taking no argument as conditions. These types of conditions should be declared with `'(predicate FUNCTION)'`.

This allows to trigger `'auto-insert'` with finer grained control.

Register

New commands `'buffer-to-register'` and `'file-to-register'`.

These allow users to interactively store files and buffers in registers. Killed buffers stored in a register using `'buffer-to-register'` are automatically converted to a file-query value if the buffer was visiting a file.

The `"*Register Preview*"` buffer shows only suitable registers.

That was already the case for the `"fancy"` UI but is now also true in the default UI you get, i.e., when `'register-use-preview'` is `'traditional'`.

The `"*Register Preview*"` buffer shows sorted items.

Tree-sitter

New user option `'treesit-enabled-modes'`.

You can customize it either to enable all available tree-sitter based modes, or to select a list of tree-sitter based modes to enable. Depending on customization, it modifies the variable `'major-mode-remap-alist'` from the corresponding variable `'treesit-major-mode-remap-alist'` prepared by tree-sitter based mode packages.

New user option `'treesit-auto-install-grammar'`.

It controls the automatic installation of tree-sitter grammar libraries needed for tree-sitter based modes, if these grammar libraries are not available when such modes are turned on.

`'treesit-extra-load-path'` now is a customizable user option.

The first directory in the list is used as the default directory to install the language grammar when `'treesit-auto-install-grammar'` is `'ask'`, `'ask-dir'` or `'always'`.

`'treesit-language-source-alist'` supports keywords.

The language and URL are mandatory, but remaining data can use keywords like

```
(json "https://github.com/tree-sitter/tree-sitter-json" :commit "4d770d3")
```

The file `treesit-x.el` defines a number of simple tree-sitter modes.

Using the new macro `'define-treesit-generic-mode'`, generic modes are defined including, but not limited to, `'gitattributes-generic-ts-mode'`. Visiting a file in such mode asks for confirmation before installing its tree-sitter grammar. Then it highlights the visited file according to the syntax defined by the grammar.

New command `'treesit-cycle-sexp-thing'`.

It cycles the type of navigation for commands that move across sexp's and lists, such as `'treesit-forward-sexp'`, `'treesit-forward-list'`, `'treesit-down-list'`, and

'treesit-up-list'. The type can be either 'list', the default, or 'sexp'. With the default 'list' type, these commands move using syntax tables for symbols and using the thing 'list' for lists. With the 'sexp' type, these commands move across nodes defined by the tree-sitter thing 'sexp' in 'treesit-thing-settings'.

Indirect buffers can have their own parser list.

Before, indirect buffers share their base buffer's parser list and parsers. Now they can have their own parser list.

New variable 'treesit-language-remap-alist'.

This variable allows a user to remap one language into another, such that creating a parser for language A actually creates a parser for language B. By extension, any font-lock rules or indentation rules for language A will be applied to language B instead.

This is useful for reusing font-lock rules and indentation rules of language A for language B, when language B is a strict superset of language A.

New accessor functions for each setting in 'treesit-font-lock-settings'.

Now users can access a setting's query, feature, enable flag, and override flag by 'treesit-font-lock-setting-query', 'treesit-font-lock-setting-feature', 'treesit-font-lock-setting-enable', and 'treesit-font-lock-setting-override'.

New tree-sitter thing 'list'.

Unlike the existing thing 'sexp' that defines both lists and atoms, 'list' defines only lists to be navigated by 'forward-sexp'. The new command 'treesit-forward-sexp-list' uses 'list' to move across lists. But to move across atoms inside the list it uses 'forward-sexp-default-function'.

New tree-sitter based functions for moving by lists.

If a major mode defines 'list' in 'treesit-thing-settings', tree-sitter setup for these modes sets 'forward-list-function' to 'treesit-forward-list', 'up-list-function' to 'treesit-up-list', and 'down-list-function' to 'treesit-down-list'. This enables the 'forward-list', 'up-list', and 'down-list' motion commands for those modes.

Tree-sitter enabled modes now properly support 'show-paren-mode'.

They do that by letting 'show-paren-mode' use the results of parsing by the tree-sitter library. The new function 'treesit-show-paren-data' is used to communicate the tree-sitter parsing results to 'show-paren-mode'.

Tree-sitter enabled modes now properly support 'hs-minor-mode'.

All commands from hideshow.el can selectively display blocks defined by the new tree-sitter thing 'list'.

New tree-sitter thing 'comment'.

The new variable `'forward-comment-function'` is set to the new function `'treesit-forward-comment'` if a major mode defines the thing `'comment'`.

New function `'treesit-query-eagerly-compiled-p'`.

This function returns non-nil if a query was eagerly compiled.

New function `'treesit-query-source'`.

This function returns the string or sexp source query of a compiled query.

New function `'treesit-language-display-name'`.

This new function returns the display name of a language given the language symbol. For example, `'cpp'` is translated to `"C++"`. A new variable `'treesit-language-display-name-alist'` holds the translations of language symbols where that translation is not trivial.

New function `'treesit-merge-font-lock-feature-list'`.

This function merges two tree-sitter font-lock feature lists. It returns a new font-lock feature list with no duplicates in the same level. It can be used to merge font-lock feature lists in a multi-language major mode.

New function `'treesit-replace-font-lock-feature-settings'`.

Given two tree-sitter font-lock settings, it replaces the feature in the second font-lock settings with the same feature in the first font-lock settings. In a multi-language major mode it is sometimes necessary to replace features from one of the major modes with others, that are better suited to the new multilingual context.

New function `'treesit-simple-indent-modify-rules'`.

Given two tree-sitter indent rules, it replaces, adds, or prepends rules in the old rules with new ones, then returns the modified rules. In a multi-language major mode it is sometimes necessary to modify rules from one of the major modes to better suit the new multilingual context.

New variable `'treesit-aggregated-simple-imenu-settings'`.

This variable allows major modes to setup Imenu for multiple languages.

New variable `'treesit-aggregated-outline-predicate'`.

This variable allows major modes to setup `'outline-minor-mode'` for multiple languages.

New function `'treesit-simple-indent-add-rules'`.

This new function makes it easier to customize indent rules for tree-sitter modes.

New variable `'treesit-simple-indent-override-rules'`.

Users can customize this variable to add simple custom indentation rules for tree-sitter major modes.

New variable 'treesit-languages-require-line-column-tracking'.

Now Emacs can optionally track line and column numbers for buffer edits and send that information to tree-sitter parsers. Parsers of languages in this list will receive line and column information. This is only needed for very few languages. So far only Haskell is known to need it.

New function 'treesit-tracking-line-column-p'.

New function to check if a buffer is tracking line and column for buffer edits.

New function 'treesit-parser-tracking-line-column-p'.

New function to check if a parser is receiving line and column information.

'treesit-language-at-point-function' is now optional.

Multi-language major modes can rely on the default return value from 'treesit-language-at' that uses the new function 'treesit-parsers-at'.

New function 'treesit-query-with-optional'.

When used in 'treesit-font-lock-rules', 'treesit-query-with-optional' returns a default query plus the valid queries from a list of optional queries.

New function 'treesit-query-with-fallback'.

When used in 'treesit-font-lock-rules', 'treesit-query-with-fallback' selects the first valid query from a list.

Tree-sitter thing functions now work better with multiple parsers.

The following functions now handle better the case when there are multiple parsers at point: 'treesit-thing-prev', 'treesit-thing-next', 'treesit-navigate-thing', 'treesit-thing-at'. When there are multiple parsers at point, instead of using whatever 'treesit-node-at' returns at point, these functions now try every relevant parser in descending order of relevance. (Deeper-embedded parsers have higher relevance.) These functions now also take an additional optional argument, `PARSER`, that allows caller to specify a parser or language to use. That also means 'treesit-beginning/end-of-defun' now can move across parsers.

New command 'treesit-explore'.

This command replaces 'treesit-explore-mode'. It turns on 'treesit-explore-mode' if it is not on, and pops up the explorer buffer if it is already on.

'treesit-explore-mode' now supports local parsers.

Now 'treesit-explore-mode' (or 'treesit-explore') prompts for a parser rather than a language, and it is now possible to select a local parser at point to explore.

Tree-sitter query predicates ':equal', ':match', and ':pred' are deprecated.

Use ':eq?', ':match?', and ':pred?' instead. The change is because newer tree-sitter libraries mandate query predicates to end with a question mark. Emacs will

transparently convert `'equal'`, `'match'`, and `'pred'` to `'eq?'`, `'match?'`, and `'pred?'`, respectively, so existing queries still work fine with the latest tree-sitter library. The predicate `'equal'` is changed to `'eq?'` to better follow tree-sitter's convention. Also, the `'match?'` predicate can now take the regexp as either the first or second argument, so it works with both tree-sitter convention (regexp arg second) and Emacs convention (regexp arg first).

Track changes

New variable `'track-changes-undo-only'` to distinguish undo changes.

Hideshow

New command `'hs-cycle'`.

This command cycles the visibility state of the current block between hide the parent block, hide the nested blocks only and show all the blocks.

New command `'hs-toggle-all'`.

This command hides or shows all the blocks in the current buffer.

`'hs-hide-level'` no longer hides all the blocks in the current buffer.

If `'hs-hide-level'` was not inside a code block, it would hide all the blocks in the buffer like `'hs-hide-all'`. Now it should only hide all the second level blocks.

New user option `'hs-display-lines-hidden'`.

If this option is non-nil, Hideshow displays the number of hidden lines next to the ellipsis. By default this is disabled.

New user option `'hs-show-indicators'`.

This user option determines if Hideshow should display indicators to show and toggle the block hiding. If non-nil, the indicators are enabled. By default this is disabled.

New user option `'hs-indicator-maximum-buffer-size'`.

This user option limits the display of Hideshow indicators to buffers that are not too large. By default, buffers larger than 2MB have the indicators disabled; the value of nil will activate the indicators regardless of the buffer size.

New user option `'hs-indicator-type'`.

This user option determines which indicator type should be used for the block indicators.

The possible values can be: `'fringe'`, display the indicators in the fringe (the default); `'margin'`, display the indicators in the margin; nil, display the indicators at end-of-line.

The new icons `'hs-indicator-show'` and `'hs-indicator-hide'` can be used to customize the indicators appearance only if `'hs-indicator-type'` is set to `'margin'` or `nil`.

The hiding behavior of some hideshow commands has changed.

`'hs-hide-block'`, `'hs-hide-level'`, `'hs-cycle'` and `'hs-toggle-hiding'` now hide the innermost block to which the current line belongs instead of the block after cursor position.

To restore the old behavior, set the new user option `'hs-hide-block-behavior'` to `'after-cursor'`.

New user option 'hs-cycle-filter' for visibility-cycling with 'TAB'.

This user option controls the positions on the headline of hideable blocks where the `'TAB'` key cycles the blocks' visibility.

The variable 'hs-special-modes-alist' is now obsolete.

Instead of customizing Hideshow for a mode by setting the elements of `'hs-special-modes-alist'`, such as `START`, `COMMENT-START`, `FORWARD-SEXP-FUNC`, etc., major mode authors should set the corresponding buffer-local variables `'hs-block-start-regexp'`, `'hs-c-start-regexp'`, `'hs-forward-sexp-function'`, etc.

'hs-hide-level' can now hide comments too.

This is controlled by `'hs-hide-comments-when-hiding-all'`.

New minor mode 'hs-indentation-mode'.

This buffer-local minor mode configures `'hs-indentation-mode'` to detect blocks based on indentation.

The new user option `'hs-indentation-respect-end-block'` can be used to adjust the hiding range for this minor mode.

C-ts mode

New user option 'c-ts-mode-enable-doxygen'.

By default, this is `nil`, and the Doxygen comment blocks in C/C++ sources are highlighted like other comments. When non-`nil`, Doxygen comment blocks are syntax-highlighted if the Doxygen grammar library is available.

Csharp-ts mode

Renamed feature in 'treesit-font-lock-feature-list'.

The feature `'property'` has been renamed to `'attribute'`, since this is what it is called in the general C# community.

Go-ts mode

New unit test commands.

Three new commands are now available to run unit tests.

The `'go-ts-mode-test-function-at-point'` command runs the unit test at point. If a region is active, it runs all the unit tests under the region. It is bound to `'C-c C-t t'` in `'go-ts-mode'`.

The `'go-ts-mode-test-this-file'` command runs all unit tests in the current file. It is bound to `'C-c C-t f'` in `'go-ts-mode'`.

The `'go-ts-mode-test-this-package'` command runs all unit tests under the package of the current buffer. It is bound to `'C-c C-t p'` in `'go-ts-mode'`.

The `'go-ts-mode-build-tags'` user option is available to set a list of build tags for the test commands.

The `'go-ts-mode-test-flags'` user option is available to set a list of additional flags to pass to the go test command line.

Lua-ts mode***New user option 'lua-ts-auto-close-block-comments'.***

When non-nil, inserting a block comment `--[[` will close it by inserting its respective `]]`. By default, this is disabled.

Java-ts mode***New user option 'java-ts-mode-enable-doxygen'.***

By default, this is nil, and the Doxygen comment blocks in Java sources are highlighted like other comments. When non-nil, Doxygen comment blocks are syntax-highlighted if the Doxygen grammar library is available.

New user option 'java-ts-method-chaining-indent-offset'.

Now method chaining is indented by 8 spaces rather than 4, and this option controls how much is indented for method chaining.

JSON-ts mode***New command 'json-ts-jq-path-at-point'.***

This command copies the path of the JSON element at point to the kill-ring, formatted for use with the `'jq'` utility.

PHP-ts mode

'php-ts-mode' now depends on 'mhtml-ts-mode'.

The direct dependency on `'js-ts-mode'`, `'css-ts-mode'` and `'html-ts-mode'` has now been replaced by `'mhtml-ts-mode'`. Navigation, Outline and Imenu work for all languages, and code maintenance is easier.

'php-ts-mode-run-php-webserver' can now accept a custom "php.ini" file.

You can use the new optional argument `CONFIG` when calling `'php-ts-mode-run-php-webserver'` to pass an alternative `"php.ini"` file to the built-in Web server. Interactively, when invoked with a prefix argument, `'php-ts-mode-run-php-webserver'` prompts for the config file as well as for other connection parameters.

The user option 'php-ts-mode-css-fontify-colors' has been removed.

`'mhtml-ts-mode-css-fontify-colors'` replaces this option.

New user option 'php-ts-mode-html-relative-indent'.

In buffers containing both PHP and HTML, this option allows you to define how the PHP code should be indented relative to the position of the HTML tags.

New user option 'php-ts-mode-html-indent-offset'.

Offset of PHP code block relative to HTML tags.

New user option 'php-ts-mode-find-sibling-rules'.

Rules for finding siblings of a PHP file.

New user option 'php-ts-mode-phpdoc-highlight-errors'.

When non nil, it highlights unknown PHPDOC tags using `'font-lock-warning-face'` so that the user can identify them more easily.

New command 'php-ts-mode-show-ini'.

Show the location of the PHP ini files. If the current buffer is associated to a remote PHP file, show the remote PHP ini files.

Rust-ts mode

New user option 'rust-ts-mode-fontify-number-suffix-as-type'.

Rust number literals may have an optional type suffix. When this option is non-nil, this suffix is fontified using `'font-lock-type-face'`.

YAML-ts mode

New user option 'yaml-ts-mode-yamllint-options'.

Additional options for `'yamllint'`, the command used for Flymake's YAML support.

EIEIO

New value 'warn' for 'eieio-backward-compatibility'.

This is the new default value and causes warnings to be emitted at run-time for the use of the associated deprecated features. `'(setq eieio-backward-compatibility t)'` can be used to recover the previous silence.

Text mode

New commands to convert between ASCII and full-width characters.

New commands `'fullwidth-region'` and `'fullwidth-word'` convert ASCII characters in region or in the word at point to the corresponding full-width characters, which are customarily used instead of ASCII characters in CJK texts. For example, `'A'` is converted to `'𐀀'`, `'1'` is converted to `'𐀁'`, etc. Companion commands `'halfwidth-region'` and `'halfwidth-word'` perform the opposite conversion.

Texinfo mode

Texinfo mode now can auto-close the ``' pairs.

Now inserting ```` in `'texinfo-mode'` will close it by inserting its respective `'`, if `'electric-pair-mode'` is enabled.

ASM mode

'asm-mode-set-comment-hook' is obsolete.

You can now set `'asm-comment-char'` from `'asm-mode-hook'` instead.

Ibuffer

New column 'recency' in Ibuffer display.

The user option `'ibuffer-formats'` configures the Ibuffer formats. Add `'recency'` to the format to display the column.

New value 'title' for the user option 'ibuffer-use-header-line'.

Display column titles in the header line if `'ibuffer-use-header-line'` is set to `'title'`.

New user option 'ibuffer-human-readable-size'.

When non-nil, buffer sizes are shown in human readable format.

'define-ibuffer-op' prompts can now be functions.

The prompts `'opstring'` and `'active-opstring'` can now either be strings or functions. This is useful when your prompts can benefit from dynamic content.

New Ibuffer-dedicated faces.

New faces `'ibuffer-marked'`, `'ibuffer-deletion'`, `'ibuffer-title'`, and `'ibuffer-filter-group-name'`. By default, they inherit from the general-purpose faces Ibuffer previously used, to preserve previous behavior.

ElDoc

New ElDoc function 'elisp-eldoc-funcall-with-docstring'.

This function includes the current function's docstring in the ElDoc echo area and can be used as a more detailed alternative to 'elisp-eldoc-funcall'.

New user option 'elisp-eldoc-funcall-with-docstring-length'.

This user option specifies how long function docstrings must be displayed in 'elisp-eldoc-funcall-with-docstring'. If set to 'short' (the default), only display the first sentence of the docstring. Otherwise, if set to 'full', display the full docstring.

New user option 'elisp-eldoc-docstring-length-limit'.

This user option controls the maximum length of docstrings in character units that 'elisp-eldoc-funcall-with-docstring' and 'elisp-eldoc-var-docstring-with-value' will show. By default, it is set to 1000 characters.

Buffer Menu

New user option 'Buffer-menu-human-readable-sizes'.

When non-nil, buffer sizes are shown in human readable format. The default is nil, which retains the old format.

Term

The terminal emulator now supports auto-margins control.

Term mode now handles DECAWM escape sequences that control whether text automatically wraps at the right margin:

- \e[?7h enables auto-margins (default) - \e[?7l disables auto-margins

When auto-margins is disabled, characters that would go beyond the right margin are discarded, which matches the behavior of physical terminals and other terminal emulators. Control sequences and escape sequences are still processed correctly regardless of margin position.

SMerge mode

New 'repeat-map' for SMerge conflict resolution commands.

With 'repeat-mode' enabled, after invoking an SMerge command (for example, 'C-c ^ n'), you can repeat further SMerge commands by typing just the final key (for example, 'n', 'p', 'u', 'l').

New command 'smerge-extend' extends a conflict over surrounding lines.

New command 'smerge-refine-exchange-point' to jump to the other side.

When used inside a refined chunk, it jumps to the matching position in the "other" side of the refinement: if you are in the new text, it jumps to the corresponding position in the old text and vice versa.

New user option 'smerge-refine-shadow-cursor'.

When 'smerge-refine' shows the conflict diffs at word granularity, a "shadow cursor" is now displayed in the "lower" version when point is in the "upper" version, and vice versa. The "shadow cursor" is just the character corresponding to the position where 'smerge-refine-exchange-point' would jump, shown in a new distinct face 'smerge-refine-shadow-cursor', by default a box face.

'smerge-refine-regions' can compare regions in different buffers.

Cursor Sensor mode

New direction 'moved' used when the cursor moved within the active area.

Image Dired

'image-dired-show-all-from-dir' takes the same first argument as 'dired'.

This allows passing a string with wildcards, or a cons cell where the first element is a list and the rest is a list of files.

Bound unused letters in 'image-dired-thumbnail-mode-map'.

For a more comfortable navigation experience (as in, no modifier keys), the keys 'f', 'b', 'n', 'p', 'a' and 'e' are now bound to the same functions as their 'C-' counterparts.

Browse URL

New user option 'browse-url-transform-alist'.

This user option is an alist that allows transforming URLs before asking a web browser to load them. For example, it could be used like this:

```
(add-to-list 'browse-url-transform-alist '("www.google.com" . "www.duckduckgo.com"))
```

New command 'browse-url-qutebrowser' for Qutebrowser.

For better integration with Qutebrowser, set 'browse-url(-secondary)-browser-function' to 'browse-url-qutebrowser'.

New GTK-native launch mode.

For better Wayland support, the pgtk toolkit exposes a new 'x-gtk-launch-uri' browse-url handler and uses it by default when URLs are browsed from a PGTK frame. For other frames, we fall back to the default URL launch function. This change allows us to properly raise browser windows under Wayland using the xdg_activation_v1 protocol.

'RET' can visit URLs in read-only buffers.

In some keymaps such as `'ansi-osc-hyperlink-map'`, `'browse-url-button-map'`, `'goto-address-highlight-keymap'`, `'bug-reference-map'` it is possible now to visit URLs by typing just `'RET'` instead of `'C-c RET'` in read-only buffers.

Removed support for some obsolete web browsers.

Conkeror (obsolete since Emacs 28.1), gnome-moz-remote (obsolete since Emacs 25.1), and gnudoit (obsolete since Emacs 25.1).

'browse-url-firefox-program' now supports LibreWolf and Zen Browser.

LibreWolf, Floorp and Zen Browser, three popular Firefox forks, have been added to the programs that are automatically recognizable as Firefox workalikes. Emacs will set `'browse-url-firefox-program'` to the first one of these found on your system.

CL-Lib

Derived types (i.e. 'cl-deftype') can now be used as method specializers.

Some cl-lib functions and macros are now built-in.

These functions or macros have been added to Emacs Lisp, and the old names are now aliases for the built-in equivalents:

- `'cl-incf` renamed to `'incf` - `'cl-decf` renamed to `'decf` - `'cl-oddp` renamed to `'oddp` - `'cl-evenp` renamed to `'evenp` - `'cl-plusp` renamed to `'plusp` - `'cl-minusp` renamed to `'minusp` - `'cl-member-if` renamed to `'member-if`

`'cl-member-if` is marked obsolete. The other names are deprecated too, and will be marked as obsolete in a future release.

'cl-labels' now also accepts '(FUNC EXP)' bindings, like 'cl-flet'.

Such bindings make it possible to compute which function to bind to FUNC.

'cl-block' names are now lexically scoped, as documented.

'cl-locally' is now obsolete.

It is an alias for the `'progn` special-form.

'cl-declare' is now obsolete; use 'defvar' instead.

'cl-gensym' is now obsolete; use 'gensym' instead.

New macro 'cl-with-accessors'.

This macro is similar to `'with-slots'`, but uses accessor functions instead of slot names. It is useful when slots' accessor functions are used repeatedly, such as reading from a slot and then writing to that slot. Symbol macros are created for the accessor functions using `'cl-symbol-macrolet'`, so that they can be used with `'setq` and `'setf`.

Whitespace

'whitespace-cleanup' now adds missing newline at end of file.

If `'whitespace-style'` includes `'missing-newline-at-eof'` (which is the default), the `'whitespace-cleanup'` function will now add the newline.

'whitespace-mode' now can prettify page delimiter characters ('^L').

If `'page-delimiters'` is set in `'whitespace-style'`, or the new minor mode `'whitespace-page-delimiters-mode'` is on, the page delimiter characters (`'^L'`) are displayed as a pretty horizontal line that spans the entire width of the window. The new `'whitespace-page-delimiter'` face can be used to customize the appearance.

New user option 'whitespace-global-mode-buffers'.

Normally, `'global-whitespace-mode'` skips special buffers whose name starts with an asterisk `"*"`. This user option provides an override: it contains a list of regular expressions used to match the names of special buffers in which `'global-whitespace-mode'` should turn on. The default value preserves the existing exception for the `"*scratch*"` buffer.

Bookmark

Bookmark history now saves each bookmark only once.

Previously, the variable `'bookmark-history'` accumulated duplicate bookmark names when bookmark features were used interactively. This made their history larger than necessary for frequent bookmark users. Bookmark names are now saved uniquely.

New user option 'bookmark-bmenu-type-column-width'.

This user option controls the width of the type column on the bookmark menu `'bookmark-bmenu-list'`. The default value is 8 which is backwards compatible.

New hook 'bookmark-after-load-file-hook'.

This hook is run by `'bookmark-load'` after loading a bookmark file. This hook can be used, for example, to reconcile `'bookmark-alist'` against bookmark state that you, or a package that you use, maintains.

Recentf

You can now regularly auto-save recently opened files.

Customize user option `'recentf-autosave-interval'` to the number of seconds between auto saving recently opened files. For example, customize this variable to 300 to save recently opened files every 5 minutes. From Lisp, use `'setopt'` not `'setq'`.

If `'recentf-autosave-interval'` is nil, auto saving is disabled; this is the default.

This user option is in sympathy with `save-place`, and `savehist` autosave timers.

New user option 'recentf-show-messages'.

'`recentf-save-list`' can print a message when saving the recentf list. The new option, if set to nil, suppresses this message.

New user option '`recentf-suppress-open-file-help`'.

By default, invoking '`recentf-open-files`' displays a message saying what action clicking or typing '`RET`' on the item at point executes, and tabbing between items in the "`*Open Recent*`" buffer likewise displays such messages. To suppress these messages, customize the user option '`recentf-suppress-open-file-help`' to non-nil. The default value of this option is nil.

New user option '`recentf-exclude-ignored-extensions`'.

Add the new predicate function '`recentf-exclude-file-by-extension-p`' to the list that is the value of the user option '`recentf-exclude`' to ignore files with certain extensions. By default, adding this function to '`recentf-exclude`' ignores files whose extensions are listed in '`completion-ignored-extensions`'; you can specify a different list of extensions by customizing the new user option '`recentf-exclude-ignored-extensions`'.

Saveplace

You can now regularly auto-save places.

Customize user option '`save-place-autosave-interval`' to the number of seconds between auto saving places. For example, customize this variable to 300 to save places every 5 minutes. From Lisp, use '`setopt`' not '`setq`'.

If '`save-place-autosave-interval`' is nil, auto saving is disabled; this is the default.

This user option is in sympathy with recentf, and savehist autosave timers.

Savehist

The history file can be modified by external tools.

Emacs can now handle this case gracefully by merging the external and internal history information. This feature is activated only when '`savehist-additional-variables`' is nil.

Savehist no longer saves additional variables more than once.

If you configured '`savehist-additional-variables`' with variables that were also dynamically accumulated in minibuffer history during minibuffer use, they are now saved only once in the file specified by '`savehist-file`'. Previously, they were saved twice.

Rectangle Mark

New user option to control whether empty rectangle selections are shown.

The new user option '`rectangle-indicate-zero-width-rectangle`' can be used to disable the default display of empty rectangular selections. The default is t; set it to nil to

disable the indication (which causes a horizontal shift of text on display, and thus could be annoying).

Message

"In-Reply-To" header contains only a message id.

The "In-Reply-To" header created when replying to a message now contains only the originating message's id, conforming to RFC 5322. The previous behavior included additional information about the originating message. The new user option `'message-header-use-obsolete-in-reply-to'`, nil by default, can be set to a non-nil value to restore the previous behavior.

'message-subject-re-regexp' default value is derived from 'mail-re-regexps'.

`'mail-re-regexps'` is a new user option that is easier to customize than `'message-subject-re-regexp'`. `'message-subject-re-regexp'` is still honored if it was already set.

'message-strip-subject-re' now matches case-insensitively.

'message-change-subject' inserts current subject into "future history".

Hashcash support has been removed.

It is believed to no longer be useful as a method to fight spam. The `'message-generate-hashcash'` option is now obsolete and has no effect.

Gnus

Replying to icalendar events now supports specifying a comment.

When called with a prefix argument, accepting, declining, or tentatively accepting an icalendar event will prompt for a comment to add to the response.

Hashcash support has been removed.

It is believed to no longer be useful as a method to fight spam. The `'spam-use-hashcash'` hook is now obsolete and has no effect.

Add 'M-i' keybinding as the symbolic prefix in the group keymap.

The symbolic prefix is another kind of universal prefix that is used in Gnus, see "(gnus) Symbolic Prefixes" in the Gnus manual.

Sorting selected groups is now possible with 'gnus-topic-mode'.

gnus-dbus.el is now obsolete.

System sleep integration is now independent of D-Bus.

The system sleep integration previously provided by customizing the variable `'gnus-dbus-close-on-sleep'` is now deprecated. A new system using the builtin `system-sleep.el` library is now available by customizing `'gnus-close-on-sleep'`. This

will work on all systems that the `'system-sleep'` library supports.

Sieve

New keybinding to refresh buffer in 'sieve-manage-mode'.

`'sieve-refresh-scriptlist'` is now bound to `'g'` to refresh the contents of the current sieve buffer.

Button

New function 'unbuttonize-region'.

It removes all the buttons in the specified region.

Disabling 'button-mode' now removes all buttons in the current buffer.

Shell

Shell buffers now support bookmarks.

You can now bookmark local and remote shell buffers using the bookmark menu `'bookmark-bmenu-list'`, or by using the command `'bookmark-set'`. Shell bookmarks can be loaded via the menu and by using the command `'bookmark-jump'`, which opens a bookmarked shell, restores its buffer name, its current directory, and creates a remote connection, if necessary. You can customize `'shell-bookmark-name-function'`.

New command to complete the shell history.

`'comint-complete-input-ring'` (`'C-x <up>'`) is like `'minibuffer-complete-history'` but completes on comint inputs.

'ansi-osc-directory-tracker' now respects remote directories.

Remote directories are now retained when changes to `'default-directory'` are detected by this filter. For example, `"/ssh:hostname:/home/username"` would have been stripped to just `"/home/username"` before.

Eshell

New interactive command 'eshell-clear'.

This command scrolls the screen so that only the current prompt is visible, optionally erasing all the previous input/output as well. Previously, the Eshell built-in command `'eshell/clear'` supported this (e.g., to call it via `'M-x'`), but this new command behaves more consistently if you have a partially-typed command at the Eshell prompt.

New user option 'eshell-command-async-buffer'.

This option lets you tell `'eshell-command'` how to respond if its output buffer is already in use by another invocation of `'eshell-command'`, much like `'async-shell-command-buffer'` does for `'shell-command'`. By default, this will prompt for confirmation before creating a new buffer when necessary. To restore the previous

behavior, set this option to `'confirm-kill-process'`.

'eshell-execute-file' is now an interactive command.

Interactively, this now prompts for a script file to execute. With the prefix argument, it will also insert any output into the current buffer at point.

'eshell-command' and 'eshell-execute-file' can now set where stderr goes.

These functions now take an optional `ERROR-TARGET` argument to control where to send the standard error output. See the `"(eshell) Entry Points"` node in the Eshell manual for more details.

You can now loop over ranges of integers with the Eshell 'for' command.

When passing a range like `'BEGIN..END'` to the Eshell `'for'` command, Eshell will now iterate over each integer between `BEGIN` and `END`, not including `END`.

Conditional statements in Eshell now use an 'else' keyword.

Eshell now prefers the following form when writing conditionals:

```
if {conditional} {true-subcommand} else {false-subcommand}
```

The old form (without the `'else'` keyword) is retained for compatibility.

You can now chain conditional statements in Eshell.

When using the newly-preferred conditional form in Eshell, you can now chain together multiple `'if'/'else'` statements. For more information, see `"(eshell) Control Flow"` in the Eshell manual.

Eshell's built-in 'wait' command now accepts a timeout.

By passing `'-t'` or `'--timeout'`, you can specify a maximum time to wait for the processes to exit. Additionally, you can now wait for external processes by passing their PIDs.

New hook 'eshell-after-initialize-hook'.

This hook runs after an Eshell session has been fully initialized, immediately before running `'eshell-post-command-hook'` for the first time.

Improved history Isearch.

History Isearch in Eshell is reworked. Two new commands `'eshell-isearch-backward-regexp'` and `'eshell-isearch-forward-regexp'` are added for incrementally searching through the input history. `'eshell-isearch-backward-regexp'` is bound to `'M-r'` by default, and `'M-s'` is freed for normal search commands. If you would like to restore the previous key-bindings for the non-incremental search commands, put in your configuration:

```
(with-eval-after-load 'em-hist (keymap-set eshell-hist-mode-map "M-r"  
#eshell-previous-matching-input) (keymap-set eshell-hist-mode-map "M-s"
```

```
#'eshell-next-matching-input))
```

New user option 'eshell-history-isearch'.

When `'eshell-history-isearch'` is nil (the default), Isearch commands search in the buffer contents. If you customize it to t, those commands only search in input history. If you customize it to the symbol `'dwim'`, those commands search in input history only when the point is after the last prompt.

Eshell 'alias' command now sorts the alias list.

When adding an alias interactively, Eshell now sorts the list of aliases before saving the alias file. This maintains the stability of the list of aliases to make the diff between versions more readable if you store your aliases in version control.

Mail Utils

New user option 'mail-re-regexps'.

This contains the list of regular expressions used to match `"Re:"` and international variants of it when modifying the Subject field in replies.

Mairix

'mairix-search' now keeps its own minibuffer history.

Imap

'imap-authenticate' can now use PLAIN authentication.

"AUTH=PLAIN" support is auto-enabled if the IMAP server supports it. If you do not wish to use "AUTH=PLAIN", pass a specific authentication type to `'imap-open'` for `'imap-authenticate'` to use, or remove `'plain'` from `'imap-authenticators'`.

Rmail

'rmail-re-abbrevs' default value is now derived from 'mail-re-regexps'.

`'mail-re-regexps'` is a new user option that is easier to customize than `'rmail-re-abbrevs'`. `'rmail-re-abbrevs'` is still honored if it was already set.

New user options for formatting Rmail summary lines.

`'rmail-summary-sender-function'` and `'rmail-summary-recipient-function'` control how the sender/recipient fields are displayed in the summary. `'rmail-summary-address-width'` controls the width of that field.

New user option 'rmail-mime-save-action'.

This option specifies an action to take after saving a MIME attachment. Predefined values include visiting the file in Emacs, jumping to the file in Dired, or opening the file with an external program. You can also provide a custom function.

Rmail now detects email messages from suspicious sender addresses.

If the "From" header of a message contains a suspicious email address, Rmail will now highlight it in a distinct face and provide a 'help-echo' tooltip explaining the reason. (What exactly is considered as suspicious email addresses is determined by the function 'textsec-suspicious-p', which see.) This is controlled by the new user option 'rmail-detect-suspicious-headers', whose default value is non-nil; customize to nil to disable the check.

Sendmail

Sending an email via sendmail.el checks for suspicious addressees.

The command 'mail-send', used to send email in Mail mode, now checks the addressees for suspicious email addresses. If such addresses are found, the command will show them and the reason they are considered suspicious, and will request a confirmation before sending the message. This follows the behavior of Message mode, and affects users who customize 'mail-user-agent' to the value 'sendmail-user-agent'.

SHR

SHR now slices large images into rows.

Sliced images allow for more intuitive scrolling up/down by letting you scroll past each slice, instead of jumping past the entire image. Previously, SHR sliced images when zoomed to their original size, no matter how large or small that was. Now, SHR slices any images taller than 'shr-sliced-image-height'. For more

information, see the "(eww) Advanced" node in the EWW manual.

You can now customize the image zoom levels to cycle through.

By customizing 'shr-image-zoom-levels', you can change the list of zoom levels that SHR cycles through when calling 'shr-zoom-image'.

New user option 'shr-fill-text'.

When 'shr-fill-text' is non-nil (the default), SHR will fill text according to the width of the window. If you customize it to nil, SHR will leave the text as-is; in that case, EWW will automatically enable 'visual-line-mode' when displaying a page so that long lines are visually wrapped at word boundaries.

EWW

EWW now enables 'visual-wrap-prefix-mode' when 'shr-fill-text' is nil.

By default, 'shr-fill-text' is t, and EWW fills the text according to the width of the window. If you customize 'shr-fill-text' to nil, EWW will now automatically turn on 'visual-wrap-prefix-mode' in addition to 'visual-line-mode', so that long lines are wrapped at word boundaries near window edge and the continuation lines are indented using prefixes computed from surrounding context.

New user option 'eww-guess-content-type-functions'.

The value is a list of functions that EWW should call to determine the content-type of Web pages which don't have a valid 'Content-Type' header. The default value is a function that considers a page with an HTML 'doctype' declaration to have context-type "text/html".

'eww-switch-to-buffer' falls back to 'eww'.

When there is no EWW buffer, 'eww-switch-to-buffer' falls back to calling 'eww'.

URL Queue***'url-queue-retrieve' now makes use of some url request variables.***

The variables 'url-request-data', 'url-request-method', and 'url-request-extra-headers' can now be bound around a call to 'url-queue-retrieve'. Binding them has the same effect as for 'url-retrieve'.

CC mode***New type of 'c-offsets-alist' element.***

The 'cdr' of such an alist element may now be a syntactic symbol. A source line with a syntactic element whose symbol is the 'car' of that alist element is indented as though it were the 'cdr'.

Enums now have their own syntactic symbols.

The new symbols 'enum-open', 'enum-close', 'enum-intro' and 'enum-entry' are used in the analysis of enum constructs. Previously, they were given 'brace-list-open', etc. These are fully described in the "(ccmode) Enum Symbols" node of the CC mode manual.

Enums are now, by default, indented like classes, not brace-lists.

To get the old behavior back, add an element '(enum-open . brace-list-open)' to 'c-offsets-alist' in your CC mode style, or amend 'c-offsets-alist' likewise in any of the other ways detailed in the "(ccmode) Config Basics" node of the CC mode manual.

Emacs Lisp mode***Semantic highlighting support for Emacs Lisp.***

'emacs-lisp-mode' can now use code analysis to highlight more symbols more accurately. Customize the new user option 'elisp-fontify-semantically' to non-nil to enable this feature, and see the Info node "(emacs) Semantic Font Lock" for more information.

Checkdoc no longer warns about missing footer lines in some cases.

Emacs Lisp libraries have traditionally ended with a footer line (sometimes referred to as "terminating comment"). Their purpose was to easily detect files that had been truncated in transit on ancient and less reliable connections:

```
;; some-cool-package.el ends here
```

'**checkdoc**' will no longer warn if that line is missing for packages that explicitly only support Emacs 30.1 or later, as specified in the "**Package-Requires**" header. The reason for keeping the warning for packages that support earlier versions of Emacs is that package.el in those versions can't install packages where that line is missing.

This change affects both '**checkdoc**' and the corresponding flymake backend.

Checkdoc no longer warns about wide docstrings.

The Checkdoc warning for wide docstrings duplicates the byte-compiler warning added in Emacs 28.1. This redundancy is now removed.

New user option 'checkdoc-arguments-missing-flag'.

Set this to nil to disable warnings for function arguments that are not documented in docstrings.

Checkdoc will now flag incorrect formatting in warnings.

This affects calls to '**warn**', '**lwarn**', '**display-warning**', and '**message-box**'.

New user option 'checkdoc-allow-quoting-nil-and-t'.

Customizing this option to a non-nil value stops checkdoc from warning for docstrings where symbols '**nil**' and '**t**' are in quotes.

The default of 'checkdoc-verb-check-experimental-flag' is now nil.

In most cases, having it enabled leads to a large amount of false positives.

New function 'checkdoc-batch'.

It checks the buffer in batch mode, prints all found errors and signals the first found error.

New file-local variable 'lisp-indent-local-overrides'.

This variable can be used to locally override the indent specification of symbols.

IELM

IELM input history is now saved also when the IELM process is killed.

When you kill the IELM process with '**C-c C-c**', the input history is now saved to the file specified by '**ielm-history-file-name**', just like when you exit the Emacs session or kill the IELM buffer.

New value 'point' for user option 'ielm-dynamic-return'.

When '**ielm-dynamic-return**' is set to '**point**', typing '**RET**' has dynamic behavior based on whether point is inside an sexp. While point is inside an sexp typing '**RET**' inserts a newline, and otherwise Emacs proceeds with evaluating the expression. This is useful when '**electric-pair-mode**', or a similar automatic pairing mode, is enabled.

Doc View

Dedicated buffer for plain text contents.

When switching to the plain text contents with `'doc-view-open-text'`, Doc View now creates a dedicated buffer to display it. `'C-c C-c'` gets you back to the real Doc View buffer if it still exists.

New commands to save and restore pages in buffer-local registers.

Doc View can store the current page to buffer-local registers with the new command `'doc-view-page-to-register'` (bound to `'m'`), and later the stored page can be restored with `'doc-view-jump-to-register'` (bound to `''`).

Doc View can generate imenu indices for DjVu and ODF documents.

When the `'djvused'` program is available, Doc View can now generate an imenu index for DjVu files from its outline. Indices for Open Document Format (ODF) files as used by OpenOffice and LibreOffice are generated using the `'mutool'` program after their initial conversion to PDF format. The name of the `'djvused'` program can be customized by changing the user option `'doc-view-djvused-program'`.

Ispell

The default value of 'ispell-help-timeout' has changed.

The default value is now 30 seconds, as the old value was too short to allow reading the help text.

Ispell can now save spelling corrections as abbrevs.

In the Ispell command loop, type `'C-u'` immediately before selecting a replacement to toggle whether that correction will be saved as a global abbrev expansion for its misspelling. The new user option `'ispell-save-corrections-as-abbrevs'` determines whether abbrev saving is enabled by default.

Flyspell

New user option 'flyspell-delay-use-timer'.

By default, Flyspell waits after so-called `"delayed"` commands by calling `'sit-for'`. If you customize this option to non-nil, Flyspell instead sets up a timer to perform spell-checking after a short delay, which allows idle timers and other code to run during this delay period. We consider making this behavior the default in a future Emacs version, so we invite Flyspell users to enable this new option and report issues.

'turn-on-flyspell' and 'turn-off-flyspell' are obsolete.

To unconditionally enable `'flyspell-mode'` from a hook, use this instead:

```
(add-hook 'text-mode-hook #'flyspell-mode)
```

Tramp

Tramp signals 'remote-file-error' in case of connection problems.

This is a subcategory of 'file-error'. Therefore, all checks for 'file-error' in 'condition-case', 'ignore-error', 'error-conditions' and alike still work.

New command 'tramp-cleanup-bufferless-connections'.

Connection-related objects for which no associated buffers exist, except for Tramp internal buffers, are flushed. This is helpful to prune connections after you close remote-file buffers without having to either cherry-pick via 'tramp-cleanup-connection' or clear them all via 'tramp-cleanup-all-connections'.

External methods can now be used in multi-hop connections.

This is implemented for 'tramp-sh' methods, like `"/scp:user@host|sudo::"`.

New command 'tramp-dired-find-file-with-sudo'.

This command, bound to '@' in Dired, visits the file or directory on the recent Dired line with superuser, or root, permissions.

'C-x x @' is now bound to 'tramp-revert-buffer-with-sudo'.

You can use 'C-u C-x x @' to select a Tramp method other than the default, "sudo".

'tramp-file-name-with-method' can now be set as connection-local variable.***New optional connection methods "surs" and "sudors".***

These connection methods are similar to "su" and "sudo", but they use the modern 'su-rs' and 'sudo-rs' commands.

Connection method "kubernetes" supports now optional namespace.

The host name for Kubernetes connections can be of kind [CONTAINER.]POD[%NAMESPACE], in order to specify the namespace to be used. This overrides the setting in 'tramp-kubernetes-namespace', if any.

Different proxies for the same destination host name can be specified.

A typical example are docker containers, which run on different hosts under the same docker name. When the user option 'tramp-show-ad-hoc-proxies' is non-nil, such ad-hoc multi-hop file names can be used in parallel. Example: on both remote hosts "host1" and "host2" there is a docker container "name", respectively:

```
/ssh:user1@host1|docker:name: /ssh:user2@host2|docker:name:
```

This feature is experimental.

Implementation of filesystem notifications for connection method "smb".***Remote process support has been rewritten for the "smb" connection method.***

For more information, see "(tramp) Running remote processes on MS Windows hosts" in the Tramp manual.

New functions to extend the set of operations with a remote implementation.

The new functions `'tramp-add-external-operation'` and `'tramp-remove-external-operation'` allow to add an implementation for other operations but the defined set of magic file name operations. This can be used by external ELPA packages for performance optimizations in special cases. For more information, see `"(tramp) New operations"` in the Tramp manual.

New user option 'tramp-propagate-emacsclient-tramp'.

When this option is non-nil, Tramp propagates the environment variable `EMACSCCLIENT_TRAMP` with a proper value to remote processes. This is helpful if you want to start emacsclient on a remote host from a process started inside Emacs.

Isearch and Replace

Typing 'd' during 'query-replace' shows the diff buffer with replacements.

Diff

'diff-mode' now refrains from automatically refining big hunks.

What is big is defined by the new `'diff-refine-threshold'` user option.

New command 'diff-kill-ring-save'.

This command copies to the `'kill-ring'` a region of text modified according to diffs in the current buffer, but without applying the diffs to the original text. If the selected range extends a hunk, the command attempts to look up and copy the text in between the hunks.

New command 'diff-revert-and-kill-hunk' bound to 'u' and 'C-c M-u'.

This command reverts the hunk at point (i.e., applies the reverse of the hunk), and then removes the hunk from the diffs. This is useful to undo or revert changes, committed and uncommitted, when you are in buffers generated by `'C-x v ='` and `'C-x v D'`. When the region is active, the command reverse-applies and kills hunks that the region overlaps.

'v' is now bound to 'vc-next-action' in read-only Diff mode buffers.

'diff-file-prev' and 'diff-hunk-prev' always move to start of header.

Previously, `'diff-file-prev'` and `'diff-hunk-prev'` would move when point is after the corresponding file or hunk header, but not when inside it. Now they will always move to the start of the current header.

New command 'diff-delete-other-hunks' bound to 'C-c RET n'.

This command deletes all hunks other than the current hunk. It is useful to prepare a `"*vc-diff*"` buffer for committing a single hunk. When the region is active, it deletes all hunks that the region does not overlap.

'vc-version-diff' and 'vc-root-version-diff' changed default for REV1.

They suggest the previous revision as the default for REV1, not the last one as before. This makes them different from `'vc-diff'` and `'vc-root-diff'` when those are called without a prefix argument.

'diff-apply-hunk' now supports creating and deleting files.

`'diff-apply-buffer'` supports creating files but not deleting them, yet.

Diff mode's application and killing commands now consider the region.

If the region is active, `'diff-apply-hunk'`, `'diff-apply-buffer'` and `'diff-hunk-kill'` now apply or kill all hunks that the region overlaps. Otherwise, they have their existing behavior.

'diff-apply-buffer' can reverse-apply.

With a prefix argument, it now reverse-applies hunks. This matches the existing prefix argument to `'diff-apply-hunk'`.

's' is now bound to 'diff-split-hunk' in read-only Diff mode buffers.**Ediff*****Ediff's copy commands now apply to all changes with 'C-u' prefix.***

The Ediff copy commands, bound to `'a'`, `'b'`, `'ab'`, etc., now copy all changes when supplied with a universal prefix argument via `'C-u'`:

- `'C-u a'` copies all changes from buffer A to buffer B (in 2-way diff) or to buffer C (in 3-way diff or merge). - `'C-u b'` copies all changes from buffer B to buffer A (in 2-way diff) or to buffer C (in 3-way diff or merge). - `'C-u a b'` copies all changes from buffer A to buffer B. - `'C-u b a'` copies all changes from buffer B to buffer A. - `'C-u a c'` copies all changes from buffer A to buffer C. - `'C-u b c'` copies all changes from buffer B to buffer C. - `'C-u c a'` copies all changes from buffer C to buffer A. - `'C-u c b'` copies all changes from buffer C to

buffer B.

Ediff now supports more flexible custom window layouts.

Custom implementations of `'ediff-window-setup-function'` no longer need to display `*all*` Ediff windows. Any of the A, B, C, and control windows can be left undisplayed and the corresponding variable set to nil. This change enables custom layouts without a control panel window.

Dired***New user option 'dired-create-empty-file-in-current-directory'.***

When non-nil, `'dired-create-empty-file'` creates a new empty file and adds an entry for it (or its topmost new parent directory if created) under the current subdirectory in

the Dired buffer by default (otherwise, it adds the new file (and new subdirectories if provided) to whichever directory the user enters at the prompt). When nil, `'dired-create-empty-file'` acts on the default directory by default.

Note that setting this user option to non-nil makes invoking `'dired-create-empty-file'` outside of a Dired buffer raise an error (like other Dired commands that always prompt with the current subdirectory, such as `'dired-create-directory'`).

New user option 'dired-check-symlinks' allows disabling validity checks.

Dired uses `'file-truename'` to check symbolic link validity when fontifying them, which can be slow for remote directories. Setting `'dired-check-symlinks'` to nil disables these checks. Defaults to t, can be set as a connection-local variable.

New user option 'dired-hide-details-hide-absolute-location'.

When Dired's `'dired-hide-details-mode'` is enabled, also hide the `'default-directory'` absolute location, typically displayed as the first line in a Dired buffer.

With `'dired-hide-details-hide-absolute-location'`:

```
project: (100 GiB available)
```

Without `'dired-hide-details-hide-absolute-location'`:

```
/absolute/path/to/my/important/project: (100 GiB available)
```

Clicking on base name of directory reverts buffer.

When `'dired-make-directory-clickable'` is non-nil, clicking on the base name of the directory now reverts the Dired buffer.

'dired-copy-filename-as-kill' supports project-relative names.

With a new value of the prefix argument (1), this command copies file names relative to the root directory of the current project.

Warning when Dired displays a file name with a literal newline.

If Dired uses an `'ls'` implementation that supports the `'-b'` switch, then on visiting a directory that contains a file whose name has a newline, and Dired displays that character as a literal newline, Emacs now

automatically pops up a buffer warning that such a display can be problematic for Dired and showing a way to change the display to use the unproblematic character `'\n'`.

New user option 'dired-auto-toggle-b-switch'.

When this user option is non-nil and Dired uses an `'ls'` implementation that supports the `'-b'` switch and `'dired-listing-switches'` does not include the `'-b'` switch, then on visiting a directory containing a file whose name has a newline, Emacs automatically adds the `'-b'` switch and redisplay the directory in Dired to show `'\n'` in the file

name instead of a literal newline. This prevents executing many Direed operations on such a file from failing and signaling an error. The default value of this user option is nil.

New Direed handling of errors from 'ls'.

When invoking a Direed command causes 'ls' to emit an error message, Emacs now displays the message in a popped up buffer instead of outputting it in the Direed buffer and signaling an error.

Grep

Grep results can be edited to reflect changes in the originating file.

Like Occur Edit mode, typing 'e' in the "`*grep*`" buffer will now make the 'grep' results editable. The edits will be reflected in the buffer visiting the originating file. Typing '`C-c C-c`' will leave the Grep Edit mode.

Imenu

New user option 'imenu-allow-duplicate-menu-items'.

This specifies whether Imenu can include duplicate menu items. Duplicate items are now allowed by default (option value t), which restores the behavior before Emacs 29. Customize this to nil to get the behavior of Emacs 29 and Emacs 30.

Time Stamp

'time-stamp' can up-case, capitalize and down-case date words.

This control can be useful in languages in which days of the week and/or month names are capitalized only at the beginning of a sentence. For details, see the built-in documentation for user option '`time-stamp-format`'.

Because this feature is new in Emacs 31.1, do not use it in the local variables section of any file that might be edited by an older version of Emacs.

Some historical 'time-stamp' conversions now warn.

'`time-stamp-pattern`' and '`time-stamp-format`' had quietly accepted several '`time-stamp`' conversions (e.g., "`:%y`") that have been deprecated since Emacs 27.1. These now generate a warning with a suggested migration.

Merely having '`(add-hook 'before-save-hook 'time-stamp)`' in your Emacs init file does not expose you to this change. However, if you set '`time-stamp-format`' or '`time-stamp-pattern`' with a file-local variable, you may be asked to update the value.

TeX modes

New Xref backend for TeX modes.

The new backend (`'tex-etags'`) is on by default, and improves the functionality of the standard `'xref'` commands in TeX buffers. You can restore the standard `'etags'` backend with the `'xref-etags-mode'` toggle.

BibTeX mode

New user options facilitate customization of BibTeX and biblatex entries.

Entry definitions via the user options `'bibtex-BibTeX-aux-entry-alist'`, `'bibtex-biblatex-aux-entry-alist'`, `'bibtex-BibTeX-aux-opt-alist'`, and `'bibtex-biblatex-aux-opt-alist'` take precedence over `'bibtex-BibTeX-entry-alist'` and `'bibtex-biblatex-entry-alist'`. These user options now support the definition of aliases that inherit the definition of another entry.

'bibtex-user-optional-fields' has been renamed to 'bibtex-aux-opt-alist'.

The old name is an obsolete alias.

'bibtex-include-OPTkey' is now obsolete and its default is nil.

Use `'bibtex-aux-opt-alist'` instead.

New user option 'bibtex-entry-ask-for-key'.

When enabled, `'bibtex-entry'` asks for a key.

'bibtex-string-file-path' and 'bibtex-file-path' are lists of directories.

For backward compatibility, considered obsolete, these user options may still be strings with colon separated lists of directories.

Midnight mode

Change for activating the mode.

Putting `'(require 'midnight)'` in your init file no longer activates the mode. Now, one needs to say `'(midnight-mode +1)'` instead.

Python mode

New 'repeat-map' for Python indentation commands.

The commands `'python-indent-shift-left'` and `'python-indent-shift-right'` can now be repeated using `'repeat-mode'`. With `'repeat-mode'` enabled, after invoking one of these commands via `'C-c <'` or `'C-c >'`, you can press `'<'` or `'>'` to repeat the command.

Prefer "python" for 'python-interpreter' and 'python-shell-interpreter'.

On recent versions of mainstream GNU/Linux distributions, `"python"` either does not exist or it points to Python 3. These user options now default to using `"python"`, falling back to `"python3"` if it does not exist. If `"python"` points to Python 2 on your system, you now have to customize these variables to `"python3"` if you want to use Python 3 instead.

Python 2 support is now optional and disabled by default.

Since Python 2 EOL was over 5 years ago, this release removes Python 2-only builtins such as `"file"` from the default highlighting in `'python-mode'` and `'python-ts-mode'`. If you would like them highlighted, customize the new user option `'python-2-support'` to a non-nil value and restart Emacs.

Support of `'electric-layout-mode'` added.***`'DEL'` now deletes the text in the active region.***

When point is between indentation, the command `'python-indent-dedent-line-backspace'` (by default bound to `'DEL'`) now deletes the text in the region and deactivates the mark if Transient Mark mode is enabled, the mark is active, and prefix argument is 1.

`'python-eldoc-function-timeout'` now accepts floating-point numbers.

To allow for finer-grained adjustment of timeout for `'python-eldoc-function'`, `'python-eldoc-function-timeout'` now accepts floating-point numbers as well as integers.

The default value of `'python-shell-completion-setup-code'` is changed.

A new function is added to the setup code. Users who modify this option may need to update the value for the Python shell completion to work.

Tmm Menubar

A new shortcut to navigate to previous menu.

The hardcoded `'^'` shortcut gets you back to the previous menu.

New user option `'tmm-shortcut-inside-entry'`.

When non-nil, highlight the character shortcut in the menu entry's string instead of prepending it and `'tmm-mid-prompt'` to said entry.

Foldout

Improved behavior of `'foldout-exit-fold'` with negative prefix argument.

When `'foldout-exit-fold'` is called with a negative argument (so that the exited fold remains visible), the position of point and window view are preserved.

New command `'foldout-widen-to-current-fold'`.

This command widens the view to the current fold level when in a fold, or behaves like `'widen'` if not in a fold.

MPC

New user option `'mpc-notifications'`.

When non-nil, MPC (the Emacs front-end to Music Player Daemon) displays a desktop notification when the song changes, using `'notifications-notify'`. The notification's

title and body can be customized using the new user options 'mpc-notifications-title' and 'mpc-notifications-body'.

New user option 'mpc-crossfade-time'.

When non-nil, MPC will crossfade between songs for the specified number of seconds. Crossfading can be toggled using the command 'mpc-toggle-crossfade' or from the MPC menu.

New command 'mpc-describe-song'.

This command displays information about the currently playing song or song at point in the "*MPC-Songs*" buffer. The list of tags to display can be customized using the new user option 'mpc-song-viewer-tags' and the appearance of the list with the new faces 'mpc-table-key', 'mpc-table-value', and 'mpc-table-empty'.

New command 'mpc-server-stats'.

This command displays information about the connected MPD server. The appearance of the list can be customized with the new faces 'mpc-table-key' and 'mpc-table-value'.

VC

Printing root branch logs has moved to 'C-x v b L'.

Previously, the command to print the root log for a branch was bound to 'C-x v b l'. It has now been renamed from 'vc-print-branch-log' to 'vc-print-root-branch-log', and bound to 'C-x v b L'. This is more consistent with the rest of the 'C-x v' keymap, and makes room for a new fileset-specific branch log command.

To undo this change you can use

```
(keymap-global-set "C-x v b l" #'vc-print-root-branch-log) (with-eval-after-load
'vc-dir (keymap-set vc-dir-mode-map "b l" #'vc-print-root-branch-log))
```

New command 'C-x v b L' ('vc-print-fileset-branch-log').

This command prints the log of VC changes to the current fileset on another branch.

VC Annotate for Mercurial repositories shows changeset hashes.

To restore showing revision numbers instead of changeset hashes, customize the new user option 'vc-hg-annotate-show-revision-numbers' to non-nil.

New commands to handle repositories with multiple working trees.

Some VCS support more than one working tree with the same backing revisions store, such as with Git's 'worktree' subcommand and Mercurial's 'share' extension. Emacs now has some commands to manage other working trees:

- 'C-x v w c': Add a new working tree.
- 'C-x v w w': Visit this file in another working tree.
- 'C-x v w k': Kill buffers visiting this file in other working trees.
- 'C-x v w s': Like 'C-x p p' but limited to other working trees.
- 'C-x v w a': Copy or

move fileset changes to another working tree. - `'C-x v w A'`: Copy or move all changes to another working tree. - `'C-x v w x'`: Delete a working tree you no longer need. - `'C-x v w R'`: Relocate a working tree to another file name.

The new user option `'vc-no-confirm-moving-changes'` controls whether `'C-x v w a'` and `'C-x v w A'` ask for confirmation when moving changes between working trees. The default is to ask for confirmation.

In addition, Lisp programs that extend VC can invoke the new backend functions to obtain a list of other working trees, and to add, remove and relocate them.

Using 'e' from Log View mode to modify change comments now works for Git.

New user option `'vc-allow-rewriting-published-history'`.

Some VCS commands can change your copy of published change history without warning. In VC we try to detect before that happens, and stop. You can customize this option to permit rewriting history even though Emacs thinks it is dangerous.

`'vc-clone'` is now an interactive command.

When called interactively, `'vc-clone'` now prompts for the remote repository address, and the directory into which to clone the repository. It tries to automatically determine the VC backend for cloning, or prompts for that, too.

`'vc-clone'` now accepts an optional argument `OPEN-DIR`.

When the argument is non-nil, the function switches to a buffer visiting the directory into which the repository was cloned.

`'vc-revert'` is now bound to `'@'` in VC Directory.

`'vc-revert'` is now additionally bound to `'C-x v @'`.

This is in addition to `'C-x v u'`.

`'vc-revert'` now works on directories listed in VC Directory.

Reverting a directory means reverting changes to all files inside it.

New global minor mode `'vc-auto-revert-mode'`.

This is like `'global-auto-revert-mode'` but limited to VCS-tracked files. As compared with VC's existing, default support for reverting files after VCS operations, the new mode is a more reliable way to ensure that Emacs reverts buffers visiting tracked files when VCS operations change the contents of those files.

New commands to cherry-pick and revert revisions.

The commands `'vc-cherry-pick'`, `'vc-revert-or-delete-revision'`, `'vc-revert-revision'` and `'vc-delete-revision'` let you copy revisions between branches, revert and delete revisions. From Log View buffers, you can use

'C' to cherry-pick the revision at point or all marked revisions, and 'R' to undo the revision at point or all marked revisions.

New commands to rewind branches.

In Log View mode, 'x' deletes revisions newer than the revision at point from the history of the current branch, though without undoing the changes made by those revisions to the working tree. 'X' is similar except that it does remove the changes from the working tree.

New command 'log-edit-done-strip-cvs-lines'.

This command strips all lines beginning with "CVS:" from the buffer. It is intended to be added to the 'log-edit-done-hook' so that 'vc-cvs-checkin' behaves like invoking 'cvs commit [files...]' from the command line.

New user options 'vc-resolve-conflicts' and 'vc-*-resolve-conflicts'.

They control whether to mark a conflicted file as resolved when saving. You can now control it globally, with 'vc-resolve-conflicts', or for specific backends with 'vc-bzr-resolve-conflicts', 'vc-hg-resolve-conflicts', and 'vc-svn-resolve-conflicts'.

New value for 'vc-git-resolve-conflicts'.

The option now accepts the symbol 'default' as a value, which is its default value. Effectively, the default value hasn't changed, since 'vc-resolve-conflicts' defaults to t, the previous default value for 'vc-git-resolve-conflicts'.

VC Directory can now automatically add and remove marks on other lines.

When you try to use a mark or unmark command where doing so would only be permitted if other lines were marked or unmarked first, Emacs will now ask you if you'd like to change the marks on those other lines. For example, if you try to mark a file contained within a directory that is already marked, Emacs will offer to unmark the directory, first. Previously, Emacs would simply refuse to make any changes. You can customize 'vc-dir-allow-mass-mark-changes' to restore the old behavior or dispense with the prompting.

'C-x v x' and VC Directory's 'd' command can now delete unregistered files.

Previously, these commands could only delete registered files. To restore the old, more limited behavior for VC Directory, you can do

```
(keymap-set vc-dir-mode-map "d" #'vc-dir-clean-files)
```

New VC Directory bindings 'z d' and 'D' to delete Git stashes.

These correspond to the existing 'z p' to pop a stash and 'P' to pop the stash at point (deleting the stash at point is also bound to 'C-k').

New VC Directory command 'V' ('vc-dir-root-next-action').

This is like `'v'` (`'vc-next-action'`) but applies to the whole VC Directory buffer, ignoring the position of point and any marks. This is useful to check in all local changes at once.

VC Directory can now register files when checking in mixed filesets.

Previously, if some files to be checked in were unregistered but others were added, removed or edited, Emacs would refuse to proceed. Now Emacs prompts to register the unregistered files, so that all files in the fileset are in a compatible state for a checkin.

'C-x v v' handles missing and removed files more consistently.

Missing files are those which have been removed from the filesystem but which are still tracked by version control. Removed files are those scheduled to be removed from version control in the next commit. Previously, different backends were inconsistent about applying these statuses to files, and `'C-x v v'` behaved subtly differently for the two statuses. The combination of these differences between backends and in `'C-x v v'` behavior was confusing. Now,

- in VC Directory, you can use `'C-x v v'` on missing files to mark them as removed - when committing, you can include missing files in a set of files with different statuses, just like you've always been able to include removed files.

In addition, the Git backend has been fixed to display missing files as `'missing'` instead of incorrectly subsuming them to the `'removed'` status.

There is still some further work to do to rationalize VC's handling of file removal.

New user option 'vc-dir-auto-hide-up-to-date'.

If you customize this option to `'revert'`, the `'g'` command to refresh the VC Directory buffer also has the effect of the `'x'` command. That is, typing `'g'` refreshes the buffer and also hides items in the `'up-to-date'` and `'ignored'` states. If you customize this option to any other non-nil value, then in addition, hide items whenever their state would change to `'up-to-date'` or `'ignored'`.

New user option 'vc-dir-save-some-buffers-on-revert'.

If you customize this option to non-nil, Emacs will offer to save relevant buffers before generating the contents of a VC Directory buffer (like the third-party package Magit does with its status buffer).

New commands to report incoming and outgoing diffs.

`'vc-root-diff-incoming'` and `'vc-root-diff-outgoing'` report diffs of all the changes that would be pulled and would be pushed, respectively. They are the diff analogues of the existing commands `'vc-root-log-incoming'` and `'vc-root-log-outgoing'`.

In particular, `'vc-root-diff-outgoing'` is useful as a way to preview your push and ensure that all and only the changes you intended to include were committed and will be

pushed.

'vc-diff-incoming' and 'vc-diff-outgoing' are similar but limited to the current VC fileset.

New commands to report information about unintegrated changes.

'C-x v T =' ('vc-diff-unintegrated') and 'C-x v T D' ('vc-root-diff-unintegrated') report diffs of changes since the merge base with the remote branch, including uncommitted changes. 'C-x v T l' ('vc-log-unintegrated') and 'C-x v T L' ('vc-root-log-unintegrated') show the corresponding revision logs. These are useful to view all outstanding (unmerged, unpushed) changes on the current branch. They are also available as 'T =', 'T D', 'T l' and 'T L' in VC Directory buffers.

'C-x v T R =' ('vc-diff-remote-unintegrated'), 'C-x v T R D' ('vc-root-diff-remote-unintegrated'), 'C-x v T R l' ('vc-log-remote-unintegrated') and 'C-x v T R L' ('vc-root-log-remote-unintegrated') are corresponding commands which report information about the remote versions of a topic branch.

New commands to report combined diffs of all local changes.

'C-x v E =' ('vc-diff-outgoing-and-edited') and 'C-x v E D' ('vc-root-diff-outgoing-and-edited') report combined diffs of all outgoing changes plus any uncommitted changes. They are useful to show all work that's present only locally.

New user option 'vc-use-incoming-outgoing-prefixes'.

If this is customized to non-nil, 'C-x v I' and 'C-x v O' become prefix commands, such that the new incoming and outgoing commands have global bindings:

- 'C-x v I L' is bound to 'vc-root-log-incoming' - 'C-x v I D' is bound to 'vc-root-diff-incoming' - 'C-x v O L' is bound to 'vc-root-log-outgoing' - 'C-x v O D' is bound to 'vc-root-diff-outgoing'.

New display of outgoing revisions count in VC Directory.

If there are outgoing revisions, VC Directory now includes a count of how many in its headers, to remind you to push them. You can disable this by customizing 'vc-dir-show-outgoing-count' to nil.

New user option 'vc-async-checkin' to enable async checkin operations.

Currently only supported by the Git and Mercurial backends.

New user option 'vc-display-failed-async-commands'.

If non-nil, displays the buffer with the output of the failed command when an asynchronous VC command (e.g., pulls and pushes) fails.

New 'log-edit-hook' option to display diff of changes to commit.

You can customize 'log-edit-hook' to include its new 'log-edit-maybe-show-diff' option to enable displaying a diff of the changes to be committed in a window. This is like

the `'C-c C-d'` command in Log Edit mode buffers, except that it does not select the `"*vc-diff*" buffer's window`, and so works well when added to `'log-edit-hook'`.

'vc-rename-file' is now bound to 'C-x v R'.

'vc-annotate' now abbreviates the Git revision in more cases.

In Emacs 30, `'vc-annotate'` gained the ability to abbreviate the Git revision in the buffer name. Now, it also abbreviates the Git revision when visiting other revisions, such as with `'vc-annotate-revision-previous-to-line'`.

New buffer-local variable 'vc-buffer-overriding-fileset'.

Primarily intended for buffers not visiting files, this specifies the VC backend and VCS-managed file name or file names to which the buffer's contents corresponds. It overrides the behavior of `'vc-deduce-fileset'`.

This replaces and generalizes the old `'vc-annotate-parent-file'`.

New buffer-local variable 'vc-buffer-revision'.

This specifies the revision to which the buffer's contents corresponds. This replaces and generalizes the old `'vc-annotate-parent-rev'`.

vc-dav.el is now obsolete.

The 'log-incoming' and 'log-outgoing' functions are deprecated.

Backend authors should implement the `'incoming-revision'` and `'mergebase'` backend functions instead. These are jointly sufficient to support the `'C-x v I'` and `'C-x v O'` commands.

Marking revisions in Log View now works more like other modes.

Previously, `'m'` toggled whether the current revision was marked, and didn't advance point. Now `'m'` only adds marks, `'u'` removes marks, and both advance point, like how marking works in Dired and VC Directory. You can get back the old behavior with something like this:

```
(with-eval-after-load 'log-view (keymap-set log-view-mode-map "m"
#'log-view-toggle-mark-entry))
```

In addition, a new command `'U'` removes all marks.

New commands 'M-RET', 'M-p' and 'M-n' in Log View mode.

`'M-RET'` expands the current entry, if relevant, and displays its diff in another window. `'M-p'` and `'M-n'` move to the previous and next entries, respectively, expand them if relevant, and display their diffs.

You can use these three commands together to more easily view all the log entries and diffs of a series of revisions: use `'M-RET'` on the first revision, then either `'M-n'` or `'M-p'` repeatedly to view the others.

New command 'w' in Log View mode.

The new command `'log-view-copy-revision-as-kill'`, bound to `'w'` in Log View mode, copies to the kill ring the ID of the revision at point in the log entry. If there are marked revisions, it copies the IDs of those, instead.

New commands 'vc-print-change-log' and 'vc-print-root-change-log'.

These are just like `'vc-print-log'` and `'vc-print-root-log'` except that they have a different prefix argument that some users may prefer. With a prefix argument, these commands prompt for a branch, tag or other reference to a revision to log, and a maximum number of revisions to print. If you find this prefix argument more useful, or more mnemonic, than the prefix arguments that `'vc-print-log'` and `'vc-print-root-log'` already have, consider replacing the default global bindings, like this:

```
(keymap-global-set "C-x v l" #'vc-print-change-log) (keymap-global-set "C-x v L"
#'vc-print-root-change-log)
```

New command alias 'vc-restore' for 'vc-revert'.***The 'diff-restrict-view' command is disabled by default.***

This command is Diff mode's specialized `'narrow-to-region'`. `'narrow-to-region'` has long been disabled by default, so for consistency, `'diff-restrict-view'` is now too. To enable it again, use `'enable-command'`.

'C-x v !' has its own input history.***'C-x v +' for Git pulls from a configured push remote.***

If the current branch has a configured push remote, the default arguments to `'git pull'` will cause a pull from the push remote. You can use `'C-u C-x v +'` to preview or change the arguments.

'C-u C-x v +' and 'C-u C-x v P' for Git have an input history.

This was already in place for Mercurial.

'vc-hg-working-revision' now returns changeset hashes.

Previously, it returned local revision numbers, but hashes are more robust for how this function is typically used.

Package

No longer warn if a package has no footer line.

`package.el` no longer warns for packages without a `"footer line"`, which is the line that usually appears at the very end of an Emacs Lisp file:

```
;;; FILENAME ends here
```

New optional argument to 'package-autoremove'.

An optional argument NOCONFIRM has been added to `'package-autoremove'`. If it is non-nil, or when invoked with a prefix argument, `'package-autoremove'` will not prompt the user for confirmation before removing packages.

'package-install-selected-packages'.

When invoked with a prefix argument, `'package-install-selected-packages'` will not prompt the user for confirmation before installing packages.

'package-refresh-contents' runs asynchronously.

Refreshing the package index will no longer block when invoked interactively.

'package-upgrade' no longer accepts a string argument.

When called from Lisp, it now only accepts a symbol.

'package-install-from-buffer' respects files marked by DireD.

When invoking the command in a DireD buffer with marked files, the command will only copy those files.

'package-isolate' can now also install packages.

If a package is missing, `'package-isolate'` will fetch the missing tarballs and prepare them to be activated in the sub-process.

package-x.el is now obsolete.

The command 'package-vc-install-from-checkout' is now obsolete.

Please use the User Lisp directory instead, see Info node "(emacs) User Lisp Directory" for more details on how to use. This also means that combining the `'use-package'` keywords `':vc'` and `':load-path'` is obsolete.

Package menu now highlights packages marked for installation or deletion.

Package menu now displays the total number of the package type.

Package menu now displays in the mode line the total number of packages installed, total number of packages from all the package archives, total number of packages to upgrade and total number of new packages available.

New functions to query builtin package information.

`'package-versioned-builtin-packages'` returns a list of symbols of built-in packages; `'package-builtin-package-version'` returns the version-list of a given package symbol. These functions provide public interfaces for external tools to query information about built-in packages.

Uninstalling a package now removes its directory from 'load-path'.

Packages can be reviewed before installation or upgrade.

The user option `'package-review-policy'` can configure which packages the user should be allowed to review before any processing takes place. The package review can include reading the downloaded source code, presenting a diff between the downloaded code and a previous installation or displaying a ChangeLog.

New command `'package-autosuggest'`.

Using a built-in database of ELPA package suggestions, this command will install viable packages if no specific major mode is available for the contents of the current buffer.

New minor mode `'package-autosuggest-mode'`.

When enabled, this displays indications about the availability of add-on ELPA packages for the current buffer and suggestions for installing those packages. The default is to show a button on the mode line that can be used to install such packages, but you can customise the presentation style of these suggestions using `'package-autosuggest-style'`.

New user option `'package-retention-policy'`.

This user option controls what previous packages versions to keep on upgrade. By default, this is set to nil, to keep the previous behavior.

Packages are now checked for recursive dependencies before installing.

If a package has dependencies not available on any of the archives in `'package-archives'`, it will appear as unavailable, with the reasons stated in its description.

Rcirc

Authentication via NickServ can access passwords with `auth-source.el`.

For details, consult `'rcirc-authinfo'`.

Xref

Xref commands that jump to some location use `'display-buffer'`.

The commands that jump to some location use `'display-buffer'` and specify the category `'xref-jump'`. As a result, you can customize how the destination window is chosen using `'display-buffer-alist'`. Example:

```
(setq display-buffer-alist '(((category . xref-jump) (display-buffer-reuse-window
display-buffer-use-some-window) (some-window . mru))))
```

New minor mode `'xref-mouse-mode'`.

This minor mode binds `'xref-find-definitions-at-mouse'` to `'C-<down-mouse-1>'`, allowing you to "control click" to jump to a definition, following the convention from other editors. The global minor mode `'global-xref-mouse-mode'` will enable this in all buffers.

New command 'xref-change-to-xref-edit-mode'.

It is bound to 'e' and it switches an Xref buffer into an "editable" mode like similar features in Occur and Grep buffers.

Revert

Variable 'revert-buffer-in-progress' has been renamed.

The old name, 'revert-buffer-in-progress-p', is kept as obsolete variable alias.

Symbol names with a trailing '-p' are reserved for predicates. Calling a variable like this was a mistake.

Autorevert

New variable 'inhibit-auto-revert-buffers'.

While a buffer is member of this variable, a list of buffers, auto-reverting of this buffer is suppressed.

New macro 'inhibit-auto-revert'.

This macro adds the current buffer to 'inhibit-auto-revert-buffers', runs its body, and removes the current buffer from 'inhibit-auto-revert-buffers' afterwards.

New variable 'auto-revert-buffer-in-progress'.

'auto-revert-buffer' binds this variable to a non-nil value while it is working. This can be used by major mode 'revert-buffer-function' implementations to suppress messages in Auto Revert modes, for example.

Strokes

'strokes-mode' no longer demands the presence of a mouse.

'strokes-mode' now permits itself to be enabled if no mouse is connected, to facilitate enabling 'strokes-mode' in sessions where the availability of a mouse device varies during execution (as is frequently observed on Android).

Yank Media

'yank-media' now auto-selects the most preferred MIME type.

Major-mode authors can customize the variables 'yank-media-autoselect-function' and/or 'yank-media-preferred-types' to change the selection rules.

Remember

Remember mode is now a minor mode.

The 'remember' command enables the major mode set in 'remember-initial-major-mode' and then the 'remember-mode' minor mode in the 'remember-buffer'. This allows users to customize the major mode used to write notes.

New handler that appends remember data in directory.

The `'remember-append-in-data-directory'` handler appends remember data in a file, that file being chosen by the user through the minibuffer.

New prefix map for remember commands.

Meant to be given a global binding convenient to the user. Example:

```
(keymap-global-set "C-c r" 'remember-prefix-map)
```

Speedbar***New commands for Speedbar.***

- `'speedbar-window-mode'` opens Speedbar in a window instead of a frame. - `'speedbar-window'` is an alias for `'speedbar-window-mode'`.

New user options for Speedbar.

- `'speedbar-prefer-window'` tells `'speedbar'` to open a side window instead of a frame. - `'speedbar-window-dedicated-window'` defines whether `'speedbar'` is displayed in a dedicated window. - `'speedbar-window-default-width'` defines the initial width of the `'speedbar-window'`. -

`'speedbar-window-max-width'` defines the maximum width of the `'speedbar-window'` when it is closed and then restored.

'speedbar-easymenu-definition-trailer' is now a function.

Icomplete***Change in meaning of 'icomplete-show-matches-on-no-input' (again).***

For Emacs 28 to Emacs 30, when `'icomplete-show-matches-on-no-input'` was non-nil, `'RET'` had special behavior when the minibuffer's contents was equal to the initial input it had right after minibuffer activation. In that case, `'RET'` would choose the first completion candidate, if there was one, instead of the minibuffer's default value. `'RET'` has now returned to selecting the default value in this case; you can use `'C-j'` to choose the completion under point instead. You can opt back in to the special behavior of `'RET'` like this:

```
(keymap-set icomplete-minibuffer-map "<remap> <minibuffer-complete-and-exit>"
#'icomplete-ret)
```

New user options for 'icomplete-vertical-mode'.

New user options have been added to enhance `'icomplete-vertical-mode'`:

- `'icomplete-vertical-in-buffer-adjust-list'` aligns in-buffer completion to the original cursor column. - `'icomplete-vertical-render-prefix-indicator'` adds a prefix indicator to completion candidates. - `'icomplete-vertical-selected-prefix-indicator'`

specifies the prefix string for the selected candidate. -
'`icomplete-vertical-unselected-prefix-indicator`' specifies the prefix string for unselected candidates.

New faces for '`icomplete-vertical-mode`'.

New faces have been added to '`icomplete-vertical-mode`':

- '`icomplete-vertical-selected-prefix-indicator-face`' controls the appearance of the selected candidate prefix. - '`icomplete-vertical-unselected-prefix-indicator-face`' controls the appearance of unselected candidate prefixes.

Custom

New function '`custom-initialize-after-file-load`'.

Useful to delay initialization to the end of the file, so it can use functions defined later than the variable, as is common for minor modes. '`define-globalized-minor-mode`' now automatically uses it if the '`:init-value`' is non-nil.

New major mode '`Custom-dirlocals-mode`'.

This is intended for customizing directory-local variables in the current directory's "`.dir-locals.el`" file.

New keybinding '`C-c C-k`' for '`Custom-reset-standard`'.

New command '`Custom-goto-first-choice`' (bound to '`C-c C-i`').

When first opening the customization interface for a user option, you can use this command as a shortcut to jump to the first actionable button or field (for instance an on/off button for boolean options, or a text field for other values).

Pulse

New function '`pulse-faces`'.

This function pulses a specified list of faces. The pulse duration is determined by the new user option '`pulse-face-duration`'.

Miscellaneous

The first client frame now shows warnings from daemon startup.

When there are warnings emitted during Emacs startup, usually due to problems in your initialization file, these are shown in a "`*Warnings*`" buffer. Until now such warnings were not made visible in the case that Emacs was started as a daemon. Now the first frame after daemon startup will show the "`*Warnings*`" buffer. So for example, starting Emacs with a command like '`emacsclient -a"" -c`' will now show "`*Warnings*`" just like a plain invocation of '`emacs`' would.

'`tooltip-mode`' now shows tooltips after delay on TTY frames.

Display of tooltips on text-only terminals now happens after `'tooltip-delay'` as it does on GUI terminals. To get back the old behavior, customize the value of `'tooltip-delay'` to zero.

New user option 'follow-mode-prefix-key'.

This user option replaces `'follow-mode-prefix'`, which had to be set before loading Follow mode. This new option allows you to change the prefix even after it was loaded, using `'customize-option'` or `'setopt'`.

cdl.el is now obsolete.

Use `'shell-command'` and `'shell-command-on-region'` instead.

echistory.el is now obsolete.

hashcash.el is now obsolete.

It is believed to no longer be useful as a method to fight spam.

kermit.el is now obsolete.

New user option 'ns-click-through' on Nextstep (GNUstep/Mac OS).

This controls whether activation clicks are passed through to Emacs commands. When nil, clicking on an inactive Emacs frame will only activate it. When t (the default), the click will both activate the frame and be interpreted as a command.

New user option 'global-hl-line-buffers'.

This specifies in which buffers should the `'global-hl-line-mode'` be switched on. The default is all buffers except the minibuffer and the buffers like the completions buffer that enable `'cursor-face-highlight-mode'`.

New value 'window' for the user option 'global-hl-line-sticky-flag'.

Unlike the value t that highlights the line with buffer's point, it highlights the line with window's point. Also it uses the new face `'hl-line-nonselected'` for highlighting the line with window's point in non-selected windows.

New user option 'display-fill-column-indicator-warning'.

Customize it to a non-nil value to have the fill-column indicators change their face if the current line exceeds the `'fill-column'`. The new face `'display-fill-column-indicator-warning-face'` is used to highlight the fill-column indicators. By default, this is disabled.

New function 'flash-face-bell-function'.

This function flashes a face briefly. It is intended to be used in `'ring-bell-function'`.

New function 'flash-echo-area-bell-function'.

This function flashes the current echo area briefly. It is intended to be used in `'ring-bell-function'`.

New user option 'pulse-face-duration'.

This option controls the flash duration for `'flash-face-bell-function'` and `'flash-echo-area-bell-function'`.

New user option 'flash-face-faces'.

This option tells `'flash-face-bell-function'` which faces should flash.

New user option 'flash-face-attributes'.

This option tells `'flash-face-bell-function'` and `'flash-echo-area-bell-function'` which face attributes should be used for flash.

'face-all-attributes' now accepts an optional argument INHERIT.

It has the same meaning as the INHERIT argument to `'face-attribute'`, which already takes this argument for a single attribute. This is useful when you want the face attributes to be absolute and not `'unspecified'`.

New user option 'ffap-prefer-remote-file'.

If non-nil, FFAP always finds remote files in buffers with remote `'default-directory'`. If nil, FFAP finds local files first for absolute file names in above buffers. The default is nil.

New theme 'newcomers-presets'.

This new theme configures user options and minor modes that might interest new users, but would otherwise be too invasive to enable by default. Recall that an Emacs theme is a generic collection of settings, that doesn't have to affect the appearance. See the Info node `"(emacs) Newcomers Theme"` for more information.

Edebug

New command 'edebug-bounce-to-previous-value' (bound to 'P').

This command temporarily displays the outside current buffer with the outside point corresponding to the previous value, where the previous value is what Edebug has evaluated before its last stop point or what the user has evaluated in the context outside of Edebug.

This replaces the binding of command `'edebug-view-outside'` to `'P'`, which is still available on `'v'`.

Flymake

Enhanced 'flymake-show-diagnostics-at-end-of-line'.

The new value `'fancy'` allowed for this user option will attempt to layout diagnostics below the affected line using unicode graphics to point to diagnostic locus.

Enhanced 'flymake-show-buffer-diagnostics'.

The command `'flymake-show-buffer-diagnostics'` is now capable of highlighting a nearby diagnostic in the resulting listing. Additionally, it is bound to mouse clicks on fringe and margin indicators, operating on the diagnostics of the corresponding line. The user may bind it in other situations such as the diagnostic overlay map.

More powerful 'flymake-make-diagnostic' API.

Flymake backends can now specify origin and code attributes, allowing Flymake and other extensions to segregate diagnostics based on this extended information.

New user option 'flymake-diagnostic-format-alist'.

This provides fine-grained control over diagnostic formatting across different contexts, allowing you to specify which components (origin, code, message or one-liner message) appear in each output destination.

Dynamic column sizing in diagnostic listings.

The tabulated listings produced by `'flymake-show-buffer-diagnostics'` and `'flymake-show-project-diagnostics'` now automatically adjust their column widths based on content, optimizing display space and readability.

New value 'auto' of user option 'flymake-indicator-type'.

This value (set by default) tries to use fringes if possible, otherwise falls back to margins.

New user option 'elisp-flymake-byte-compile-executable'.

This allows customizing the Emacs executable used for Flymake byte compilation in `'emacs-lisp-mode'`. This option should be set when editing Lisp code which will run with a different Emacs version than the running Emacs, such as code from an older or newer version of Emacs. This will provide more accurate warnings from byte compilation.

SQLite

SQLite databases can now be opened in read-only mode.

The new optional argument `READONLY` of function `'sqlite-open'` allows to open an existing database only for reading.

'sqlite-open' now recognizes 'file:/' URIs as well as file names.

The `'file:/'` URIs are supported by default. In the unusual case that a normal file name starts with `"file:"`, you can disable the URI recognition by invoking `'sqlite-open'` with the new optional argument `DISABLE-URI` non-nil.

'sqlite-close' now does nothing if the connection is already closed.

GUD

'pdb', 'perldebug', and 'guile' suggest debugging the current file via 'M-n'.

When starting these debuggers (e.g., 'pdb') while visiting a file, pressing 'M-n' in the command prompt suggests a command line including the file name, using the minibuffer's "future history".

Calendar

New command 'calendar-recenter'.

This command recenters the month of the date at point. By default, it is bound to 'C-l' in the calendar buffer.

Mouse wheel bindings for scrolling the calendar.

You can now use the mouse wheel to scroll the calendar by 3 months. With the shift modifier, it scrolls by one month. With the meta modifier, it scrolls by year.

Simpler key bindings for navigation in calendar by months and by years.

The month and year navigation key bindings 'M-}', 'M-{', 'C-x]' and 'C-x [' now have the alternative keys '}', '{', ']' and '['.

Avoid modifying Calendar's user options.

The user options 'calendar-mark-holidays-flag' and 'calendar-mark-diary-entries-flag' are not modified anymore when changing the marking state in the calendar buffer.

New library for iCalendar data.

A new library has been added to the calendar for handling iCalendar (RFC 5545) data. The library is designed for reuse in other parts of Emacs and in third-party packages. Package authors can find the new library in the Emacs distribution under "lisp/calendar/icalendar-*.el".

Most of the functions and variables in the older icalendar.el have been marked obsolete and now suggest appropriate replacements from the new library. diary-icalendar.el provides replacements for the diary-related features from icalendar.el; see below.

Diary

New user option 'diary-date-insertion-form'.

This user option determines how dates are inserted into the diary by Lisp functions. Its value is a pseudo-pattern of the same type as in 'diary-date-forms'. It is used by 'diary-insert-entry' when inserting entries from the calendar, or when importing them from other formats.

New library diary-icalendar.el.

This library reimplements features previously provided by `icalendar.el`: import from iCalendar format to the diary, and export from the diary to iCalendar. It also adds the ability to include iCalendar files in the diary and display and mark their contents in the calendar without importing them to the diary file. The library uses the new iCalendar library (see above) and makes diary import and export more customizable.

New commands to display more months in the calendar.

`'calendar-show-more-months'` and `'calendar-show-fewer-months'` display more or fewer months in the calendar, respectively. The calendar shows at most 12 months and at least 3 months.

'calendar-scroll-left-three-months' and its variants are obsolete aliases.

Because the calendar can display more than three months now, commands `'calendar-scroll-left-three-months'` and `'calendar-scroll-right-three-months'` have been renamed to `'calendar-scroll-calendar-left'` and `'calendar-scroll-calendar-right'`. The old names are kept as obsolete aliases.

Calc

New user option 'calc-string-maximum-character'.

Previously, the `'calc-display-strings'`, `'string'`, and `'bstring'` functions only considered integer vectors whose elements are all in the Latin-1 range 0-255. This hard-coded maximum is replaced by `'calc-string-maximum-character'`, and setting it to a higher value allows the display of matching vectors as Unicode strings. The default value is 0xFF or 255 to preserve the existing behavior.

New user option 'calc-inhibit-startup-message'.

If it is non-nil, inhibit Calc from printing its startup message. The default value is nil to preserve the existing behavior.

Time

New user option 'world-clock-sort-order'.

This option controls the order of timezone entries in the `'world-clock'`. By default, no sorting is done, and entries appear in the same order as in `'world-clock-list'`. Any format understood by `'format-time-string'` can be used to specify a key for the sort order, which is updated upon each refresh. The sort direction can be controlled by using a cons cell of a format string and a boolean. Alternatively, a sorting function can be provided directly.

New user option 'display-time-help-echo-format'.

This option controls the format of the help echo when hovering over the time.

Fill

New variable 'fill-region-as-paragraph-function'.

The new variable `'fill-region-as-paragraph-function'` provides a way to override how functions like `'fill-paragraph'` and `'fill-region'` fill text. Major modes can bind this variable to a function that fits their needs. It defaults to `'fill-region-as-paragraph-default'`.

'report-emacs-bug' now checks whether the bug report is about Org.

The command `'report-emacs-bug'` looks in the report text for symbols that indicate problems in Org, and if found, will ask whether the bug report is actually about Org (in which case users should use the Org-specific command for reporting bugs).

The `elint.el` package is now obsolete.

Use the byte-compiler instead; it provides more and more useful warnings.

Newsticker

New user option 'newsticker-hide-old-feed-header'.

It controls whether to automatically hide the header of feeds whose items are all old or obsolete in the plainview `"*newsticker*" buffer`. This is only visually interesting if the content of those feeds are also hidden (see `'newsticker-hide-old-items-in-newsticker-buffer'` and `'newsticker-show-descriptions-of-new-items'`).

New commands to hide and show headers of old newsticker feeds.

The new commands `'newsticker-hide-old-feed-header'` and `'newsticker-show-old-feed-header'`, bound to `'h h'` and `'s h'` keys, respectively, hide and show the headers of feeds whose items are all old or obsolete.

CPerl mode

Syntax of Perl up to version 5.42 is supported.

CPerl mode creates imenu entries for `":writer"` generated accessors and recognizes the new functions `"all"` and `"any"`. See <https://perldoc.perl.org/5.42.0/perldelta> for details.

Zone

Zone can scramble multiple windows across multiple frames; it may also reorganize frames to be a single window. As before, when a key or mouse event occurs, all of the frames and windows are restored to their original state. This is controlled by three new user options which control the use of frames and windows beyond the currently active ones. It identifies suitable buffers for zoning out so that potentially important buffer contents are not exposed.

New user option 'zone-delete-other-windows'.

When non-nil, the frame is made into a single full frame window to hold the zoned buffer. If all frames were to be used ('zone-all-frames' set to non-nil), then all frames are converted to single window frames.

New user option 'zone-all-frames'.

When non-nil, Zone will appear on all visible frames. While the buffer scrambling will appear on each frame, it will be the same buffer so they will all behave the same way.

New user option 'zone-all-windows-in-frame'.

When non-nil, the zoned buffer will be mapped to all of the windows present on the frame. If the option is nil, then only the selected window will show the zoned buffer. Note, however, that each window holding the zoned buffer is showing the same zoned buffer.

New variable 'zone-ignored-buffers'.

This variable is a list of criteria for excluding a buffer from consideration as the source of zoning. The list has entries that are tested against each buffer until a suitable one is found. The criteria can be a symbol that ends in '-mode' which excludes buffers that are in a mode derived from the specified mode. It may also be a function-bound symbol or lambda expression that is called with a buffer that returns a non-nil value if it should not be the Zone source. Finally, an entry can also be a regular expression that must not match the buffer's name.

Initially, the list excludes buffers in 'special-mode', in 'image-mode', contains an encrypted file, is an empty buffer, is a hidden buffer, or is the "**scratch**" buffer. If it cannot locate any acceptable buffers, it will begrudgingly use the scratch buffer.

Abbrev mode

You can now enable Abbrev mode by default using Easy Customization.

Customize the user option 'abbrev-mode' to non-nil to enable Abbrev mode by default in all buffers.

Antlr mode

Variable 'antlr-tool-version' is no user option anymore.

If becomes buffer-local and has the symbol 'antlr-v2', 'antlr-v3' or 'antlr-v4' as value. The value determines which of the tool-version dependent customization options are used. For example, the command 'antlr-run-tool' uses the option 'antlr-v3-tool-command' (with default value "java org.antlr.Tool") when 'antlr-tool-version' has the value 'antlr-v3'.

Command 'antlr-mode' now also works on ANTLR v3 or v4 grammars.

If the variable `'antlr-tool-version'` is not set locally, e.g., by the file's `local` variables specs, the command sets its local value to `'antlr-v2'` if a keyword `"class"` or `"header"` appears at the beginning of the source, or to `'antlr-v3'` otherwise.

New command `'antlr-v4-mode'` is a derived mode of `'antlr-mode'`.

It sets `'antlr-tool-version'` to value `'antlr-v4'`, and is automatically used for files with extension `".g4"`.

The variable `'antlr-language'` is now used more generally.

The variable has a symbol as value which determines which of the language-dependent customization options are used. These options influence font-locking and indentation commands. Its value is usually set according to the grammar option `"language"`, see the default values of `'antlr-v2-language-list'`, `'antlr-v3-language-list'` and `'antlr-v4-language-list'`. This mode now supports C, Delphi, JavaScript, ObjC, Python and Ruby, additional to Java and Cpp.

New user option `'antlr-run-tool-on-buffer-file'`.

Command `'antlr-run-tool'` now usually runs on the file for the current buffer. Customize this user option to nil to get the previous behavior back.

Hi Lock

Use active region for default values in more functions.

If an active region exists, the commands `'hi-lock-line-face-buffer'` and `'hi-lock-face-phrase-buffer'` now use its contents as their default value. Previously, only `'hi-lock-face-buffer'` supported this.

Shadowfile

`'shadow-info-buffer'` and `'shadow-todo-buffer'` use ephemeral buffer names now.

Display Battery mode

UPower battery status can update automatically without polling.

On systems where the user option `'battery-status-function'` is set to `'battery-upower'`, it is now possible to get battery status updates on the mode line without polling for changes every `'battery-update-interval'` seconds. Setting this user option to nil means the mode line will update only when the battery power state, percentage, or presence in the bay changes.

Etags Regen mode

Tags table is not created during completion anymore.

Previously, when there was no tags table loaded and the default completion function was called, `'etags-regen-mode'` ensured that tags were created. This has been disabled, and the new user option `'etags-regen-create-on-completion'` can be used to enable it again.

New Modes and Packages in Emacs 31.1

New major mode 'icalendar-mode'.

A major mode for displaying and editing iCalendar (RFC 5545) data. This mode handles line unfolding and fontification, including highlighting syntax errors in invalid data.

New minor mode 'delete-trailing-whitespace-mode'.

A simple buffer-local mode that runs 'delete-trailing-whitespace' before saving the buffer.

New major mode 'conf-npmrc-mode'.

A major mode based on 'conf-mode' for editing ".npmrc" files.

New major modes based on the tree-sitter library

New major mode 'mhtml-ts-mode'.

An optional major mode based on the tree-sitter library for editing HTML files. This mode handles indentation, fontification, and commenting for embedded JavaScript and CSS.

New major mode 'go-work-ts-mode'.

A major mode based on the tree-sitter library for editing "go.work" files. If tree-sitter is properly set-up by the user, it can be enabled for files named "go.work".

New package 'lua-mode'.

The 'lua-mode' package from NonGNU ELPA is now included in Emacs.

New library 'timeout'.

This library provides functions to throttle or debounce Emacs Lisp functions. This is useful for corraling overeager code that is slow and blocks Emacs, or does not provide ways to limit how often it runs.

New mode 'system-taskbar-mode'.

This is a global minor mode and companion functions that integrate Emacs with system GUI taskbars (also called docks or launchers or something similar) to display a taskbar icon "badge" overlay, a progress bar report overlay, alert the user that an Emacs session needs attention, often by flashing or bouncing the Emacs application icon. Supported capable systems are GNU/Linux via D-Bus, macOS/GNUstep 10.5+, MS-Windows 7+.

On GNU/Linux systems, shell extensions or similar helpers such as "dash-to-dock" may be required. See <<https://extensions.gnome.org/extension/307/dash-to-dock/>> and <<https://wiki.ubuntu.com/Unity/LauncherAPI>>.

New package 'system-sleep'.

This package provides platform-neutral interfaces to block your system from entering idle sleep and a hook to process pre-sleep and post-wake events. You can use this to avoid the system entering an idle sleep state and interrupting a long-running process due to lack of user activity. The sleep event hook lets you, for example, close external connections or serial ports before sleeping, and reestablish them when the system wakes up.

Supported capable systems are GNU/Linux via D-Bus (sleep blocking and sleep events require the org.freedesktop.login1 service, display sleep blocking requires org.freedesktop.Screensaver service), macOS (sleep/display blocking requires version 10.9 or later, sleep events are supported on all versions), MS-Windows (sleep blocking is supported on all versions, sleep events require Windows 8 or later).

Incompatible Lisp Changes in Emacs 31.1

Boundaries of 'cursor-sensor-functions' now obey stickiness.

'cursor-sensor-mode' now uses 'get-pos-property' to decide whether a boundary is considered as inside or outside.

This means that by default, the boundaries have changed: the end position of a stretch of a 'cursor-sensor-functions' text property used to be considered outside of the stretch whereas it is now considered as inside. You can recover the previous behavior by controlling the stickiness, for example with a call like:

```
(add-text-properties BEG END '(cursor-sensor-functions (MY-FUNCTION) rear-nonsticky
(cursor-sensor-functions)))
```

'makunbound' on a variable alias undoes the alias.

Previously, it had the effect of applying the 'makunbound' on the target of the alias (which can fail on some builtin variables).

'F00-ts-mode-indent-offset' renamed to 'F00-ts-indent-offset'.

When the new TS modes were introduced, a mistake was made where those modes used 'F00-mode-indent-offset' instead of the previous conventional 'F00-indent-offset'. The following names are the new names: 'toml-ts-indent-offset', 'mhtml-ts-js-css-indent-offset', 'html-ts-indent-offset', 'typescript-ts-indent-offset', 'php-ts-indent-offset', 'php-ts-html-indent-offset', 'json-ts-indent-offset', 'java-ts-indent-offset', 'go-ts-indent-offset', 'csharp-ts-indent-offset', 'cmake-ts-indent-offset', 'c-ts-indent-offset'.

String mutation has been restricted further.

'aset' on unibyte strings now requires the new character to be a single byte (0-255). On multibyte strings the new character and the character being replaced must both be

ASCII (0-127).

These rules ensure that mutation will never transform a unibyte string to multibyte, and that the size of a string in bytes (as reported by `'string-bytes'`) never changes. They also allow strings to be represented more efficiently in the future.

Other functions that use `'aset'` to modify string data, such as `'subst-char-in-string'` with a non-nil INPLACE argument, will signal an error if called with arguments that would violate these rules.

More program constants are combined by the compiler.

The compiler now unifies more constants that are `'equal'` for better code generation. This does not affect correct programs but may expose some coding mistakes. For example,

```
(eq (cdr '(1 2 3)) '(2 3)))
```

may return either nil or t.

Nested backquotes are not supported any more in Pcase patterns.

The obsolete variable `'redisplay-dont-pause'` has been removed.

The `'rx'` category name `'chinese-two-byte'` must now be spelled correctly.

An old alternative name (without the first `'e'`) has been removed.

`'text-property-default-nonsticky'` is now buffer-local.

This variable now becomes buffer-local when set. Use `'setq-default'` in (the unlikely) case you want to change the global value.

All the digit characters now have the `'digit'` category.

All the characters whose Unicode general-category is Nd now have the `'digit'` category, whose mnemonic is `'6'`. This includes both ASCII and non-ASCII digit characters.

All the symbol characters now have the `'symbol'` category.

All the characters that belong to the `'symbol'` script (according to `'char-script-table'`) now have the `'symbol'` category, whose mnemonic is `'5'`.

Some libraries obsolete since Emacs 24.4 and 24.5 have been removed:

`cc-compat.el`, `info-edit.el`, `meese.el`, `otodo-mode.el`, `rcompile.el`, `sup-mouse.el`, `terminal.el`, `vi.el`, `vip.el`, `ws-mode.el`, and `yow.el`.

`'if-let'` and `'when-let'` are now obsolete.

Use `'if-let*'`, `'when-let*'` and `'and-let*'` instead.

This effectively obsoletes the old `'(if-let (SYMBOL SOMETHING) ...)'` single binding syntax, which we'd kept only for backwards compatibility.

The Eshell `'pwd'` command now expands the directory name on all systems.

This ensures that user directories are properly expanded to their full name. Previously, Eshell only did this for MS-Windows systems. To restore the old behavior, you can set `'eshell-pwd-convert-function'` to `'identity'`.

The rx `'eval'` form now uses the current Elisp dialect for evaluation.

Previously, its argument was always evaluated using dynamic binding.

Unused block comment variables have been removed.

The unused variables `'block-comment-start'` and `'block-comment-end'`, which never actually had any effect when set by major modes, have been removed.

`'delete-frame'` now needs non-nil FORCE argument to delete daemon frame.

The initial terminal frame of an Emacs process running as daemon can be deleted via `'delete-frame'` if and only if its optional FORCE argument is non-nil.

`'date-to-time'` no longer accepts malformed times with time zone like "EDT".

Time strings like `"2025-06-04T13:21:00 EDT"` are not in valid ISO 8601 time format, and `'date-to-time'` now signals an error for them. Use a numerical time-zone specification, like `"2025-06-04T13:21:00-0400"`, instead, which gives the time offset as +/-hh or +/-hh:mm. A designator "Z" for UTC time is also supported. Less formal space-separated time formats, like `"2025-06-04 13:21:00 EDT"`, without the ISO 8601 "T" separator, are also supported.

The obsolete variable `'load-convert-to-unibyte'` has been removed.

The experimental variable `'binary-as-unsigned'` has been removed.

Instead of `'(let ((binary-as-unsigned t)) (format "%x" N))'` you can use `'(format "%x" (logand N MASK))'` where MASK is for the desired word size, e.g., `#x3fffffffffffffff` for typical Emacs fixnums.

The `'exec-path'` variable now uses same default PATH as other programs.

That is, if the PATH environment variable is unset or empty, `'exec-path'` now acts as if PATH is the system default, which is `"/bin:/usr/bin"` on GNU/Linux systems.

New variable `'tty-cursor-movement-use-TAB-BS'`.

The display optimization where the combination 'TAB' characters + 'BACKSPACE' is used to move to a position on a TTY frame is now disabled by default and controlled by this variable; it can be set to non-nil to keep the old behavior. This change is to accommodate screen readers.

'next-completion' and 'previous-completion' now use 'completions-format'.

Previously, these commands only took horizontal format into account; now, they call either `'{next,previous}-line-completion'` or the new commands `'{next,previous}-column-completion'`, depending on the value of `'completions-format'`. The latter two commands improve and extend the previous

implementations of `'{next,previous}-completion'`, which better reflect that they only take the (default) horizontal completions format into account. Any external code using `'{next,previous}-completion'`, that assumes the previous implementation, must be adjusted accordingly; see `'minibuffer-next-completion'` for an example of such an adjustment in Emacs core.

A thread's current buffer can now be killed.

We introduce a new attribute for threads called `"buffer disposition"`. See the new argument in `'make-thread'` as well as the `'thread-buffer-disposition'` and `'thread-set-buffer-disposition'` functions. The default value allows the thread's current buffer to be killed by another thread. This does not apply to the main thread's buffer.

Defining or modifying a face so that it inherits from itself signals error.

Calling any function that defines or modifies a face in a way that causes cyclical inheritance (i.e., the face inherits from itself, either directly or indirectly) now signals an error. Previously, Lisp programs could get away with this, and the problem would either be detected at display time or even cause Emacs to hang trying to display such a face. Affected APIs include `'defface'`, `'set-face-attribute'`, their callers, and other similar functions.

Original behavior of 'overlays-in' and 'overlays-at' has been restored.

Before Emacs 28.1, the list of overlays returned by these two functions included overlays outside of the current narrowing of the buffer, and there wasn't a special exception for including empty overlays at end of accessible portion of the buffer. This behavior has been restored, and the special behavior for empty overlays is again reserved only to the actual end of buffer, disregarding the narrowing. As result, `'remove-overlays'` can now again remove overlays outside of the narrowing, as it did before Emacs 28.1.

'help-setup-xref' now re-enables the major mode of the Help buffer.

As a result, in many cases the buffer will be read-only afterwards. This should not cause any trouble as long as the actual buffer modification takes place inside `'with-help-window'` or `'with-output-to-temp-buffer'` after the call to `'help-setup-xref'`.

Xref commands don't automatically suggest to visit a tags table anymore.

When no tags file is loaded, symbol completion just won't provide any suggestions. So the 'M-?' command now works without a tags table. And the 'M-.' command will show a message describing the several built-in options that will provide an Xref backend when used.

Calling 'debug' in batch sessions no longer kills Emacs.

If you want Emacs to exit, your program will now have to call 'kill-emacs' explicitly.

Lisp Changes in Emacs 31.1

The API to manipulate error descriptors has been improved.

There are new functions: 'error-type-p', 'error-type', 'error-has-type-p', and 'error-slot-value'. And you can now do '(signal err)' instead of '(signal (car err) (cdr err))', which is not only more concise but also preserves the 'equality' of the error descriptor.

'secure-hash' now supports generating SHA-3 message digests.

The list returned by 'secure-hash-algorithms' now contains the symbols 'sha3-224', 'sha3-256', 'sha3-384', and 'sha3-512'. These symbols can be used as the ALGORITHM argument of 'secure-hash' to generate SHA-3 hashes.

New function 'garbage-collect-heapsize'.

Same as 'garbage-collect' but just returns the info from the last GC without performing a collection.

Improve 'replace-region-contents' to accept more forms of sources.

It has been promoted from subr-x.el to the C code. You can now directly pass it a string or a buffer rather than a function. Actually passing it a function is now deprecated.

New function 'char-displayable-on-frame-p'.

'char-displayable-on-frame-p' returns non-nil if Emacs ought to be able to display its char argument on a given frame. This new function, unlike 'char-displayable-p', does not check whether the character can be encoded by the underlying terminal.

New function 'frame-initial-p'.

This predicate returns non-nil if a given frame or terminal is or holds, respectively, the initial text frame that is used internally during daemon mode, batch mode, and the early stages of startup. Interactive and graphical programs, for instance, can use this predicate to avoid operating on the initial frame, which is never displayed.

New macros 'static-when' and 'static-unless'.

Like `'static-if'`, these macros evaluate their condition at macro-expansion time and are useful for writing code that can work across different Emacs versions.

New feature to speed up repeated lookup of Lisp files in `'load-path'`.

If the new variable `'load-path-filter-function'` is set to the new function `'load-path-filter-cache-directory-files'`, calling `'load'` will cache the directories it scans and their files, and the following lookups should be faster.

`'let-alist'` supports indexing into lists.

The macro `'let-alist'` now interprets symbols containing numbers as list indices. For example, `'.key.0'` looks up `'key'` in the alist and then returns its first element.

Lexical binding

You can change the default value of `'lexical-binding'`.

While the default is still the use of dynamic binding dialect of Elisp in those places that don't explicitly set `'lexical-binding'` you can change it globally with:

```
(set-default-toplevel-value 'lexical-binding t)
```

Loading a file displays a warning if there is no `'lexical-binding'` cookie.

Files loaded from `'-x'` and `'--script'` now use lexical binding.

If you don't have time to adapt your script's code to the lexical binding dialect (see `"(elisp) Converting to Lexical Binding"`), you can wrap your code in:

```
#!/usr/bin/env -S emacs --batch --script (eval '(progn YOUR CODE HERE))
```

New function `'set-local'`.

This is the buffer-local equivalent of the function `'set'`.

New macro `'setopt-local'`.

This is the buffer-local version of `'setopt'` for user options rather than plain variables and uses `'custom-set'`/`'set-local'` to set variable values. A new argument, `BUFFER-LOCAL`, is passed to `'custom-set'` functions to indicate the buffer-local context.

New macros `'incf'` and `'decf'`.

They increment or decrement the value stored in a variable (a symbol), or in a generalized variable.

New functions `'plusp'` and `'minusp'`.

They return non-nil if a number is positive or negative, respectively, and signal an error if they are given a non-number.

New functions 'oddp' and 'evenp'.

They return non-nil if an integer is odd or even, respectively, and signal an error if they are given a non-integer.

New functions 'drop-while' and 'take-while'.

These work like 'drop' and 'take' but use a predicate instead of counting.

New function 'all' and function alias 'any'.

These return non-nil for lists where all and any elements, respectively, satisfy a given predicate.

'equal' now compares circular lists without raising an error.

Comparing very deeply nested objects will still fail, but 'equal' will no longer signal the 'circular-list' error.

The 'defvar-local' macro second argument is now optional.

This means that you can now call it with just one argument, like 'defvar', to declare a variable both special, and buffer-local.

The rx atom 'any' is obsolete and will cause warning when used.

When used as an atom, 'any' is an old alias for 'not-newline' but is often mistakenly used where 'anychar' was intended. Note that the construct '(any ...)' is unrelated and not obsolete.

ERT***Some experimental ERT macros are now considered stable.***

The following macros, previously only available in the experimental 'ert-x' module, are now considered stable and have been moved to 'ert':

- 'ert-with-test-buffer' - 'ert-with-buffer-selected' - 'ert-with-buffer-renamed'

See "(ert) Helper Functions" node in the ERT manual for more information.

New function 'ert-play-keys'.

Previously, 'ert-simulate-keys' could be used for sending keys to input functions such as 'read-from-minibuffer', but not for other interactive input such as starting key-mapped commands.

Show executed tests from erts files via the ERT results buffer.

For tests that call 'ert-test-erts-file', the ERT results buffer now allows you to list the tests defined in the referenced erts files that have been executed by the test at point. See "(ert) Running Tests Interactively" for more information.

Time & Date

'seconds-to-string' supports new formatting options.

Optional arguments are provided to produce human-readable time-duration strings in a variety of formats, for example `"6 months 3 weeks"` or `"5m 52.5s"`.

New function `'hash-table-contains-p'`.

This function returns non-nil if a given key is present in a hash table.

The function `'purecopy'` is now an obsolete alias for `'identity'`.

New function `'native-compile-directory'`.

This function natively compiles all Lisp files in a directory and in its sub-directories, recursively, which were not already natively compiled.

New function `'color-blend'`.

This function takes two RGB lists and optional ALPHA and returns an RGB list whose elements are blended in linear space proportional to ALPHA.

New function `'dom-inner-text'`.

This function gets all the text within a DOM node recursively, returning it as a concatenated string. It replaces the now-obsolete functions `'dom-text'` and `'dom-texts'`.

The `'defcustom'` `':local'` keyword can now be `'permanent-only'`.

This means that the variable's `'permanent-local'` property is set to t, without marking it as automatically buffer-local.

The obsolete face attribute `':reverse-video'` has been removed.

Use `':inverse-video'` instead.

D-Bus

Support interactive D-Bus authorization.

A new `':authorizable t'` parameter has been added to `'dbus-call-method'` and `'dbus-call-method-asynchronously'` to allow the user to interactively authorize the invoked D-Bus method (for example via polkit).

Support D-Bus file descriptor manipulation.

A new `':keep-fd'` parameter has been added to `'dbus-call-method'` and `'dbus-call-method-asynchronously'` to instruct D-Bus to keep a file descriptor, which has been returned by a method call, internally. The functions `'dbus--fd-open'`, `'dbus--fd-close'` and `'dbus--registered-fds'` implement managing these file descriptors. See the Info node `"(dbus) File Descriptors"` for details.

The customization group 'wp' has been removed.

It has been obsolete since Emacs 26.1. Use the group `'text'` instead.

New optional BUFFER argument for 'string-pixel-width'.

If supplied, `'string-pixel-width'` will use any face remappings from BUFFER when computing the string's width.

New function 'truncate-string-pixelwise'.

This function truncates a string to the specified maximum number of pixels rather than by characters, as in `'truncate-string-to-width'`, and respects face remappings if BUFFER is specified. You can also specify an optional ellipsis string to append, similar to `'truncate-string-to-width'`.

New macro 'with-work-buffer'.

This macro is similar to the already existing macro `'with-temp-buffer'`, except that it does not allocate a new temporary buffer on each call, but tries to reuse those previously allocated (up to a number defined by the new variable `'work-buffer-limit'`, which defaults to 10).

'date-to-time' now defaults to local time.

The function now assumes local time instead of Universal Time when its argument lacks explicit time zone information. This has been the de-facto behavior since Emacs 24 although documentation said otherwise. Also, the fallback on `'timezone-make-date-arpa-standard'` has been removed because its supported date styles can be handled by `'parse-time-string'`. To restore the previously documented behavior, specify `" +0000 "` or `"Z"` as the time zone in the argument.

The 'min-width' property is now supported for overlays as well.

This `'display'` property was previously supported only as text property. Now overlays can also have this property, with the same effect for the text `"covered"` by the overlay.

New function 'remove-display-text-property'.

This function removes a display property from the specified region of text, preserving any other display properties already set for that region.

New macro 'cond*'.

The new macro `'cond*` is an alternative to `'cond'` and `'pcase'`. Like them, it can be used to define several clauses, each one with its own condition; the first clause that matches will cause its body to be evaluated. `'cond*` can use Pcase's pattern matching syntax and also provides another pattern matching syntax that is different from that of `'pcase'`, which some users might find less cryptic. See the Info node `"(elisp) cond* Macro"` for details.

New function 'shell-command-do-open'.

This lets a Lisp program access the core functionality of the `'dired-do-open'` command. It opens a file or files using an external program, choosing the program according to the operating system's conventions.

'make-vtable' can create an empty vtable.

It is now possible to create a vtable without data, by leaving the `':objects'` list empty, or by providing an `':objects-function'` that (initially) produces no data. In such a case, it is necessary to provide a `':columns'` spec, so that the number of columns and their widths can be determined. Column widths can be set explicitly, or they will be calculated based on the window width.

New symbol property 'repeat-continue' for 'repeat-mode'.

A command with the `'repeat-continue'` symbol property, which can be a list of keymaps or t, will continue an already active repeating sequence for a keymap in that list (resp. all keymaps). The new property does not affect whether the command starts a repeating sequence, which remains governed by the `'repeat-map'` property. `'defvar-keymap'` supports a new keyword `':continue'`, a list of commands, and adds the keymap to the `'repeat-continue'` property of each command in that list. The `'use-package'` and `'bind-keys'` macros support a similar keyword `':continue-only'`.

New function 'completion-table-with-metadata'.

It offers a more concise way to create a completion table with metadata.

'all-completions' and 'unintern' no longer support old calling conventions.**New symbol property 'find-function-type-alist' used by 'find-function' etc.**

Macros that define an object in a way that makes the object's name and the macro call site defining the object hard to associate can add an entry to the property `'find-function-type-alist'` on the object's name to provide instructions for finding the definition.

New convenience function `'find-function-update-type-alist'` offers a concise way to update a symbol's `'find-function-type-alist'` property.

New function variable 'comment-setup-function' for multi-language modes.

It can set comment-related variables such as `'comment-start'` depending on the language under point.

'inhibit-message' can now inhibit clearing of the echo area.

Binding `'inhibit-message'` to a non-nil value will now suppress both the display of messages and the clearing of the echo area, such as caused by calling `'message'` with a nil argument.

'minibuffer-message' no longer blocks while displaying message.

'minibuffer-message' now uses a timer to clear the message printed to the minibuffer, instead of waiting with 'sit-for' and then clearing it. This makes 'minibuffer-message' usable in Lisp programs which want to

print a message and then continue to perform work.

Special Events

New primitive 'insert-special-event'.

This function inserts the special EVENT into the input event queue.

New event type 'sleep-event'.

This event is sent when the device running Emacs enters or leaves the sleep state.

Function aliases obsolete since Emacs 23.2 have been removed:

'advertised-undo', 'advertised-widget-backward', and 'dired-advertised-find-file'.

New functions to get and set top-level buffer-local values.

'buffer-local-toplevel-value' and 'set-buffer-local-toplevel-value' get and set the top-level buffer-local value of a variable. A top-level value is the one that variable has outside of any let-bindings.

New function 'exec-suffixes'.

This function by default returns the value of the corresponding user option, but can optionally return the equivalent of 'exec-suffixes' from a remote host. It must be used in conjunction with the function 'exec-path'.

'read-directory-name' now accepts an optional PREDICATE argument.

JSON parse error line and column are now obsolete.

The column number is no longer available; the line number will be removed in next Emacs release.

'defvar-keymap' can now take a ':prefix t' option.

This is an abbreviation for using the name of the keymap as the prefix command name. E.g., '(defvar-keymap foo-map :prefix t)' is equivalent to '(defvar-keymap foo-map :prefix 'foo-map)'.

New 'R' code letter for 'interactive' forms.

This specifies the beginning and end of an active region, and nil twice if the region is inactive. The interactive specification

(interactive "R")

is equivalent to

```
(interactive (list (use-region-beginning) (use-region-end)))
```

Toolkit

The Emacs PGTK toolkit respects dark and light modes.

Emacs when built with the pure GTK toolkit now respects desktop dark and light modes for drawing a GTK toolbar and widgets, automatically toggling between them.

'toolkit-theme-set-functions' called when the toolkit theme is set for Emacs.

When the theme is set on PGTK, Android, or MS-Windows systems,

'toolkit-theme-set-functions' is called. The result is stored in the variable 'toolkit-theme' as either symbol 'dark' or 'light', but may be extended to encompass other toolkit-specific symbols in the future.

Progress reporter context.

'make-progress-reporter' now accepts the optional argument CONTEXT, which if it is the symbol 'async', inhibits updates in the echo area when it is busy. This is useful, for example, if you want to monitor progress of an inherently asynchronous command such as 'compile'.

Binary format specifications '%b' and '%B' added.

These produce the binary representation of a number. '%#b' and '%#B' prefix the bits with '0b' and '0B', respectively.

'pp-eval-expression' can now insert results into the current buffer.

With a prefix argument, 'pp-eval-expression' inserts the result into the current buffer, just like 'eval-expression' already did.

New function 'multiple-command-partition-arguments'.

This function partitions a list of command arguments that might be arbitrarily long. It can be used in cases in which it is known to be safe to run the command multiple times on subsequent partitions of the list of arguments. The variable 'command-line-max-length' controls the partitioning.

New function 'ensure-proper-list'.

This function is a variation on 'ensure-list' that checks if an object is a proper list, in which case the list will be returned as is, otherwise the function will return the object wrapped in a singleton list.

In batch mode, 'C-c' (i.e. SIGINT) can either 'quit' or kill Emacs.

By default it kills Emacs, as before, but 'kill-emacs-on-sigint' can be set to nil to change that. The response to SIGINT in interactive sessions is unaffected, e.g., in a

normal GUI session it still kills Emacs whereas in a terminal it causes `'quit'` since it is used for `'C-g'`.

New `:interactive-only` way to add an advice.

While it is marginally more efficient than `:after` or `:before`, the main purpose is to make the intention more obvious when the advice modifies only the interactive form and not the actual behavior of the function.

Changes in Emacs 31.1 on Non-Free Operating Systems

Support macOS Accessibility Zoom focus tracking.

This is an important change for visually-impaired users. If macOS Accessibility Zoom is enabled via (System Settings, Accessibility, Zoom...) with keyboard focus tracking (Advanced...), Zoom is informed of updated cursor positions during each redisplay cycle.

New macOS function `'ns-process-is-accessibility-trusted'`.

This function returns `t` if the macOS Accessibility Framework trusts Emacs. This is a necessary condition for Accessibility Zoom and other accessibility features. Enable Emacs via (System Settings, Privacy & Security, Accessibility...) and add the Emacs.app installed directory to the enabled application list.

Process execution has been optimized on Android.

The run-time performance of subprocesses on recent Android releases, where a userspace executable loader is required, has been optimized on systems featuring Linux 3.5.0 and above.

It is now possible to read GUI events from non-main Lisp threads on Android.

Put differently, this enables input events to be read and recursive editing sessions to be started from non-main threads. The only platform where this remains unsupported is Nextstep (GNUstep or Mac OS).

`'desktop-restore-frames'` has been disabled by default on Android systems.

Restrictions imposed on clients by the window manager on these systems are too prohibitive and don't allow restoring frame configurations. (For the same reason many window management facilities are also not implemented by Emacs.)

Emacs responds to runtime display configuration changes on Android.

The upshot of this is that Emacs will adapt to display resolution / layout changes applied while an Emacs session is active, which is possible on some recently released devices.

`'NSSpeechRecognitionUsageDescription'` now included in `"Info.plist"` (macOS).

Should Emacs (or any built-in shell) invoke a process using macOS speech recognition APIs, the relevant permission dialog is now displayed, thus allowing Emacs users access to speech recognition utilities.

Note: Accepting this permission allows the use of system APIs, which may send user data to Apple's speech recognition servers.

Re-introduced dictation, lost in Emacs 30 (macOS).

We lost macOS dictation in Emacs 30 when migrating to NSTextInputClient. Implemented `'selectedRange'` in `'nsterm.m'` to enable in new subsystem. You may notice a slight change in dictation UI provided by macOS.

On Mac OS X, stipples now render with color.

Emacs on MS-Windows now supports GUI dialogs and message boxes better.

In particular, it is now possible to show text with embedded newlines in a dialog popped by `'message-box'`. This is supported on Windows Vista and later versions.

Emacs on MS-Windows now supports drag-n-drop of text into a buffer.

This is in addition to drag-n-drop of files, that was already supported. As on X, the user options `'dnd-scroll-margin'` and `'dnd-indicate-insertion-point'` can be used to customize the process.

Emacs on MS-Windows now supports color fonts.

On Windows 8.1 and later versions, Emacs now uses DirectWrite to draw text, which supports color fonts. This can be disabled by setting the variable `'w32-inhibit-dwrite'` to t. Also see `'w32-dwrite-available'` and `'w32-dwrite-reinit'` to check availability and to configure the DirectWrite rendering parameters.

To show color Emojis in Emacs, customize the default fontset to use a color Emoji font installed on your system for the `'emoji'` script.

Emacs on MS-Windows now supports `'yank-media'`.

This command inserts clipboard data of different formats into the current buffer, if the major mode supports it.

Emacs on MS-Windows now supports up to 1024 sub-processes.

Changes in implementation of monitoring sub-processes allow Emacs on MS-Windows to start up to 1024 sub-processes, similar to GNU/Linux and other free systems.

Images on MS-Windows now support the `':transform-smoothing'` flag.

Transformed images are smoothed using the bilinear interpolation by means of the GDI+ library.

Emacs on MS-Windows is now capable of exporting frame screenshots to files.

The new primitive `'w32-export-frame'` can be used to export a screenshot of a specified frame to an image file in one of the supported image formats, such as JPEG or PNG.

Emacs on MS-Windows now supports the `':data'` keyword for `'play-sound'`.

In addition to `':file FILE'` for playing a sound from a file, `':data DATA'` can now be used to play a sound from memory.

New primitive `'w32-sound-volume'`.

This primitive allows getting and setting the volume of the system's default audio device (or the `"optimal device"`, if there are several devices).

The MS-DOS port of Emacs now supports more recent releases of GCC and Binutils.

Accordingly, we have revised our recommendations for a suitable DJGPP toolchain to GCC 14.2.0 and Binutils 2.35.1 in lieu of GCC 3.4.x and Binutils 2.26.

Windows Terminal can now display 256 and 24-bit RGB color.

Previously, terminal sessions on Windows supported display of 16 colors. There is now support for 8-bit (256 color) and 24-bit RGB (true color). The new mechanism will be enabled automatically when supported. It defaults to 24-bit RGB color, but can be set to 8, 16, `'8bit'`, or `'24bit'` by passing the `'--color'` flag or setting the `'tty-color-mode'` frame parameter. Use of the new mechanism is controlled by an internal variable that can be set and inspected via the functions `'w32-use-virtual-terminal'` and `'w32-use-virtual-terminal-p'` (respectively). See the manual entry `"(emacs) Windows Misc"` for more details.

This file is part of GNU Emacs.

GNU Emacs is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

GNU Emacs is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with GNU Emacs. If not, see [<https://www.gnu.org/licenses/>](https://www.gnu.org/licenses/).

Local variables: coding: utf-8 mode: outline mode: emacs-news paragraph-separate: "[]"
end: